

CHAPTER 1

Software Engineering Principles and C++ Classes

IN THIS CHAPTER, YOU WILL:

- Learn about software engineering principles
- Discover what an algorithm is and explore problem-solving techniques
- Become aware of structured design and object-oriented design programming methodologies
- Learn about classes
- Become aware of **`private`**, **`protected`**, and **`public`** members of a class
- Explore how classes are implemented
- Become aware of Unified Modeling Language (UML) notation
- Examine constructors and destructors
- Become aware of abstract data type (ADT)
- Explore how classes are used to implement ADT

Most everyone working with computers is familiar with the term *software*. Software are computer programs designed to accomplish a specific task. For example, word processing software is a program that enables you to write term papers, create impressive-looking résumés, and even write a book. This book, for example, was created with the help of a word processor. Students no longer type their papers on typewriters or write them by hand. Instead, they use word processing software to complete their term papers. Many people maintain and balance their checkbooks on computers.

Powerful, yet easy-to-use software has drastically changed the way we live and communicate. Terms such as *the Internet*, which was unfamiliar just a decade ago, are very common today. With the help of computers and the software running on them, you can send letters to, and receive letters from, loved ones within seconds. You no longer need to send a résumé by mail to apply for a job; in many cases, you can simply submit your job application via the Internet. You can watch how stocks perform in real time, and instantly buy and sell them.

Without software a computer is of no use. It is the software that enables you to do things that were, perhaps, fiction a few years ago. However, software is not created overnight. From the time a software program is conceived until it is delivered, it goes through several phases. There is a branch of computer science, called software engineering, which specializes in this area. Most colleges and universities offer a course in software engineering. This book is not concerned with the teaching of software engineering principles. However, this chapter briefly describes some of the basic software engineering principles that can simplify program design.

Software Life Cycle

A program goes through many phases from the time it is first conceived until the time it is retired, called the *life cycle* of the program. The three fundamental stages through which a program goes are *development*, *use*, and *maintenance*. Usually a program is initially conceived by a software developer because a customer has some problem that needs to be solved and the customer is willing to pay money to have it solved. The new program is created in the *software development* stage. The next section describes this stage in some detail.

Once the program is considered complete, it is released for the user to use. Once users start using the program, they most certainly discover problems or have suggestions to improve it. The problems and/or ideas for improvements are conveyed to the software developer, and the program goes through the maintenance phase.

In the *software maintenance* process, the program is modified to fix the (identified) problems and/or to enhance it. If there are serious/numerous changes, typically, a new version of the program is created and released for use.

When a program is considered too expensive to maintain, the developer might decide to *retire* the program and no new version of the program will be released.

The software development phase is the first and perhaps most important phase of the software life cycle. A program that is well developed will be easy and less expensive to maintain. The next section describes this phase.

Software Development Phase

Software engineers typically break the software development process into the following four phases:

- Analysis
- Design
- Implementation
- Testing and debugging

The next few sections describe these four phases in some detail.

Analysis

Analyzing the problem is the first and most important step. This step requires you to do the following:

- Thoroughly understand the problem.
- Understand the problem requirements. Requirements can include whether the program requires interaction with the user, whether it manipulates data, whether it produces output, and what the output looks like.

  Suppose that you need to develop a program to make an automated teller machine (ATM) operational. In the analysis phase, you determine the functionality of the machine. Here, you determine the necessary operations performed by the machine, such as withdraw money, deposit money, transfer money, check account balance, and so on. During this phase, you also talk to potential customers who would use the machine. To make it user-friendly, you must understand their requirements and add any necessary operations.

  If the program manipulates data, the programmer must know what the data is and how it is represented. That is, you need to look at sample data. If the program produces output, you should know how the results should be generated and formatted.
- If the problem is complex, divide the problem into subproblems, analyze each subproblem, and understand each subproblem's requirements.

Design

After you carefully analyze the problem, the next step is to design an algorithm to solve the problem. If you broke the problem into subproblems, you need to design an algorithm for each subproblem.

Algorithm: A step-by-step problem-solving process in which a solution is arrived at in a finite amount of time.

STRUCTURED DESIGN

Dividing a problem into smaller subproblems is called **structured design**. The structured design approach is also known as **top-down design**, **stepwise refinement**, and **modular programming**. In structured design, the problem is divided into smaller problems. Each subproblem is then analyzed, and a solution is obtained to solve the subproblem. The solutions of all the subproblems are then combined to solve the overall problem. This process of implementing a structured design is called **structured programming**.

OBJECT-ORIENTED DESIGN

In object-oriented design (OOD), the first step in the problem-solving process is to identify the components called objects, which form the basis of the solution, and determine how these objects interact with one another. For example, suppose you want to write a program that automates the video rental process for a local video store. The two main objects in this problem are the video and the customer.

After identifying the objects, the next step is to specify for each object the relevant data and possible operations to be performed on that data. For example, for a video object, the data might include the movie name, starring actors, producer, production company, number of copies in stock, and so on. Some of the operations on a video object might include checking the name of the movie, reducing the number of copies in stock by one after a copy is rented, and incrementing the number of copies in stock by one after a customer returns a particular video.

This illustrates that each object consists of data and operations on that data. An object combines data and operations on the data into a single unit. In OOD, the final program is a collection of interacting objects. A programming language that implements OOD is called an **object-oriented programming (OOP)** language. You will learn about the many advantages of OOD in later chapters.

OOD has the following three basic principles:

- **Encapsulation**—The ability to combine data and operations in a single unit
- **Inheritance**—The ability to create new (data) types from existing (data) types
- **Polymorphism**—The ability to use the same expression to denote different operations

In C++, encapsulation is accomplished via the use of data types called classes. How classes are implemented in C++ is described later in this chapter. Chapter 2 discusses inheritance and polymorphism.

In object-oriented design, you decide what classes you need and their relevant data members and member functions. You then describe how classes interact with each other.

Implementation

In the *implementation* phase, you write and compile programming code to implement the classes and functions that were discovered in the design phase.

This book uses the OOD technique (in conjunction with structured programming) to solve a particular problem. It contains many case studies—called Programming Examples—to solve real-world problems.

The final program consists of several functions, each accomplishing a specific goal. Some functions are part of the main program; others are used to implement various operations on objects. Clearly, functions interact with each other, taking advantage of each other's capabilities. To use a function, the user needs to know only how to use the function and what the function does. The user should not be concerned with the details of the function, that is, how the function is written. Let us illustrate this with the help of the following example.

Suppose that you want to write a function that converts a measurement given in inches into equivalent centimeters. The conversion formula is 1 inch = 2.54 centimeters. The following function accomplishes the job:

```
double inchesToCentimeters(double inches)
{
    if (inches < 0)
    {
        cerr << "The given measurement must be nonnegative." << endl;
        return -1.0;
    }
    else
        return 2.54 * inches;
}
```

NOTE

The object `cerr` corresponds to the unbuffered standard error stream. Unlike the object `cout` (whose output first goes to the buffer), the output of `cerr` is immediately sent to the standard error stream, which is usually the screen.

If you look at the body of the function, you can recognize that if the value of inches is less than 0, that is, negative, the function returns –1.0; otherwise, the function returns the equivalent length in centimeters. The user of this function does not need to know the specific details of how the algorithm that finds the equivalent length in centimeters is implemented. However, the user must know that in order to get the valid answer, the input must be a nonnegative number. If the input to this function is a negative number, the program returns –1.0. This information can be provided as part of the documentation of this function using specific statements, called preconditions and postconditions.

Precondition: A statement specifying the condition(s) that must be true before the function is called.

Postcondition: A statement specifying what is true after the function call is completed.

The precondition and postcondition for the function `inchesToCentimeters` can be specified as follows:

```
  //Precondition: The value of inches must be nonnegative.
  //Postcondition: If the value of inches is < 0, the function
  //    returns -1.0; otherwise, the function returns the
  //    equivalent length in centimeters.
double inchesToCentimeters(double inches)
{
    if (inches < 0)
    {
        cerr << "The given measurement must be nonnegative." << endl;
        return -1.0;
    }
    else
        return 2.54 * inches;
}
```

In certain situations, you could use C++'s `assert` statement to validate the input. For example, the preceding function can be written as follows:

```
  //Precondition: The value of inches must be nonnegative.
  //Postcondition: If the value of inches is < 0, the function
  //    terminates; otherwise, the function returns the
  //    equivalent length in centimeters.
double inchesToCentimeters(double inches)
{
      assert(inches >= 0);
      return 2.54 * inches;
}
```

However, if the `assert` statement fails, the entire program will terminate, which might be appropriate if the remainder of the program depends on the execution of the function. On the other hand, the user can check the value returned by the function, determine if the returned value is appropriate, and proceed accordingly. To use the `assert` function, you need to include the header file `cassert` in your program.

NOTE

To turn off the `assert` statements in a program, use the preprocessor directive `#define NDEBUG`. This directive must be placed before the statement `#include <cassert>`.

As you can see, the same function can be implemented differently by different programmers. Because the user of a function need not be concerned with the details of the function, the preconditions and postconditions are specified with the function prototype. That is, the user is given the following information:

```
double inchesToCentimeters(double inches);
  //Precondition: The value of inches must be nonnegative.
  //Postcondition: If the value of inches is < 0, the function
  //    returns -1.0; otherwise, the function returns the
  //    equivalent length in centimeters.
```

As another example, to use a function that searches a list for a specific item, the list must exist before the function is called. After the search is complete, the function returns `true` or `false` depending on whether the search was successful.

```
bool search(int list[], int listLength, int searchItem);
  //Precondition: The list must exist.
  //Postcondition: The function returns true if searchItem is in
  //    list; otherwise, the function return false.
```

Testing and Debugging

The term *testing* refers to testing the correctness of the program; that is, making sure that the program does what it is supposed to do. The term *debugging* refers to finding and fixing the errors, if they exist.

Once a function and/or an algorithm is written, the next step is to verify that it works properly. However, in a large and complex program, errors almost certainly exist. Therefore, to increase the reliability of the program, errors must be discovered and fixed before the program is released to the user.

You can certainly prove this by using some (perhaps mathematical) analysis of the correctness of a program. However, for large and complex programs, this technique alone might not be enough because errors can be made in the proof. Therefore, we also rely on testing to determine the quality of the program. The program is run through a series of specific tests, called test cases, in an attempt to find problems.

A test case consists of a set of inputs, user actions, or other initial conditions, and the expected output. Because a test case can be repeated several times, it must be properly documented. Typically a program manipulates a large set of data. It is, therefore, impractical (although possible) to create test cases for all possible inputs. For example, suppose that a program manipulates integers. Clearly, it is not possible to create a test case for each integer. You can categorize test cases into separate categories, called equivalence categories. An equivalence category is a set of input values that are likely to produce the same output. For example, suppose that you have a function that takes an integer as input and returns `true` if the integer is nonnegative, and `false` otherwise. In this case, you can form two equivalence categories—one consisting of negative numbers and the other consisting of nonnegative numbers.

There are two types of testing—*black-box* testing and *white-box* testing. In black-box testing, you do not know the internal working of the algorithm or function. You know only what the function does. Black-box testing is based on inputs and outputs. The test cases for black-box testing are usually selected by creating equivalence

categories. If a function works for one input in the equivalence category, it is expected to work for other inputs in the same category.

Suppose that the function `isWithInRange` returns a value `true` if an integer is greater than or equal to 0 and less than or equal to 100. In black-box testing, the function is tested on values that surround and fall on the boundaries, called **boundary values**, as well as general values from the equivalence categories. For the function `isWithInRange`, in black-box testing, the boundary values might be: -1, 0, 1, 99, 100, and 101; and so the test values might be -500, -1, 0, 1, 50, 99, 100, 101, and 500.

White-box testing relies on the internal structure and implementation of a function or algorithm. The objective is to ensure that every part of the function or algorithm is executed at least once. Suppose that you want to ensure whether an `if` statement works properly. The test cases must consist of at least one input for which the `if` statement evaluates to `true` and at least one case for which it evaluates to `false`. Loops and other structures can be tested similarly.

Algorithm Analysis: The Big-O Notation

Just as a problem is analyzed before writing the algorithm and the computer program, after an algorithm is designed it should also be analyzed. Usually, there are various ways to design a particular algorithm. Certain algorithms take very little computer time to execute, whereas others take a considerable amount of time.

Let us consider the following problem. The holiday season is approaching and a gift shop is expecting sales to be double or even triple the regular amount. They have hired extra delivery people to deliver the packages on time. The company calculates the shortest distance from the shop to a particular destination and hands the route to the driver. Suppose that 50 packages are to be delivered to 50 different houses. The shop, while making the route, finds that the 50 houses are one mile apart and are in the same area. (See Figure 1-1, in which each dot represents a house and the distance between houses is 1 mile.)

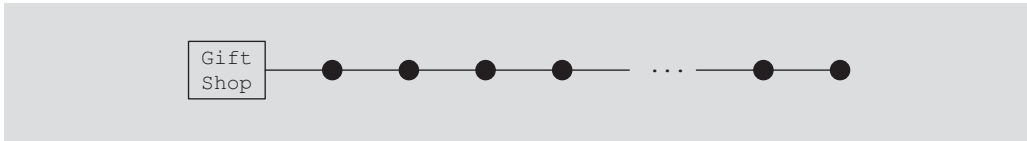


FIGURE 1-1 Gift shop and each dot representing a house

To deliver 50 packages to their destinations, one of the drivers picks up all 50 packages, drives one mile to the first house and delivers the first package. Then he drives another mile and delivers the second package, drives another mile and delivers the third package, and so on. Figure 1-2 illustrates this delivery scheme.

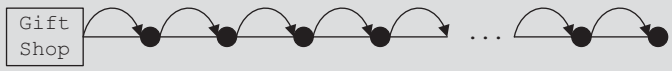


FIGURE 1-2 Package delivering scheme

It now follows that using this scheme, the distance driven by the driver to deliver the packages is:

$$1 + 1 + 1 + ... + 1 = 50 \text{ miles}$$

Therefore, the total distance traveled by the driver to deliver the packages and then getting back to the shop is:

$$50 + 50 = 100 \text{ miles}$$

Another driver has a similar route to deliver another set of 50 packages. The driver looks at the route and delivers the packages as follows: The driver picks up the first package, drives one mile to the first house, delivers the package, and then comes back to the shop. Next, the driver picks up the second package, drives 2 miles, delivers the second package, and then returns to the shop. The driver then picks up the third package, drives 3 miles, delivers the package, and comes back to the shop. Figure 1-3 illustrates this delivery scheme.

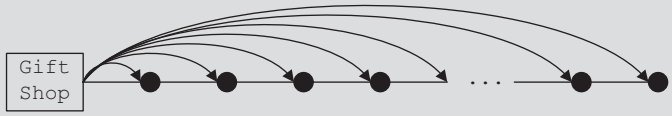


FIGURE 1-3 Another package delivery scheme

The driver delivers only one package at a time. After delivering a package, the driver comes back to the shop to pick up and deliver the second package. Using this scheme, the total distance traveled by this driver to deliver the packages and then getting back to the store is:

$$2 \cdot (1 + 2 + 3 + \ldots + 50) = 2550 \text{ miles}$$

Now suppose that there are n packages to be delivered to n houses, and each house is one mile apart from each other, as shown in Figure 1-1. If the packages are delivered using the first scheme, the following equation gives the total distance traveled:

$$1 + 1 + \ldots + 1 + n = 2n \tag{1–1}$$

If the packages are delivered using the second method, the distance traveled is:

$$2 \cdot (1 + 2 + 3 + \ldots + n) = 2 \cdot (n(n + 1)/2) = n^2 + n \tag{1–2}$$

In Equation (1-1), we say that the distance traveled is a function of n. Let us consider Equation (1-2). In this equation, for large values of n, we will find that the term consisting of n^2 will become the dominant term and the term containing n will be negligible. In this case, we say that the distance traveled is a function of n^2. Table 1-1 evaluates Equations (1-1) and (1-2) for certain values of n. (The table also shows the value of n^2.)

TABLE 1-1 Various values of n, $2n$, n^2, and $n^2 + n$

n	$2n$	n^2	$n^2 + n$
1	2	1	2
10	20	100	110
100	200	10,000	10,100
1000	2000	1,000,000	1,001,000
10,000	20,000	100,000,000	100,010,000

While analyzing a particular algorithm, we usually count the number of operations performed by the algorithm. We focus on the number of operations, not on the actual computer time to execute the algorithm. This is because a particular algorithm can be implemented on a variety of computers and the speed of the computer can affect the execution time. However, the number of operations performed by the algorithm would be the same on each computer. Let us consider the following examples.

EXAMPLE 1-1

Consider the following algorithm. (Assume that all variables are properly declared.)

```
cout << "Enter two numbers";                          //Line 1

cin >> num1 >> num2;                                  //Line 2

if (num1 >= num2)                                     //Line 3
    max = num1;                                       //Line 4
else                                                  //Line 5
    max = num2;                                       //Line 6

cout << "The maximum number is: " << max << endl;     //Line 7
```

Line 1 has one operation, `<<`; Line 2 has two operations; Line 3 has one operation, `>=`; Line 4 has one operation, `=`; Line 6 has one operation; and Line 7 has three operations. Either Line 4 or Line 6 executes. Therefore, the total number of operations executed in the preceding code is 1 + 2 + 1 + 1 + 3 = 8. In this algorithm, the number of operations executed is fixed.

EXAMPLE 1-2

Consider the following algorithm:

```
cout << "Enter positive integers ending with -1" << endl;  //Line 1

count = 0;                                                  //Line 2
sum = 0;                                                    //Line 3

cin >> num;                                                 //Line 4

while (num != -1)                                           //Line 5
{
    sum = sum + num;                                        //Line 6
    count++;                                                //Line 7
    cin >> num;                                             //Line 8
}

cout << "The sum of the numbers is: " << sum << endl;       //Line 9

if (count != 0)                                             //Line 10
    average = sum / count;                                  //Line 11
else                                                        //Line 12
    average = 0;                                            //Line 13

cout << "The average is: " << average << endl;              //Line 14
```

This algorithm has five operations (Lines 1 through 4) before the `while` loop. Similarly, there are nine or eight operations after the `while` loop, depending on whether Line 11 or Line 13 executes.

Line 5 has one operation, and four operations within the `while` loop (Lines 6 through 8). Thus, Lines 5 through 8 have five operations. If the `while` loop executes 10 times, these five operations execute 10 times. One extra operation is also executed at Line 5 to terminate the loop. Therefore, the number of operations executed is 51 from Lines 5 through 8.

If the `while` loop executes 10 times, the total number of operations executed is:

$10 \cdot 5 + 1 + 5 + 9$ or $10 \cdot 5 + 1 + 5 + 8$

that is,

$10 \cdot 5 + 15$ or $10 \cdot 5 + 14$

We can generalize it to the case when the `while` loop executes n times. If the `while` loop executes n times, the number of operations executed is:

$5n + 15$ or $5n + 14$

In these expressions, for very large values of n, the term $5n$ becomes the dominating term and the terms 15 and 14 become negligible.

Usually, in an algorithm, certain operations are dominant. For example, in the preceding algorithm, to add numbers, the dominant operation is in Line 6. Similarly, in a search algorithm, because the search item is compared with the items in the list, the dominant operations would be comparison, that is, the relational operation. Therefore, in the case of a search algorithm, we count the number of comparisons. For another example, suppose that we write a program to multiply matrices. The multiplication of matrices involves addition and multiplication. Because multiplication takes more computer time to execute, to analyze a matrix multiplication algorithm, we count the number of multiplications.

In addition to developing algorithms, we also provide a reasonable analysis of each algorithm. If there are various algorithms to accomplish a particular task, the algorithm analysis allows the programmer to choose between various options.

Suppose that an algorithm performs $f(n)$ basic operations to accomplish a task, where n is the size of the problem. Suppose that you want to determine whether an item is in a list. Moreover, suppose that the size of the list is n. To determine whether the item is in the list, there are various algorithms, as you will see in Chapter 9. However, the basic method is to compare the item with the items in the list. Therefore, the performance of the algorithm depends on the number of comparisons.

Thus, in the case of a search, n is the size of the list and $f(n)$ becomes the count function, that is, $f(n)$ gives the number of comparisons done by the search algorithm. Suppose that, on a particular computer, it takes c units of computer time to execute one operation. Thus, the computer time it would take to execute $f(n)$ operations is $cf(n)$. Clearly, the constant c depends on the speed of the computer and, therefore, varies from computer to computer. However, $f(n)$, the number of basic operations, is the same on each computer. If we know how the function $f(n)$ grows as the size of the problem grows, we can determine the efficiency of the algorithm. Consider Table 1-2.

TABLE 1-2 Growth rates of various functions

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
1	0	0	1	2
2	1	2	2	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1024	4,294,967,296

Table 1-2 shows how certain functions grow as the parameter n, that is, the problem size, grows. Suppose that the problem size is doubled. From Table 1-2, it follows that if the number of basic operations is a function of $f(n) = n^2$, the number of basic operations is quadrupled. If the number of basic operations is a function of $f(n) = 2^n$, the number of basic operations is squared. However, if the number of operations is a function of $f(n) = \log_2 n$, the change in the number of basic operations is insignificant.

Suppose that a computer can execute 1 billion basic operations per second. Table 1-3 shows the time that the computer takes to execute $f(n)$ basic operations.

TABLE 1-3 Time for $f(n)$ instructions on a computer that executes 1 billion instructions per second

n	$f(n) = n$	$f(n) = \log_2 n$	$f(n) = n\log_2 n$	$f(n) = n^2$	$f(n) = 2^n$
10	0.01μs	0.003μs	0.033μs	0.1μs	1μs
20	0.02μs	0.004μs	0.086μs	0.4μs	1ms
30	0.03μs	0.005μs	0.147μs	0.9μs	1s
40	0.04μs	0.005μs	0.213μs	1.6μs	18.3min
50	0.05μs	0.006μs	0.282μs	2.5μs	13 days
100	0.10μs	0.007μs	0.664μs	10μs	4×10^{13} years
1000	1.00μs	0.010μs	9.966μs	1ms	
10,000	10μs	0.013μs	130μs	100ms	
100,000	0.10ms	0.017μs	1.67ms	10s	
1,000,000	1 ms	0.020μs	19.93ms	16.7m	
10,000,000	0.01s	0.023μs	0.23s	1.16 days	
100,000,000	0.10s	0.027μs	2.66s	115.7 days	

In Table 1-3, 1μs = 10^{-6} seconds and 1ms = 10^{-3} seconds.

Figure 1-4 shows the growth rate of functions in Table 1-3.

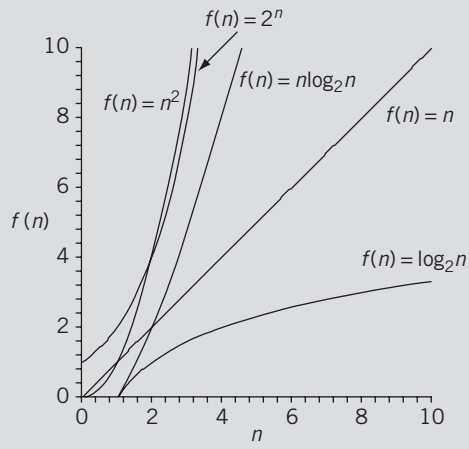


FIGURE 1-4 Growth rate of various functions

The remainder of this section develops a notation that shows how a function $f(n)$ grows as n increases without bound. That is, we develop a notation that is useful in describing the behavior of the algorithm, which gives us the most useful information about the algorithm. First, we define the term *asymptotic*.

Let f be a function of n. By the term **asymptotic**, we mean the study of the function f as n becomes larger and larger without bound.

Consider the functions $g(n) = n^2$ and $f(n) = n^2 + 4n + 20$. Clearly, the function g does not contain any linear term, that is, the coefficient of n in g is zero. Consider Table 1-4.

TABLE 1-4 Growth rate of n^2 and $n^2 + 4n + 20$

n	$g(n) = n^2$	$f(n) = n^2 + 4n + 20$
10	100	160
50	2500	2720
100	10,000	10,420
1000	1,000,000	1,004,020
10,000	100,000,000	100,040,020

Clearly, as n becomes larger and larger the term $4n + 20$ in $f(n)$ becomes insignificant, and the term n^2 becomes the dominant term. For large values of n, we can predict the behavior of $f(n)$ by looking at the behavior of $g(n)$. In algorithm analysis, if the complexity of a function can be described by the complexity of a quadratic function without the linear term, we say that the function is of $O(n^2)$, called Big-O of n^2.

Let f and g be real-valued functions. Assume that f and g are nonnegative, that is, for all real numbers n, $f(n) \geq 0$ and $g(n) \geq 0$.

Definition: We say that $f(n)$ is **Big-O** of $g(n)$, written $f(n) = O(g(n))$, if there exists positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

EXAMPLE 1-3

Let $f(n) = a$, where a is a nonnegative real number and $n \geq 0$. Note that f is a constant function. Now

$f(n) = a \leq a \cdot 1$ for all $n \geq a$.

Let $c = a$, $n_0 = a$, and $g(n) = 1$. Then $f(n) \leq cg(n)$ for all $n \geq n_0$. It now follows that $f(n) = O(g(n)) = O(1)$.

From Example 1-3, it follows that if f is a nonnegative constant function, then f is $O(1)$.

EXAMPLE 1-4

Let $f(n) = 2n + 5$, $n \geq 0$. Note that

$f(n) = 2n + 5 \leq 2n + n = 3n$ for all $n \geq 5$.

Let $c = 3$, $n_0 = 5$, and $g(n) = n$. Then $f(n) \leq cg(n)$ for all $n \geq 5$. It now follows that $f(n) = O(g(n)) = O(n)$.

EXAMPLE 1-5

Let $f(n) = n^2 + 3n + 2$, $g(n) = n^2$, $n \geq 0$. Note that $3n + 2 \leq n^2$ for all $n \geq 4$. This implies that

$f(n) = n^2 + 3n + 2 \leq n^2 + n^2 \leq 2n^2 = 2g(n)$ for all $n \geq 4$.

Let $c = 2$ and $n_0 = 4$. Then $f(n) \leq cg(n)$ for all $n \geq 4$. It now follows that $f(n) = O(g(n)) = O(n^2)$.

In general, we can prove the following theorem. Here we state the theorem without proof.

Theorem: Let $f(n)$ be a nonnegative real-valued function such that

$f(n) = a_m n^m + a_{m-1} n^{m-1} + \cdots + a_1 n + a_0$,

where a_i's are real numbers, $a_m \neq 0$, $n \geq 0$, and m is a nonnegative integer. Then $f(n) = O(n^m)$.

In Example 1-6, we use the preceding theorem to establish the Big-O of certain functions.

EXAMPLE 1-6

In the following, $f(n)$ is a nonnegative real-valued function.

Function	Big-O
$f(n) = an + b$, where a and b are real numbers and a is nonzero.	$f(n) = O(n)$
$f(n) = n^2 + 5n + 1$	$f(n) = O(n^2)$
$f(n) = 4n^6 + 3n^3 + 1$	$f(n) = O(n^6)$
$f(n) = 10n^7 + 23$	$f(n) = O(n^7)$
$f(n) = 6n^{15}$	$f(n) = O(n^{15})$

EXAMPLE 1-7

Suppose that $f(n) = 2\log_2 n + a$, where a is a real number. It can be shown that $f(n) = O(\log_2 n)$.

EXAMPLE 1-8

Consider the following code, where `m` and `n` are `int` variables and their values are nonnegative:

```
for (int i = 0; i < m; i++)          //Line 1
    for (int j = 0; j < n; j++)      //Line 2
        cout << i * j << endl;       //Line 3
```

This code contains nested `for` loops. The outer `for` loop, at Line 1, executes m times. For each iteration of the outer loop, the inner loop, at Line 2, executes n times. For each iteration of the inner loop, the output statement in Line 3 executes. It follows that the total number of iterations of the nested `for` loop is mn. So the number of times the statement in Line 3 executes is mn. Therefore, this algorithm is $O(mn)$. Note that if $m = n$, then this algorithm is $O(n^2)$.

Table 1-5 shows some common Big-O functions that appear in the algorithm analysis. Let $f(n) = O(g(n))$ where n is the problem size.

TABLE 1-5 Some Big-O functions that appear in algorithm analysis

Function $g(n)$	Growth rate of $f(n)$
$g(n) = 1$	The growth rate is constant and so does not depend on n, the size of the problem.
$g(n) = \log_2 n$	The growth rate is a function of $\log_2 n$. Because a logarithm function grows slowly, the growth rate of the function f is also slow.
$g(n) = n$	The growth rate is linear. The growth rate of f is directly proportional to the size of the problem.
$g(n) = n\log_2 n$	The growth rate is faster than the linear algorithm.
$g(n) = n^2$	The growth rate of such functions increases rapidly with the size of the problem. The growth rate is quadrupled when the problem size is doubled.
$g(n) = 2^n$	The growth rate is exponential. The growth rate is squared when the problem size is doubled.

NOTE

It can be shown that

$O(1) \leq O(\log_2 n) \leq O(n) \leq O(n\log_2 n) \leq O(n^2) \leq O(2^n)$.

Classes

In this section, we review C++ classes. If you are familiar with how classes are implemented in C++, you can skip this section.

Recall that in OOD, the first step is to identify the components called objects; an object combines data and the operations on that data in a single unit, called *encapsulation*. In C++, the mechanism that allows you to combine data and the operations on that data in a single unit is called a class. A **class** is a collection of a fixed number of components. The components of a class are called the **members** of the class.

The general syntax for defining a class is

```
class classIdentifier
{
    class members list
};
```

where `class members list` consists of variable declarations and/or functions. That is, a member of a class can be either a variable (to store data) or a function.

- If a member of a class is a variable, you declare it just like any other variable. Furthermore, in the definition of the class, you cannot initialize a variable when you declare it.
- If a member of a class is a function, you typically use the function prototype to define that member.
- If a member of a class is a function, it can (directly) access any member of the class—data members and function members. That is, when you write the definition of the member function, you can directly access any data member of the class without passing it as a parameter. The only obvious condition is that you must declare an identifier before you can use it.

In C++, **`class`** is a reserved word, and it defines only a data type; no memory is allocated. It announces the declaration of a class. Moreover, note the semicolon (;) after the right brace. The semicolon is part of the syntax. A missing semicolon, therefore, will result in a syntax error.

The members of a **`class`** are classified into three categories: **`private`**, **`public`**, and **`protected`**, called member **access specifiers**. This chapter mainly discusses the first two types—that is, **`private`** and **`public`**.

Following are some facts about **`private`** and **`public`** members of a class:

- By default, all members of a class are **`private`**.
- If a member of a class is **`private`**, you cannot access it outside the class.
- A **`public`** member is accessible outside the class.
- To make a member of a class **`public`**, you use the member access specifier **`public`** with a colon.

In C++, **`private`**, **`protected`**, and **`public`** are reserved words.

EXAMPLE 1-9

Suppose that we want to define a **`class`**, **`clockType`**, to implement the time of day in a program. Furthermore, suppose that the time is represented as a set of three integers: one to represent the hours, one to represent the minutes, and one to represent the seconds. We also want to perform the following operations on the time:

1. Set the time.
2. Return the time.
3. Print the time.
4. Increment the time by one second.
5. Increment the time by one minute.
6. Increment the time by one hour.
7. Compare two times for equality.

From this discussion, it is clear that the `class clockType` has 10 members: three data members and seven function members.

Some members of the `class clockType` will be `private`; others will be `public`. Deciding which member to make `private` and which to make `public` depends on the nature of the member. The general rule is that any member that needs to be accessed outside the class is declared `public`; any member that should not be accessed directly by the user should be declared `private`. For example, the user should be able to set the time and print the time. Therefore, the members that set the time and print the time should be declared `public`.

Similarly, the members to increment the time and compare the time for equality should be declared `public`. On the other hand, to control the *direct* manipulation of the data members `hr`, `min`, and `sec`, we will declare these data members `private`. Furthermore, note that if the user has direct access to the data members, member functions such as `setTime` are not needed.

The following statements define the `class clockType`:

```
class clockType
{
public:
    void setTime(int hours, int minutes, int seconds);
      //Function to set the time
      //The time is set according to the parameters
      //Postcondition: hr = hours; min = minutes; sec = seconds
      //    The function checks whether the values of hours,
      //    minutes, and seconds are valid. If a value is invalid,
      //    the default value 0 is assigned.

    void getTime(int& hours, int& minutes, int& seconds) const;
      //Function to return the time
      //Postcondition: hours = hr; minutes = min; seconds = sec

    void printTime() const;
      //Function to print the time
      //Postcondition: Time is printed in the form hh:mm:ss.

    void incrementSeconds();
      //Function to increment the time by one second
      //Postcondition: The time is incremented by one second.
      //    If the before-increment time is 23:59:59, the time
      //    is reset to 00:00:00.

    void incrementMinutes();
      //Function to increment the time by one minute
      //Postcondition: The time is incremented by one minute.
      //    If the before-increment time is 23:59:53, the time
      //    is reset to 00:00:53.
```

```
    void incrementHours();
      //Function to increment the time by one hour
      //Postcondition: The time is incremented by one hour.
      //    If the before-increment time is 23:45:53, the time
      //    is reset to 00:45:53.

    bool equalTime(const clockType& otherClock) const;
      //Function to compare the two times
      //Postcondition: Returns true if this time is equal to
      //    otherClock; otherwise, returns false

private:
    int hr;  //stores the hours
    int min; //store the minutes
    int sec; //store the seconds
};
```

We note the following in the definition of the `class clockType`:

- The `class clockType` has seven function members: `setTime`, `getTime`, `printTime`, `incrementSeconds`, `incrementMinutes`, `incrementHours`, and `equalTime`. It has three data members: `hr`, `min`, and `sec`.
- The three data members—`hr`, `min`, and `sec`—are private to the class and cannot be accessed outside the class.
- The seven function members—`setTime`, `getTime`, `printTime`, `incrementSeconds`, `incrementMinutes`, `incrementHours`, and `equalTime`—can directly access the data members (`hr`, `min`, and `sec`). In other words, we do not pass data members as parameters to member functions.
- In the function `equalTime`, the parameter `otherClock` is a constant reference parameter. That is, in a call to the function `equalTime`, the parameter `otherClock` receives the address of the actual parameter, but `otherClock` cannot modify the value of the actual parameter. You could have declared `otherClock` as a value parameter, but that would require `otherClock` to copy the value of the actual parameter, which could result in poor performance. (For an explanation, see the section, "Reference Parameters and Class Objects (Variables)" located later in this chapter.)
- The word `const` at the end of the member functions `getTime`, `printTime`, and `equalTime` specifies that these functions cannot modify the data members of a variable of type `clockType`.

NOTE

(Order of `public` and `private` members of a class) C++ has no fixed order in which you declare `public` and `private` members; you can declare them in any order. The only thing you need to remember is that, by default, all members of a class are `private`. You must use the `public` label to make a member available for `public` access. If you decide to declare the `private` members after the `public` members (as is done in the case of `clockType`), you must use the `private` label to begin the declaration of the `private` members.

NOTE

In the definition of the `class clockType`, all data members are `private` and all function members are `public`. However, a function member can also be `private`. For example, if a member function is used only to implement other member functions of the class, and the user does not need to access this function, you make it `private`. Similarly, a data member of a class can also be `public`.

Note that we have not yet written the definitions of the function members of the `class clockType`. You will learn how to write them shortly.

The function `setTime` sets the three data members—`hr`, `min`, and `sec`—to a given value. The given values are passed as parameters to the function `setTime`. The function `printTime` prints the time, that is, the values of `hr`, `min`, and `sec`. The function `incrementSeconds` increments the time by one second, the function `incrementMinutes` increments the time by one minute, the function `incrementHours` increments the time by one hour, and the function `equalTime` compares the two times for equality.

Constructors

C++ does not automatically initialize variables when they are declared. Therefore, when an object is instantiated, there is no guarantee that the data members of the object will be initialized. To guarantee that the instance variables of a class are initialized, you use constructors. There are two types of constructors: with parameters and without parameters. The constructor without parameters is called the **default constructor**.

Constructors have the following properties:

- The name of a constructor is the same as the name of the class.
- A constructor, even though it is a function, has no type. That is, it is neither a value-returning function nor a `void` function.
- A class can have more than one constructor. However, all constructors of a class have the same name.
- If a class has more than one constructor, the constructors must have different formal parameter lists. That is, either they have a different number of formal parameters or, if the number of formal parameters is the same, the data type of the formal parameters, in the order you list, must differ in at least one position.
- Constructors execute automatically when a class object enters its scope. Because they have no types, they cannot be called like other functions.
- Which constructor executes depends on the types of values passed to the class object when the class object is declared.

Let us extend the definition of the `class clockType` by including two constructors:

```
class clockType
{
public:
    //Place the function prototypes of the functions setTime,
    //getTime, printTime, incrementSeconds, incrementMinutes,
    //incrementHours, and equalTime as described earlier, here.

    clockType(int hours, int minutes, int seconds);
      //Constructor with parameters
      //The time is set according to the parameters.
      //Postconditions: hr = hours; min = minutes; sec = seconds
      //    The constructor checks whether the values of hours,
      //    minutes, and seconds are valid. If a value is invalid,
      //    the default value 0 is assigned.

    clockType();
      //Default constructor with parameters
      //The time is set to 00:00:00.
      //Postcondition: hr = 0; min = 0; sec = 0

private:
    int hr;  //stores the hours
    int min; //store the minutes
    int sec; //store the seconds
};
```

Unified Modeling Language Diagrams

A class and its members can be described graphically using a notation known as **Unified Modeling Language (UML)** notation. For example, Figure 1-5 shows the UML class diagram of the `class clockType`.

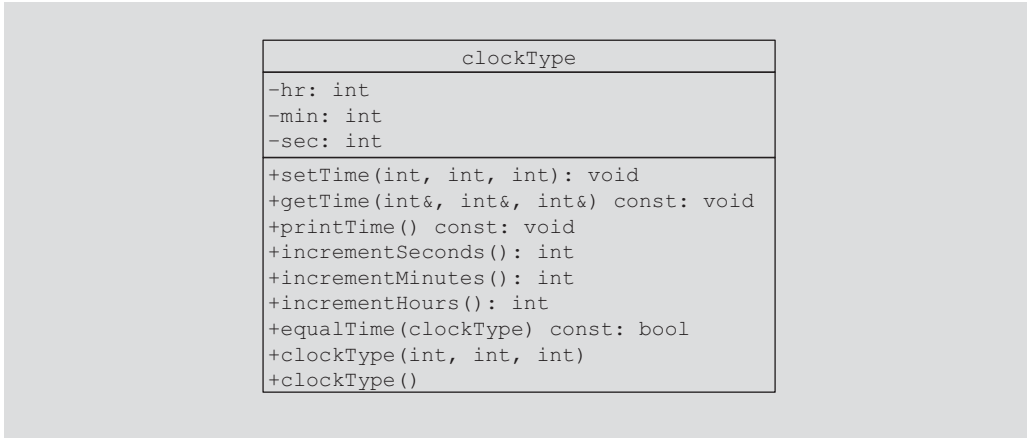


FIGURE 1-5 UML class diagram of the **class** clockType

The top box contains the name of the class. The middle box contains the data members and their data types. The last box contains the member function name, parameter list, and the return type of the function. A + (plus) sign in front of a member indicates that this member is a **public** member; a – (minus) sign indicates that this is a **private** member. The symbol # before the member name indicates that the member is a **protected** member.

Variable (Object) Declaration

Once a class is defined, you can declare variables of that type. In C++ terminology, a class variable is called a **class object** or **class instance**. To help you become familiar with this terminology, from now on we will use the term class object, or simply **object**, for a class variable.

A class can have both types of constructors—default constructor and constructors with parameters. Therefore, when you declare a class object, either the default constructor executes or the constructor with parameters executes. The general syntax for declaring a class object that invokes the default constructor is:

```
className classObjectName;
```

For example, the statement

```
clockType myClock;
```

declares `myClock` to be an object of type `clockType`. In this case, the default constructor executes and the instance variables of `myClock` are initialized to `0`.

NOTE

If you declare an object and want the default constructor to be executed, the empty parentheses after the object name are not required in the object declaration statement. In fact, if you accidentally include the empty parentheses, the compiler generates a syntax error message. For example, the following statement to declare the object `myClock` is illegal:

```
clockType myClock();  //illegal object declaration
```

The general syntax for declaring a class object that invokes a constructor with a parameter is

```
className classObjectName(argument1, argument2, ...);
```

where each of `argument1`, `argument2`, and so on is either a variable or an expression. Note the following:

- The number of arguments and their type should match the formal parameters (in the order given) of one of the constructors.
- If the type of the arguments does not match the formal parameters of any constructor (in the order given), C++ uses type conversion and looks for the best match. For example, an integer value might be converted to a floating-point value with a zero decimal part. Any ambiguity will result in a compile-time error.

Consider the following statement:

```
clockType myClock(5, 12, 40);
```

This statement declares the object `myClock` of type `clockType`. Here, we are passing three values of type `int`, which matches the type of the formal parameters of the constructor with a parameter. Therefore, the constructor with parameters of the `class clockType` executes and the three instance variables of the object `myClock` are set to `5`, `12`, and `40`.

Consider the following statements that declare two objects of type `clockType`:

```
clockType myClock(8, 12, 30);
clockType yourClock(12, 35, 45);
```

Each object has 10 members: seven member functions and three instance variables. Each object has separate memory allocated for `hr`, `min`, and `sec`.

In actuality, memory is allocated only for the instance variables of each class object. The C++ compiler generates only one physical copy of a member function of a class, and each class object executes the same copy of the member function.

Accessing Class Members

Once an object of a class is declared, it can access the members of the class. The general syntax for an object to access a member of a class is:

```
classObjectName.memberName
```

In C++, the dot, `.` (period), is an operator called the **member access operator**.

The class members that a class object can access depend on where the object is declared.

- If the object is declared in the definition of a member function of the class, the object can access both the `public` and `private` members. (We will elaborate on this when we write the definition of the member function `equalTime` of the `class clockType` in the section "Implementation of Member Functions," later in this chapter.)
- If the object is declared elsewhere (for example, in a user's program), the object can access *only* the `public` members of the class.

Example 1-10 illustrates how to access the members of a class.

EXAMPLE 1-10

Suppose we have the following declaration (say, in a user's program):

```
clockType myClock;
clockType yourClock;
```

Consider the following statements:

```
myClock.setTime(5, 2, 30);
myClock.printTime();

if (myClock.equalTime(yourClock))
.
.
.
```

These statements are legal; that is, they are syntactically correct.

In the first statement, `myClock.setTime(5, 2, 30);`, the member function `setTime` is executed. The values `5`, `2`, and `30` are passed as parameters to the function `setTime`, and the function uses these values to set the values of the three instance variables `hr`, `min`, and `sec` of `myClock` to `5`, `2`, and `30`, respectively. Similarly, the second statement executes the member function `printTime` and outputs the contents of the three instance variables of `myClock`.

In the third statement, the member function `equalTime` executes and compares the three instance variables of `myClock` to the corresponding instance variables of `yourClock`. Because in this statement `equalTime` is a member of the object `myClock`, it has direct access to the three instance variables of `myClock`. So it needs one more object, which in this case is `yourClock`, to compare. This explains why the function `equalTime` has only one parameter.

The objects `myClock` and `yourClock` can access only `public` members of the class. Thus, the following statements are illegal because `hr` and `min` are declared as `private` members of the `class clockType` and, therefore, cannot be accessed by the objects `myClock` and `yourClock`:

```
myClock.hr = 10;               //illegal
myClock.min = yourClock.min;   //illegal
```

Implementation of Member Functions

When we defined the `class clockType`, we included only the function prototype for the member functions. For these functions to work properly, we must write the related algorithms. One way to implement these functions is to provide the function definition rather than the function prototype in the class itself. Unfortunately, the class definition would then be long and difficult to comprehend. Another reason for providing function prototypes instead of function definitions relates to information hiding; that is, we want to hide the details of the operations on the data.

Next, let us write the definitions of the member functions of the `class clockType`. That is, we will write the definitions of the functions `setTime`, `getTime`, `printTime`, `incrementSeconds`, `equalTime`, and so on. Because the identifiers `setTime`, `printTime`,

and so forth are local to the class, we cannot reference them (directly) outside the class. To reference these identifiers, we use the **scope resolution operator**, `::` (double colon). In the function definition's heading, the name of the function is the name of the class, followed by the scope resolution operator, followed by the function name. For example, the definition of the function `setTime` is as follows:

```
void clockType::setTime(int hours, int minutes, int seconds)
{
    if (0 <= hours && hours < 24)
        hr = hours;
    else
        hr = 0;

    if (0 <= minutes && minutes < 60)
        min = minutes;
    else
        min = 0;

    if (0 <= seconds && seconds < 60)
        sec = seconds;
    else
        sec = 0;
}
```

Note that the definition of the function `setTime` checks for the valid values of `hours`, `minutes`, and `seconds`. If these values are out of range, the instance variables `hr`, `min`, and `sec` are initialized to `0`.

Suppose that `myClock` is an object of type `clockType` (as declared previously). The object `myClock` has three instance variables. Consider the following statement:

```
myClock.setTime(3, 48, 52);
```

In the statement `myClock.setTime(3, 48, 52);`, `setTime` is accessed by the object `myClock`. Therefore, the three variables—`hr`, `min`, and `sec`—to which the body of the function `setTime` refers, are the three instance variables of `myClock`. Thus, the values `3`, `48`, and `52`, which are passed as parameters in the preceding statement, are assigned to the three instance variables of `myClock` by the function `setTime` (see the body of the function `setTime`). After the previous statement executes, the object `myClock` is as shown in Figure 1-6.

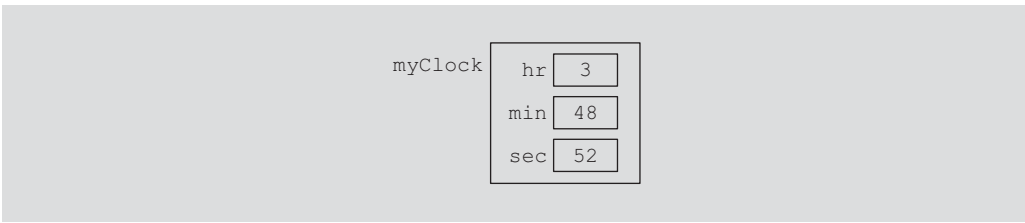


FIGURE 1-6 Object `myClock` after the statement `myClock.setTime(3, 48, 52);` executes

Next, let us give the definitions of the other member functions of the `class clockType`. The definitions of these functions are simple and easy to follow.

```
void clockType::getTime(int& hours, int& minutes, int& seconds) const
{
    hours = hr;
    minutes = min;
    seconds = sec;
}

void clockType::printTime() const
{
    if (hr < 10)
        cout << "0";
    cout << hr << ":";

    if (min < 10)
        cout << "0";
    cout << min << ":";

    if (sec < 10)
        cout << "0";
    cout << sec;
}

void clockType::incrementHours()
{
    hr++;
    if (hr > 23)
        hr = 0;
}

void clockType::incrementMinutes()
{
    min++;
    if (min > 59)
    {
        min = 0;
        incrementHours(); //increment hours
    }
}

void clockType::incrementSeconds()
{
    sec++;

    if (sec > 59)
    {
        sec = 0;
        incrementMinutes(); //increment minutes
    }
}
```

From the definitions of the functions **incrementMinutes** and **incrementSeconds**, it is clear that a member function of a class can call other member functions of the class.

The function **equalTime** has the following definition:

```
bool clockType::equalTime(const clockType& otherClock) const
{
    return (hr == otherClock.hr
            && min == otherClock.min
            && sec == otherClock.sec);
}
```

Let us see how the member function **equalTime** works.

Suppose that **myClock** and **yourClock** are objects of type **clockType**, as declared previously. Further suppose that we have **myClock** and **yourClock**, as shown in Figure 1-7.



FIGURE 1-7 Objects myClock and yourClock

Consider the following statement:

```
if (myClock.equalTime(yourClock))
.
.
.
```

In the expression

```
myClock.equalTime(yourClock)
```

the object **myClock** accesses the member function **equalTime**. Because **otherClock** is a reference parameter, the address of the actual parameter **yourClock** is passed to the formal parameter **otherClock**, as shown in Figure 1-8.

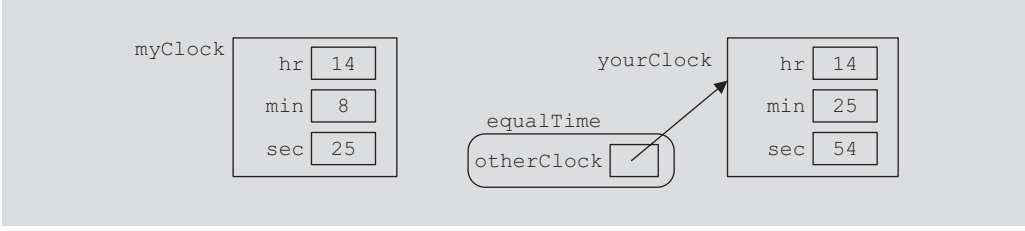


FIGURE 1-8 Object myClock and parameter otherClock

The instance variables hr, min, and sec of otherClock have the values 14, 25, and 54, respectively. In other words, when the body of the function equalTime executes, the value of otherClock.hr is 14, the value of otherClock.min is 25, and the value of otherClock.sec is 54. The function equalTime is a member of myClock. When the function equalTime executes, the variables hr, min, and sec in the body of the function equalTime are the instance variables of the variable myClock. Therefore, the member hr of myClock is compared with otherClock.hr, the member min of myClock is compared with otherClock.min, and the member sec of myClock is compared with otherClock.sec.

Once again, from the definition of the function equalTime, it is clear why this function has only one parameter.

Let us again look at the definition of the function equalTime. Notice that within the definition of this function, the object otherClock accesses the instance variables hr, min, and sec. However, these instance variables are private. So is there any violation? The answer is no. The function equalTime is a member of the class clockType and hr, min, and sec are the instance variables. Moreover, otherClock is an object of type clockType. Therefore, the object otherClock can access its private instance variables within the definition of the function equalTime.

The same is true for any member function of a class. In general, when you write the definition of a member function, say dummyFunction, of a class, say dummyClass, and the function uses an object, dummyObject of the class dummyClass, then within the definition of dummyFunction, the object dummyObject can access its private instance variables (in fact, any private member of the class).

This definition of the class clockType includes two constructors: one with three parameters and one without any parameters. Let us now write the definitions of these constructors.

```
clockType::clockType()  //default constructor
{
    hr = 0;
    min = 0;
    sec = 0;
}

clockType::clockType(int hours, int minutes, int seconds)
{
    if (0 <= hours && hours < 24)
        hr = hours;
    else
        hr = 0;

    if (0 <= minutes && minutes < 60)
        min = minutes;
    else
        min = 0;
```

```
    if (0 <= seconds && seconds < 60)
        sec = seconds;
    else
        sec = 0;
}
```

From the definitions of these constructors, it follows that the default constructor sets the three instance variables—`hr`, `min`, and `sec`—to `0`. Also, the constructor with parameters sets the instance variables to whatever values are assigned to the formal parameters. Moreover, we can write the definition of the constructor with parameters by calling the function `setTime`, as follows:

```
clockType::clockType(int hours, int minutes, int seconds)
{
    setTime(hours, minutes, seconds);
}
```

Once a class is properly defined and implemented, it can be used in a program. A program or software that uses and manipulates the objects of a class is called a **client** of that class.

When you declare objects of the `class clockType`, every object has its own copy of the instance variables `hr`, `min`, and `sec`. In object-oriented terminology, variables such as `hr`, `min`, and `sec` are called **instance variables** of the class because every object has its own instance of the data.

Reference Parameters and Class Objects (Variables)

Recall that when a variable is passed by value, the formal parameter copies the value of the actual parameter. That is, memory to copy the value of the actual parameter is allocated for the formal parameter. As a parameter, a class object can be passed by value.

Suppose that a class has several instance variables requiring a large amount of memory to store data, and you need to pass a variable by value. The corresponding formal parameter then receives a copy of the data of the variable. That is, the compiler must allocate memory for the formal parameter, so as to copy the value of the instance variables of the actual parameter. This operation might require, in addition to a large amount of storage space, a considerable amount of computer time to copy the value of the actual parameter into the formal parameter.

On the other hand, if a variable is passed by reference, the formal parameter receives only the address of the actual parameter. Therefore, an efficient way to pass a variable as a parameter is by reference. If a variable is passed by reference, then when the formal parameter changes, the actual parameter also changes. Sometimes, however, you do not want the function to be able to change the values of the instance variables. In C++, you can pass a variable by reference and still prevent the function from changing its value by using the keyword `const` in the formal parameter declaration. As an example, consider the following function definition:

```
void testTime(const clockType& otherClock)
{
    clockType dClock;
    .
    .
    .
}
```

The function `testTime` contains a reference parameter, `otherClock`. The parameter `otherClock` is declared using the keyword `const`. Thus, in a call to the function `testTime`, the formal parameter `otherClock` receives the address of the actual parameter, but `otherClock` cannot modify the contents of the actual parameter. For example, after the following statement executes, the value of `myClock` will not be altered:

```
testTime(myClock);
```

Generally, if you want to declare a class object as a value parameter, you declare it as a reference parameter using the keyword `const`, as described previously.

Recall that if a formal parameter is a value parameter, within the function definition you can change the value of the formal parameter. That is, you can use an assignment statement to change the value of the formal parameter (which, of course, would have no effect on the actual parameter). However, if a formal parameter is a constant reference parameter, you cannot use an assignment statement to change its value within the function, nor can you use any other function to change its value. Therefore, within the definition of the function `testTime`, you cannot alter the value of `otherClock`. For example, the following would be illegal in the definition of the function `testTime`:

```
otherClock.setTime(5, 34, 56); //illegal
otherClock = dClock;           //illegal
```

BUILT-IN OPERATIONS ON CLASSES

The two built-in operations that are defined for class objects are member access (`.`) and assignment (`=`). You have seen how to access an individual member of a class by using the name of the class object, then a dot, and then the member name.

We now show how an assignment statement works with the help of an example.

Assignment Operator and Classes

Suppose that `myClock` and `yourClock` are variables of type `clockType` as defined previously. The statement

```
myClock = yourClock;          //Line 1
```

copies the value of `yourClock` into `myClock`. That is, the value of `yourClock.hr` is copied into `myClock.hr`; the value of `yourClock.min` is copied into `myClock.min`; and the value of `yourClock.sec` is copied into `myClock.sec`. In other words, the values of the three instance variables of `yourClock` are copied into the corresponding instance variables of `myClock`. Therefore, an assignment statement performs a memberwise copy.

Class Scope

A **class** object can be either automatic (that is, created each time the control reaches its declaration, and destroyed when the control exits the surrounding block) or static (that is, created once, when the control reaches its declaration, and destroyed when the program terminates). Also, you can declare an array of **class** objects. A **class** object has the same scope as other variables. A member of a **class** is local to the **class**. You access a (**public**) **class** member outside the **class** by using the **class** object name and the member access operator (**.**).

Functions and Classes

The following rules describe the relationship between functions and classes:

- Class objects can be passed as parameters to functions and returned as function values.
- As parameters to functions, class objects can be passed either by value or by reference.
- If a class object is passed by value, the contents of the instance variables of the actual parameter are copied into the corresponding instance variables of the formal parameter.

Constructors and Default Parameters

A constructor can also have default parameters. In such a case, the rules for declaring formal parameters are the same as those for declaring default formal parameters in a function. Moreover, actual parameters to a constructor with default parameters are passed according to the rules for functions with default parameters. Using the rules for defining default parameters, in the definition of the `class clockType`, you can replace both constructors using the following statement. (Notice that in the function prototype, the name of a formal parameter is optional.)

```
clockType clockType(int = 0, int = 0, int = 0); //Line 1
```

In the implementation file, the definition of this constructor is the same as the definition of the constructor with parameters.

If you replace the constructors of the `class clockType` with the constructor in Line 1, (the constructor with the default parameters), you can declare `clockType` objects with 0, 1, 2, or 3 arguments as follows:

```
clockType clock1;              //Line 2
clockType clock2(5);           //Line 3
clockType clock3(12, 30);      //Line 4
clockType clock4(7, 34, 18);   //Line 5
```

The data members of `clock1` are initialized to `0`. The data member `hr` of `clock2` is initialized to `5`, and the data members `min` and `sec` of `clock2` are initialized to `0`. The

data member `hr` of `clock3` is initialized to `12`, the data member `min` of `clock3` is initialized to `30`, and the data member `sec` of `clock3` is initialized to `0`. The data member `hr` of `clock4` is initialized to `7`, the data member `min` of `clock4` is initialized to `34`, and the data member `sec` of `clock4` is initialized to `18`.

Using these conventions, we can say that a constructor that has no parameters, or has all default parameters, is called the **default constructor**.

Destructors

Like constructors, destructors are also functions. Moreover, like constructors, a destructor does not have a type. That is, it is neither a value-returning function nor a `void` function. However, a class can have only one destructor, and the destructor has no parameters. The name of a destructor is the *tilde* character `(~)`, followed by the name of the class. For example, the name of the destructor for the `class clockType` is:

```
~clockType();
```

The destructor automatically executes when the class object goes out of scope.

Structs

Structs are a special type of classes. By default, all members of a `class` are `private`, whereas by default all members of a `struct` are `public`. In C++, you define structs by using the reserved word `struct`. If all members of a class are `public`, C++ programmers prefer to use a `struct` to group the members, as we will do in this book. A `struct` is defined just like a `class`.

Data Abstraction, Classes, and Abstract Data Types

For the car that we drive, most of us want to know how to start the car and drive it. Most people are not concerned with the complexity of how the engine works. By separating the design details of a car's engine from its use, the manufacturer helps the driver focus on how to drive the car. Our daily life has other similar examples. For the most part, we are concerned only with how to use certain items, rather than with how they work.

Separating the design details (that is, how the car's engine works) from its use is called **abstraction**. In other words, abstraction focuses on what the engine does and not on how it works. Thus, abstraction is the process of separating the logical properties from the implementation details. Driving the car is a logical property; the construction of the engine constitutes the implementation details. We have an abstract view of what the engine does, but are not interested in the engine's actual implementation.

Abstraction can also be applied to data. Earlier sections of this chapter defined a data type `clockType`. The data type `clockType` has three instance variables and the following basic operations:

1. Set the time.
2. Return the time.
3. Print the time.
4. Increment the time by one second.
5. Increment the time by one minute.
6. Increment the time by one hour.
7. Compare two times to see whether they are equal.

The actual implementation of the operations, that is, the definitions of the member functions of the class, `clockType` was postponed.

Data abstraction is defined as a process of separating the logical properties of the data from its implementation. The definition of `clockType` and its basic operations are the logical properties; storing `clockType` objects in the computer, and the algorithms to perform these operations, are the implementation details of `clockType`.

Abstract data type (ADT): A data type that separates the logical properties from the implementation details.

Like any other data type, an ADT has three things associated with it: the name of the ADT, called the **type name**; the set of values belonging to the ADT, called the **domain**; and the set of **operations** on the data. Following these conventions, we can define the `clockType` ADT as follows:

```
dataTypeName
   clockType

domain
   Each clockType value is a time of day in the form of hours,
   minutes, and seconds.

operations
   Set the time.
   Return the time.
   Print the time.
   Increment the time by one second.
   Increment the time by one minute.
   Increment the time by one hour.
   Compare the two times to see whether they are equal.
```

To implement an ADT, you must represent the data and write algorithms to perform the operations.

The previous section used classes to group data and functions together. Furthermore, our definition of a class consisted only of the specifications of the operations; functions to

implement the operations were written separately. Thus, we see that classes are a convenient way to implement an ADT. In fact, in C++, classes were specifically designed to handle ADTs.

EXAMPLE 1-11

A list is defined as a set of values of the same type. Because all values in a list are of the same type, a convenient way to represent and process a list is to use an array. You can define a list as an ADT as follows:

```
typeName
   listType
domain
   Every element of listType is a set of, say at most 1000 numbers.
operations
   Check to see whether the list is empty.
   Check to see whether the list is full.
   Search the list for a given item.
   Delete an item from the list.
   Insert an item in the list.
   Sort the list.
   Print the list.
```

The following class implements the ADT list. To be specific, suppose that the list is a set of elements of the type `int`.

```
class intListType
{
public:
    bool isEmpty();
      //Function to determine whether the list is empty.
      //Precondition: The list must exist.
      //Postcondition: Returns true if the list is empty,
      //    false otherwise.
    bool isFull();
      //Function to determine whether the list is full.
      //Precondition: The list must exist.
      //Postcondition: Returns true if the list is full,
      //    false otherwise.
    int search(int searchItem);
      //Function to determine whether searchItem is in the list.
      //Postcondition: If searchItem is in the list, returns its
      //    index, that is, its position in the list;
      //    otherwise, it returns -1.
    void insert(int newItem);
      //Function to insert newItem in the list.
      //Precondition: The list must exist and must not be full.
      //Postcondition: newItem is inserted in the list and
      //    length is incremented by one.
```

```
    void remove(int removeItem);
      //Function to delete removeItem from the list.
      //Precondition: The list must exist and must not be empty
      //Postcondition: If found, removeItem is deleted from the
      //    list and the length is decremented by one;
      //    otherwise, an appropriate message is printed.
    void printList();
      //Function to output the elements of the list.
      //Precondition: The list must exist.
      //Postcondition: The elements of the list are
      //    printed on the standard output device.
    intListType();
      //Default constructor
      //Postcondition: length = 0

private:
    int list[1000];
    int length;
};
```

The `class personType` that is designed in Example 1-12 is quite useful; we will use this class in subsequent chapters.

EXAMPLE 1-12

The most common attributes of a person are the person's first name and last name. The typical operations on a person's name are to set the name and print the name. The following statements define a class with these properties.

```
//***********************************************************
// Author: D.S. Malik
//
// class personType
// This class specifies the members to implement a name.
//***********************************************************

#include <string>

using namespace std;

class personType
{
public:
    void print() const;
      //Function to output the first name and last name
      //in the form firstName lastName.

    void setName(string first, string last);
      //Function to set firstName and lastName according to the
      //parameters.
      //Postcondition: firstName = first; lastName = last
```

```
    string getFirstName() const;
      //Function to return the first name.
      //Postcondition: The value of firstName is returned.

    string getLastName() const;
      //Function to return the last name.
      //Postcondition: The value of lastName is returned.

    personType();
      //Default constructor
      //Sets firstName and lastName to null strings.
      //Postcondition: firstName = ""; lastName = "";

    personType(string first, string last);
      //Constructor with parameters.
      //Sets firstName and lastName according to the parameters.
      //Postcondition: firstName = first; lastName = last;

private:
    string firstName; //variable to store the first name
    string lastName;  //variable to store the last name
};
```

Figure 1-9 shows the UML class diagram of the **class** personType.

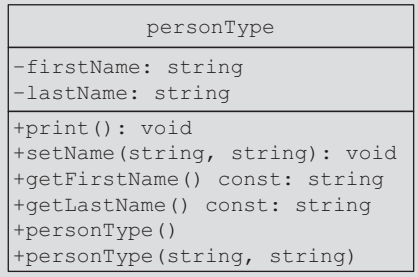


FIGURE 1-9 UML class diagram of the **class** personType

We now give the definitions of the member functions of the **class** personType.

```
void personType::print() const
{
    cout << firstName << " " << lastName;
}

void personType::setName(string first, string last)
{
    firstName = first;
    lastName = last;
}
```

```
string personType::getFirstName() const
{
    return firstName;
}

string personType::getLastName() const
{
    return lastName;
}

    //Default constructor
personType::personType()
{
    firstName = "";
    lastName = "";
}

    //Constructor with parameters
personType::personType(string first, string last)
{
    firstName = first;
    lastName = last;
}
```

PROGRAMMING EXAMPLE: Fruit Juice Machine

A new fruit juice machine has been purchased for the cafeteria, and a program is needed to make the machine function properly. The machine dispenses apple juice, orange juice, mango lassi, and fruit punch in recyclable containers. In this programming example, we write a program for the fruit juice machine so that it can be put into operation.

The program should do the following:

1. Show the customer the different products sold by the juice machine.
2. Let the customer make the selection.
3. Show the customer the cost of the item selected.
4. Accept money from the customer.
5. Release the item.

Input The item selection and the cost of the item.

Output The selected item.

PROBLEM ANALYSIS AND ALGORITHM DESIGN

A juice machine has two main components: a built-in cash register and several dispensers to hold and release the products.

Cash Register

Let us first discuss the properties of a cash register. The cash register has some cash on hand, it accepts the amount from the customer, and if the amount deposited is more than the cost of the item, then—if possible—the cash register returns the change. For simplicity, we assume that the user deposits at least the amount of money for the product. The cash register should also be able to show the juice machine's owner the amount of money in the register at any given time. The following class defines the properties of a cash register.

```
//*****************************************************************
// Author: D.S. Malik
//
// class cashRegister
// This class specifies the members to implement a cash register.
//*****************************************************************

class cashRegister
{
public:
    int getCurrentBalance() const;
      //Function to show the current amount in the cash register.
      //Postcondition: The value of cashOnHand is returned.

    void acceptAmount(int amountIn);
      //Function to receive the amount deposited by
      //the customer and update the amount in the register.
      //Postcondition: cashOnHand = cashOnHand + amountIn;

    cashRegister();
       //Default constructor
       //Sets the cash in the register to 500 cents.
       //Postcondition: cashOnHand = 500.

    cashRegister(int cashIn);
       //Constructor with a parameter.
       //Sets the cash in the register to a specific amount.
       //Postcondition: cashOnHand = cashIn;

private:
     int cashOnHand; //variable to store the cash in the register
};
```

Figure 1-10 shows the UML class diagram of the **class cashRegister**.

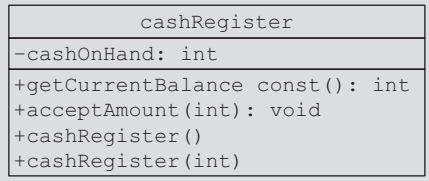


FIGURE 1-10 UML class diagram of the **class** cashRegister

Next, we give the definitions of the functions to implement the operations of the **class cashRegister**. The definitions of these functions are simple and easy to follow.

The function **getCurrentBalance** shows the current amount in the cash register. It returns the value of the instance variable **cashOnHand**. So, its definition is the following:

```
int cashRegister::getCurrentBalance() const
{
    return cashOnHand;
}
```

The definitions of the remaining function(s) and constructors are as follows:

```
void cashRegister::acceptAmount(int amountIn)
{
    cashOnHand = cashOnHand + amountIn;
}

cashRegister::cashRegister()
{
    cashOnHand = 500;
}

cashRegister::cashRegister(int cashIn)
{
    if (cashIn >= 0)
        cashOnHand = cashIn;
    else
        cashOnHand = 500;
}
```

Dispenser The dispenser releases the selected item if it is not empty. The dispenser should show the number of items in the dispenser and the cost of the item. The following class defines the properties of a dispenser. Let us call this `class dispenserType`.

```
//***********************************************************
// Author: D.S. Malik
//
// class dispenserType
// This class specifies the members to implement a dispenser.
//***********************************************************

class dispenserType
{
public:
    int getNoOfItems() const;
      //Function to show the number of items in the machine.
      //Postcondition: The value of numberOfItems is returned.

    int getCost() const;
      //Function to show the cost of the item.
      //Postcondition: The value of cost is returned.

    void makeSale();
      //Function to reduce the number of items by 1.
      //Postcondition: numberOfItems--;

    dispenserType();
      //Default constructor
      //Sets the cost and number of items in the dispenser to 50.
      //Postcondition: numberOfItems = 50; cost = 50;

    dispenserType(int setNoOfItems, int setCost);
      //Constructor with parameters
      //Sets the cost and number of items in the dispenser
      //to the values specified by the user.
      //Postcondition: numberOfItems = setNoOfItems;
      //    cost = setCost;

private:
    int numberOfItems;   //variable to store the number of
                         //items in the dispenser
    int cost;  //variable to store the cost of an item
};
```

Figure 1-11 shows the UML class diagram of the **class dispenserType**.

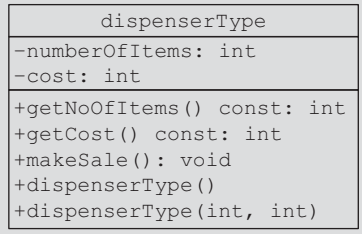


FIGURE 1-11 UML class diagram of the **class** dispenserType

Because the juice machine sells four types of items, we shall declare four objects of type **dispenserType**. For example, the statement

```
dispenserType appleJuice(100, 50);
```

declares **appleJuice** to be an object of type **dispenserType**, and sets the number of apple juice cans in the dispenser to 100 and the cost of each can to 50 cents.

Following the definitions of the **class dispenserType**, the definitions of the member functions and constructors are as follows:

```
int dispenserType::getNoOfItems() const
{
    return numberOfItems;
}

int dispenserType::getCost() const
{
    return cost;
}

void dispenserType::makeSale()
{
    numberOfItems--;
}

dispenserType::dispenserType()
{
    numberOfItems = 50;
    cost = 50;
}
```

```
dispenserType::dispenserType(int setNoOfItems, int setCost)
{
    if (setNoOfItems >= 0)
        numberOfItems = setNoOfItems;
    else
        numberOfItems = 50;

    if (setCost >= 0)
        cost = setCost;
    else
        cost = 50;
}
```

MAIN PROGRAM

When the program executes, it must do the following:

1. Show the different products sold by the juice machine.
2. Show how to select a particular product.
3. Show how to terminate the program.

Furthermore, these instructions must be displayed after processing each selection (except exiting the program), so that the user need not remember what to do if he or she wants to buy two or more items. Once the user has made the appropriate selection, the juice machine must act accordingly. If the user has opted to buy a product and if that product is available, the juice machine should show the cost of the product and ask the user to deposit the money. If the amount deposited is at least the cost of the item, the juice machine should sell the item and display an appropriate message.

This discussion translates into the following algorithm:

1. Show the selection to the customer.
2. Get the selection.
3. If the selection is valid and the dispenser corresponding to the selection is not empty, sell the product.

We divide this program into three functions—`showSelection`, `sellProduct`, and `main`.

`showSelection` This function displays the information necessary to help the user select and buy a product. The definition of this function is:

```
void showSelection()
{
    cout << "*** Welcome to Shelly's Fruit Juice Shop ***" << endl;
    cout << "To select an item, enter " << endl;
    cout << "1 for apple juice" << endl;
    cout << "2 for orange juice" << endl;
    cout << "3 for mango lassi" << endl;
    cout << "4 for fruit punch" << endl;
    cout << "9 to exit" << endl;
}//end showSelection
```

`sellProduct` This function attempts to sell the product selected by the customer. Therefore, it must have access to the dispenser holding the product. The first thing that this function does is check whether the dispenser holding the product is empty. If the dispenser is empty, the function informs the customer that this product is sold out. If the dispenser is not empty, it tells the user to deposit the necessary amount to buy the product.

If the user does not deposit enough money to buy the product, `sellProduct` tells the user how much additional money must be deposited. If the user fails to deposit enough money, in two tries, to buy the product, the function simply returns the money. (Programming Exercise 5, at the end of this chapter, asks you to revise the definition of the function `sellProduct` so that it keeps asking the user to enter the additional amount as long as the user has not entered enough money to buy the product.) If the amount deposited by the user is sufficient, it accepts the money and sells the product. Selling the product means to decrement the number of items in the dispenser by `1`, and to update the money in the cash register by adding the cost of the product. (We also assume that this program does not return the extra money deposited by the customer. So the cash register is updated by adding the money entered by the user.)

From this discussion, it is clear that the function `sellProduct` must have access to the dispenser holding the product (to decrement the number of items in the dispenser by 1 and to show the cost of the item) as well as the cash register (to update the cash). Therefore, this function has two parameters: one corresponding to the dispenser and the other corresponding to the cash register. Furthermore, both parameters must be referenced.

In pseudocode, the algorithm for this function is:

1. If the dispenser is not empty
    a. Show and prompt the customer to enter the cost of the item.
    b. Get the amount entered by the customer.
    c. If the amount entered by the customer is less than the cost of the product,
        i. Show and prompt the customer to enter the additional amount.
        ii. Calculate the total amount entered by the customer.
    d. If the amount entered by the customer is at least the cost of the product,
        i. Update the amount in the cash register.
        ii. Sell the product—that is, decrement the number of items in the dispenser by 1.
        iii. Display an appropriate message.
    e. If the amount entered by the user is less than the cost of the item, return the amount.

2. If the dispenser is empty, tell the user that this product is sold out.

The definition of the function `sellProduct` is:

```
void sellProduct(dispenserType& product, cashRegister& pCounter)
{
    int amount;  //variable to hold the amount entered
    int amount2; //variable to hold the extra amount needed

    if (product.getNoOfItems() > 0) //if the dispenser is not empty
    {
        cout << "Please deposit " << product.getCost()
             << " cents" << endl;
        cin >> amount;

        if (amount < product.getCost())
        {
            cout << "Please deposit another "
                 << product.getCost()- amount << " cents" << endl;
            cin >> amount2;
            amount = amount + amount2;
        }

        if (amount >= product.getCost())
        {
            pCounter.acceptAmount(amount);
            product.makeSale();
            cout << "Collect your item at the bottom and enjoy."
                 << endl;
        }
        else
            cout << "The amount is not enough. "
                 << "Collect what you deposited." << endl;

        cout << "*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*"
             << endl << endl;
    }
    else
        cout << "Sorry, this item is sold out." << endl;
}//end sellProduct
```

main The algorithm for the function `main` is as follows:

1. Create the cash register—that is, declare a variable of type `cashRegister`.
2. Create four dispensers—that is, declare four objects of type `dispenserType` and initialize these objects. For example, the statement

```
dispenserType mangoLassi(75, 45);
```

creates a dispenser object, `mangoLassi`, to hold the juice cans. The number of items in the dispenser is `75`, and the cost of an item is `45` cents.

3. Declare additional variables as necessary.
4. Show the selection; call the function `showSelection`.
5. Get the selection.
6. While not done (a selection of 9 exits the program),
   a. Sell the product; call the function `sellProduct`.
   b. Show the selection; call the function `showSelection`.
   c. Get the selection.

The definition of the function `main` is as follows:

```
int main()
{
    cashRegister counter;
    dispenserType appleJuice(100, 50);
    dispenserType orangeJuice(100, 65);
    dispenserType mangoLassi(75, 45);
    dispenserType fruitPunch(100, 85);

    int choice;  //variable to hold the selection

    showSelection();
    cin >> choice;

    while (choice != 9)
    {
        switch (choice)
        {
        case 1:
            sellProduct(appleJuice, counter);
            break;

        case 2:
            sellProduct(orangeJuice, counter);
            break;

        case 3:
            sellProduct(mangoLassi, counter);
            break;

        case 4:
            sellProduct(fruitPunch, counter);
            break;

        default:
            cout << "Invalid selection." << endl;
        }//end switch
```

```
            showSelection();
            cin >> choice;
        }//end while

        return 0;
    }//end main
```

PROGRAM LISTING

```
//***********************************************************
// Author: D.S. Malik
//
// This program uses the classes cashRegister and dispenserType
// to implement a fruit juice machine.
// ***********************************************************

#include <iostream>
#include "cashRegister.h"
#include "dispenserType.h"

using namespace std;

void showSelection();
void sellProduct(dispenserType& product, cashRegister& pCounter);

//Place the definitions of the functions main, showSelection, and
//sellProduct here.
```

Sample Run: In this sample run, the user input is shaded.

```
*** Welcome to Shelly's Fruit Juice Shop ***
To select an item, enter
1 for apple juice
2 for orange juice
3 for mango lassi
4 for fruit punch
9 to exit
1
Please deposit 50 cents
50
Collect your item at the bottom and enjoy.
*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
```

```
*** Welcome to Shelly's Fruit Juice Shop ***
To select an item, enter
1 for apple juice
2 for orange juice
3 for mango lassi
4 for fruit punch
9 to exit
9
```

The complete definitions of the classes `cashRegister`, `dispenserType`, the implementation files, and the `main` program is available at the Web site accompanying this book.

Identifying Classes, Objects, and Operations

The hardest part of OOD is to identify the classes and objects. This section describes a common and simple technique to identify classes and objects.

We begin with a description of the problem and then identify all of the nouns and verbs. From the list of nouns we choose our classes, and from the list of verbs we choose our operations.

For example, suppose that we want to write a program that calculates and prints the volume and surface area of a cylinder. We can state this problem as follows:

Write a **program** to *input* the **dimensions** of a **cylinder** and *calculate* and *print* its **surface area** and **volume**.

In this statement, the nouns are bold and the verbs are italic. From the list of nouns—**program**, **dimensions**, **cylinder**, **surface area**, and **volume**—we can easily visualize **cylinder** to be a class—say, `cylinderType`—from which we can create many cylinder objects of various dimensions. The nouns—**dimensions**, **surface area**, and **volume**—are characteristics of a **cylinder** and, thus, can hardly be considered classes.

After we identify a class, the next step is to determine three pieces of information:

- Operations that an object of that class type can perform
- Operations that can be performed on an object of that class type
- Information that an object of that class type must maintain

From the list of verbs identified in the problem description, we choose a list of possible operations that an object of that class can perform, or has performed, on itself. For example, from the list of verbs for the cylinder problem description—*write*, *input*, *calculate*, and *print*—the possible operations for a cylinder object are *input*, *calculate*, and *print*.

For the `cylinderType` class, the dimensions represent the data. The `center` of the base, `radius` of the base, and `height` of the cylinder are the characteristics of the dimensions. You can input data to the object either by a constructor or by a function.

The verb *calculate* applies to determining the volume and the surface area. From this, you can deduce the operations: `cylinderVolume` and `cylinderSurfaceArea`. Similarly, the verb *print* applies to the display of the volume and the surface area on an output device.

Identifying classes via the nouns and verbs from the descriptions to the problem is not the only technique possible. There are several other OOD techniques in the literature. However, this technique is sufficient for the programming exercises in this book.

QUICK REVIEW

1. Software are programs run by the computer.
2. A program goes through many phases from the time it is first conceived until the time it is retired, called the life cycle of the program.
3. The three fundamental stages through which a program goes are development, use, and maintenance.
4. The new program is created in the software development stage.
5. In the software maintenance process, the program is modified to fix the (identified) problems and/or to enhance it.
6. A program is retired if no new version of the program will be released.
7. The software development phases are analysis, design, implementation, and testing and debugging.
8. During the design phase, algorithms are designed to solve the problem.
9. An algorithm is a step-by-step problem-solving process in which a solution is arrived at in a finite amount of time.
10. Two well-known design techniques are structured-design and object-oriented design.
11. In structured design, a problem is divided into smaller subproblems. Each subproblem is solved, and the solutions of all the subproblems are then combined to solve the problem.
12. In object-oriented design (OOD), a program is a collection of interacting objects.
13. An object consists of data and operations on those data.
14. The three basic principles of OOD are encapsulation, inheritance, and polymorphism.
15. In the implementation phase, you write and compile programming code to implement the classes and functions that were discovered in the design phase.
16. A precondition is a statement specifying the condition(s) that must be true before the function is called.
17. A postcondition is a statement specifying what is true after the function call is completed.

18. During the testing phase, the program is tested for its correctness; that is, for making sure that the program does what it is supposed to do.
19. Debugging refers to finding and fixing the errors, if they exist.
20. To find problems in a program, it is run through a series of test cases.
21. A test case consists of a set of inputs, user actions, or other initial conditions, and the expected output.
22. There are two types of testing—black-box testing and white-box testing.
23. While analyzing a particular algorithm, we usually count the number of operations performed by the algorithm.
24. Let *f* be a function of *n*. The term asymptotic refers to the study of the function *f* as *n* becomes larger and larger without bound.
25. A class is a collection of a fixed number of components.
26. Components of a class are called the members of the class.
27. Members of a class are accessed by name.
28. In C++, **`class`** is a reserved word.
29. Members of a class are classified into one of three categories: **`private`**, **`protected`**, and **`public`**.
30. The **`private`** members of a class are not accessible outside the class.
31. The **`public`** members of a class are accessible outside the class.
32. By default, all members of a class are **`private`**.
33. The **`public`** members are declared using the member access specifier **`public`**.
34. The **`private`** members are declared using the member access specifier **`private`**.
35. A member of a class can be a function or a variable (that is, data).
36. If any member of a class is a function, you usually use the function prototype to declare it.
37. If any member of a class is a variable, it is declared like any other variable.
38. In the definition of the class, you cannot initialize a variable when you declare it.
39. In the Unified Modeling language (UML) diagram of a class, the top box contains the name of the class. The middle box contains the data members and their data types. The last box contains the member function name, parameter list, and the return type of the function. A **`+`** (plus) sign in front of a member indicates that this member is a **`public`** member; a **`-`** (minus) sign indicates that this is a **`private`** member. The symbol **`#`** before the member name indicates that the member is a **`protected`** member.

40. In C++, a **class** is a definition. No memory is allocated; memory is allocated for the class variables when you declare them.
41. In C++, class variables are called class objects or simply objects.
42. A class member is accessed using the class variable name, followed by the dot operator (.), followed by the member name.
43. The only built-in operations on classes are the assignment and member selection.
44. Class objects can be passed as parameters to functions and returned as function values.
45. As parameters to functions, classes can be passed either by value or by reference.
46. Constructors guarantee that the data members are initialized when an object is declared.
47. The name of a constructor is the same as the name of the class.
48. A class can have more than one constructor.
49. A constructor without parameters is called the default constructor.
50. Constructors automatically execute when a **class** object enters its scope.
51. Destructors automatically execute when a **class** object goes out of scope.
52. A class can have only one destructor with no parameters.
53. The name of a destructor is the tilde (~), followed by the class name (no spaces in between).
54. Constructors and destructors are functions without any type; that is, they are neither value-returning nor **void**. As a result, they cannot be called like other functions.
55. A data type that specifies the logical properties without the implementation details is called an abstract data type (ADT).
56. An easy way to identify classes, objects, and operations is to describe the problem in English and then identify all of the nouns and verbs. Choose your classes (objects) from the list of nouns and operations from the list of verbs.

EXERCISES

1. Mark the following statements as true or false.
   a. The life cycle of software refers to the phases from the point the software was conceived until it is retired.
   b. The three fundamental stages of software are development, use, and discard.
   c. The expression $4n + 2n^2 + 5$ is $O(n)$.

d. The instance variables of a class must be of the same type.
e. The function members of a class must be **public**.
f. A class can have more than one constructor.
g. A class can have more than one destructor.
h. Both constructors and destructors can have parameters.

2. What is black-box testing?
3. What is white-box testing?
4. Consider the following function prototype, which returns the square root of a real number:

```
double sqrt(double x);
```

What should be the pre- and postconditions for this function?

5. Each of the following expressions represents the number of operations for certain algorithms. What is the order of each of these expressions?

a. $n^2 + 6n + 4$
b. $5n^3 + 2n + 8$
c. $(n^2 + 1)\ (3n + 5)$
d. $5(6n + 4)$
e. $n + 2\log_2 n - 6$
f. $4n \log_2 n + 3n + 8$

6. Consider the following function:

```
void funcExercise6(int x, int y)
{
    int z;

    z = x + y;
    x = y;
    y = z;
    z = x;
    cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
}
```

Find the exact number of operations executed by the function `funcExercise6`.

7. Consider the following function:

```
int funcExercise7(int list[], int size)
{
    int sum = 0;

    for (int index = 0; index < size; index++)
        sum = sum + list[index];

    return sum;
}
```

a. Find the number of operations executed by the function `funcExercise7` if the value of `size` is `10`.

b. Find the number of operations executed by the function `funcExercise7` if the value of `size` is `n`.

c. What is the order of the function `funcExercise7`?

8. Consider the following function prototype:

```
int funcExercise8(int x);
```

The function `funcExercise8` returns the value as follows: If `0 <= x <= 50`, it returns `2x`; if `-50 <= x < 0`, it returns x^2; otherwise it returns `-999`. What are the reasonable boundary values for the function `funcExercise8`?

9. Write a function that uses a loop to find the sum of the squares of all integers between `1` and *n*. What is the order of your function?

10. Characterize the following algorithm in terms of Big-O notation. Also find the exact number of additions executed by the loop. (Assume that all variables are properly declared.)

```
for (int i = 1; i <= n; i++)
    sum = sum + i * (i + 1);
```

11. Characterize the following algorithm in terms of Big-O notation. Also find the exact number of additions, subtractions, and multiplications executed by the loop. (Assume that all variables are properly declared.)

```
for (int i = 5; i <= 2 * n; i++)
    cout << 2 * n + i - 1 << endl;
```

12. Characterize the following algorithm in terms of Big-O notation.

```
for (int i = 1; i <= 2 * n; i++)
    for (int j = 1; j <= n; j++)
        cout << 2 * i + j;
cout << endl;
```

13. Characterize the following algorithm in terms of Big-O notation.

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        for (int k = 1; k <= n; k++)
            cout << i + j + k;
```

14. Find the syntax errors in the definitions of the following classes:

a.
```
class AA
{
public:
    void print();
    int sum();
    AA();
    int AA(int, int);
private:
    int x;
    int y;
};
```

b.
```
class BB
{
    int one ;
    int two;
public:
    bool equal();
    print();
    BB(int, int);
}
```

c.
```
class CC
{
public;
    void set(int, int);
    void print();
    CC();
    CC(int, int);
    bool CC(int, int);
private:
    int u;
    int v;
};
```

15. Consider the following declarations:

```
class xClass
{
public:
    void func();
    void print() const;
    xClass ();
    xClass (int, double);
private:
    int u;
    double w;
};

xClass x;
```

a. How many members does `class xClass` have?

b. How many `private` members does `class xClass` have?

c. How many constructors does `class xClass` have?

d. Write the definition of the member function `func` so that `u` is set to `10` and `w` is set to `15.3`.

e. Write the definition of the member function `print` that prints the contents of `u` and `w`.

f. Write the definition of the default constructor of the `class xClass` so that the `private` data members are initialized to `0`.

g. Write a C++ statement that prints the values of the data members of the object x.

h. Write a C++ statement that declares an object `t` of the type `xClass`, and initializes the data members of `t` to `20` and `35.0`, respectively.

16. Consider the definition of the following class:

```
class CC
{
public:
    CC();                   //Line 1
    CC(int);                //Line 2
    CC(int, int);           //Line 3
    CC(double, int);        //Line 4
.
.
.
private:
    int u;
    double v;
};
```

a. Give the line number containing the constructor that is executed in each of the following declarations:

   i. `CC one;`

   ii. `CC two(5, 6);`

   iii. `CC three(3.5, 8);`

b. Write the definition of the constructor in Line 1 so that the **`private`** data members are initialized to `0`.

c. Write the definition of the constructor in Line 2 so that the **`private`** data member `u` is initialized according to the value of the parameter, and the **`private`** data member `v` is initialized to `0`.

d. Write the definition of the constructors in Lines 3 and 4 so that the **`private`** data members are initialized according to the values of the parameters.

17. Given the definition of the **`class clockType`** with constructors (as described in this chapter), what is the output of the following C++ code?

```
clockType clock1;
clockType clock2(23, 13, 75);

clock1.printTime();
cout << endl;
clock2.printTime();
cout << endl;

clock1.setTime(6, 59, 39);
clock1.printTime();
cout << endl;
```

```
clock1.incrementMinutes();
clock1.printTime();
cout << endl;

clock1.setTime(0, 13, 0);

if (clock1.equalTime(clock2))
    cout << "Clock1 time is the same as clock2 time." << endl;
else
    cout << "The two times are different." << endl;
```

18. Write the definition of a class that has the following properties:

    a. The name of the `class` is `secretType`.

    b. The `class secretType` has four instance variables: `name` of type `string`, `age` and `weight` of type `int`, and `height` of type `double`.

    c. The `class secretType` has the following member functions:

       `print`—Outputs the data stored in the instance variables with the appropriate titles

       `setName`—Function to set the name

       `setAge`—Function to set the age

       `setWeight`—Function to set the weight

       `setHeight`—Function to set the height

       `getName`—Value-returning function to return the name

       `getAge`—Value-returning function to return the age

       `getWeight`—Value-returning function to return the weight

       `getHeight`—Value-returning function to return the height

       Default constructor—Sets `name` to the empty string and `age`, `weight`, and `height` to `0`

       Constructor with parameter—Sets the values of the instance variables to the values specified by the user

    d. Write the definition of the member functions of the `class secretType` as described in Part c.

19. Assume the definition of the `class personType` as given in this chapter.

    a. Write a C++ statement that declares `student` to be a `personType` object, and initialize its first name to `"Buddy"` and last name to `"Arora"`.

    b. Write a C++ statement that outputs the data stored in the object `student`.

    c. Write C++ statements that change the first name of `student` to `"Susan"` and the last name to `"Miller"`.

PROGRAMMING EXERCISES

1. Write a program that converts a number entered in Roman numerals to decimal form. Your program should consist of a `class`, say `romanType`. An object of `romanType` should do the following:

   a. Store the number as a Roman numeral.

   b. Convert and store the number into decimal form.

   c. Print the number as a Roman numeral or decimal number as requested by the user. (Write two separate functions—one to print the number as a Roman numeral and the other to print the number as a decimal number.)

   The decimal values of the Roman numerals are:

   ```
   M    1000
   D     500
   C     100
   L      50
   X      10
   V       5
   I       1
   ```

   Remember, a larger numeral preceding a smaller numeral means addition, so `LX` is `60`. A smaller numeral preceding a larger numeral means subtraction, so `XL` is `40`. Any place in a decimal number, such as the 1s place, the 10s place, and so on, requires from zero to four Roman numerals.

   d. Test your program using the following Roman numerals: `MCXIV`, `CCCLIX`, and `MDCLXVI`.

2. Write the definition of the `class dayType` that implements the day of the week in a program. The `class dayType` should store the day, such as `Sunday` for Sunday. The program should be able to perform the following operations on an object of type `dayType`:

   a. Set the day.

   b. Print the day.

   c. Return the day.

   d. Return the next day.

   e. Return the previous day.

   f. Calculate and return the day by adding certain days to the current day. For example, if the current day is Monday and we add 4 days, the day to be returned is Friday. Similarly, if today is Tuesday and we add 13 days, the day to be returned is Monday.

   g. Add the appropriate constructors.

3. Write the definitions of the functions to implement the operations for the `class dayType` as defined in Programming Exercise 2. Also, write a program to test various operations on this class.

4. Example 1-12 defined a `class personType` to store the name of a person. The member functions that we included merely print the name and set the name of a person. Redefine the `class personType` so that, in addition to what the existing class does, you also can do the following:

   a. Set the first name only.
   b. Set the last name only.
   c. Store and set the middle name.
   d. Check whether a given first name is the same as the first name of this person.
   e. Check whether a given last name is the same as the last name of this person.

   Write the definitions of the member functions to implement the operations for this class. Also, write a program to test various operations on this class.

5. The function `sellProduct` of the Fruit Juice Machine programming example gives the user only two chances to enter enough money to buy the product. Rewrite the definition of the function `sellProduct` so that it keeps prompting the user to enter more money as long as the user has not entered enough money to buy the product. Also, write a program to test your function.

6. The equation of a line in standard form is $ax + by = c$, where a and b both cannot be zero, and a, b, and c are real numbers. If $b \neq 0$, then $-a / b$ is the slope of the line. If $a = 0$, then it is a horizontal line, and if $b = 0$, then it is a vertical line. The slope of a vertical line is undefined. Two lines are parallel if they have the same slope or both are vertical lines. Two lines are perpendicular if either one of the lines is horizontal and another is vertical, or if the product of their slopes is –1. Design the `class lineType` to store a line. To store a line, you need to store the values of a (coefficient of x), b (coefficient of y), and c. Your class must contain the following operations:

   a. If a line is nonvertical, then determine its slope.
   b. Determine if two lines are equal. (Two lines $a_1x + b_1y = c_1$ and $a_2x + b_2y = c_2$ are equal if either $a_1 = a_2$, $b_1 = b_2$, and $c_1 = c_2$ or $a_1 = ka_2$, $b_1 = kb_2$, and $c_1 = kc_2$ for some real number k.)
   c. Determine if two lines are parallel.
   d. Determine if two lines are perpendicular.
   e. If two lines are not parallel, then find the point of intersection.

   Add appropriate constructors to initialize variables of `lineType`. Also write a program to test your class.

7. (**Tic-Tac-Toe**) Write a program that allows two players to play the tic-tac-toe game. Your program must contain the `class ticTacToe` to implement a `ticTacToe` object. Include a 3 by 3 two-dimensional array, as a `private` instance variable, to create the board. If needed, include additional member variables. Some of the operations on a `ticTacToe` object are printing the current board, getting a move, checking if a move is valid, and determining the winner after each move. Add additional operations as needed.



CHAPTER 2

Object-Oriented Design (OOD) and C++

IN THIS CHAPTER, YOU WILL:

- Learn about inheritance
- Learn about derived and base classes
- Explore how to redefine the member functions of a base class
- Examine how the constructors of base and derived classes work
- Learn how to construct the header file of a derived class
- Explore three types of inheritance: `public`, `protected`, and `private`
- Learn about composition
- Become familiar with the three basic principles of object-oriented design
- Learn about overloading
- Become aware of the restrictions on operator overloading
- Examine the pointer `this`
- Learn about `friend` functions
- Explore the members and nonmembers of a class
- Discover how to overload various operators
- Learn about templates
- Explore how to construct function templates and class templates

Chapter 1 introduced classes, abstract data types (ADT), and ways to implement ADT in C++. By using classes, you can combine data and operations in a single unit. An object, therefore, becomes a self-contained entity. Operations can directly access the data, but the internal state of an object cannot be manipulated directly.

In addition to implementing ADT, classes have other features. For instance, you can create new classes from existing classes. This important feature encourages code reuse.

Inheritance

Suppose that you want to design a class, `partTimeEmployee`, to implement and process the characteristics of a part-time employee. The main features associated with a part-time employee are the name, pay rate, and number of hours worked. In Example 1-12 (in Chapter 1), we designed a class to implement a person's name. Every part-time employee is a person. Therefore, rather than design the `class partTimeEmployee` from scratch, we want to be able to extend the definition of the `class personType` (from Example 1-12) by adding additional members (data and/or functions).

Of course, we do not want to make the necessary changes directly to the `class personType`—that is, edit the `class personType`, and add and/or delete members. In fact, we want to create the `class partTimeEmployee` without making any physical changes to the `class personType`, by adding only the members that are necessary. For example, because the `class personType` already has data members to store the first name and last name, we will not include any such members in the `class partTimeEmployee`. In fact, these data members will be inherited from the `class personType`. (We will design such a `class` in Example 2-2.)

In Chapter 1, we extensively studied and designed the `class clockType` to implement the time of day in a program. The `class clockType` has three data members to store hours, minutes, and seconds. Certain applications—in addition to hours, minutes, and seconds—might also require us to store the time zone. In this case, we would likely extend the definition of the `class clockType` and create a `class`, `extClockType`, to accommodate this new information. That is, we want to derive the `class extClockType` by adding a data member—say, `timeZone`—and the necessary function members to manipulate the time (see Programming Exercise 1 at the end of this chapter). In C++, the mechanism that allows us to accomplish this task is the principle of inheritance. Inheritance is an "is-a" relationship; for instance, "every employee is a person."

Inheritance lets us create new classes from existing classes. The existing classes are called the **base classes**; the new class that we create from the existing classes is called the **derived class**. The derived class inherits the properties of the base classes. So rather than create completely new classes from scratch, we can take advantage of inheritance and reduce software complexity.

Each derived class, in turn, becomes a base class for a future derived class. Inheritance can be either a **single inheritance** or a **multiple inheritance**. In a single inheritance,

the derived class is derived from a single base class; in a multiple inheritance, the derived class is derived from more than one base class. This chapter concentrates on single inheritance.

Inheritance can be viewed as a treelike, or hierarchical, structure wherein a base class is shown with its derived classes. Consider the tree diagram shown in Figure 2-1.

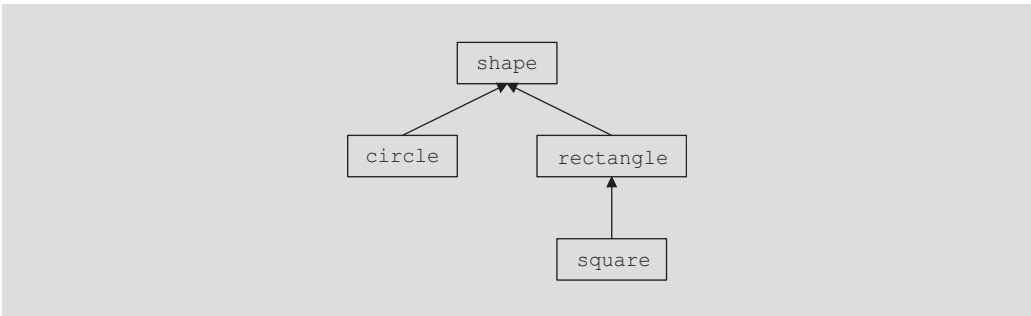


FIGURE 2-1 Inheritance hierarchy

In this diagram, `shape` is the base class. The `class`es `circle` and `rectangle` are derived from `shape`, and the `class square` is derived from `rectangle`. Every `circle` and every `rectangle` is a `shape`. Every `square` is a `rectangle`.

The general syntax of a derived class is:

```
class className: memberAccessSpecifier baseClassName
{
    member list
};
```

where `memberAccessSpecifier` is `public`, `protected`, or `private`. When no `memberAccessSpecifier` is specified, it is assumed to be a `private` inheritance. (We discuss `protected` inheritance later in this chapter.)

EXAMPLE 2-1

Suppose that we have defined a class called `shape`. The following statements specify that the `class circle` is derived from `shape`, and it is a `public` inheritance:

```
class circle: public shape
{
    .
    .
    .
};
```

On the other hand, consider the following definition of the `class circle`:

```
class circle: private shape
{
    .
    .
    .
};
```

This is a `private` inheritance. In this definition, the `public` members of `shape` become `private` members of the `class circle`. So any object of type `circle` cannot directly access these members. The previous definition of `circle` is equivalent to the following:

```
class circle: shape
{
    .
    .
    .
};
```

That is, if we do not use either the `memberAccessSpecifier` `public` or `private`, the `public` members of a base class are inherited as `private` members.

The following facts about the base and the derived classes should be kept in mind.

1. The `private` members of a base class are `private` to the base class; hence, the members of the derived class cannot directly access them. In other words, when you write the definitions of the member functions of the derived class, you cannot directly access the `private` members of the base class.
2. The `public` members of a base class can be inherited either as `public` members or as `private` members by the derived class. That is, the `public` members of the base class can become either `public` or `private` members of the derived class.
3. The derived class can include additional members—data and/or functions.
4. The derived class can redefine the `public` member functions of the base class. That is, in the derived class, you can have a member function with the same name, number, and types of parameters as a function in the base class. However, this redefinition applies only to the objects of the derived class, not to the objects of the base class.
5. All member variables of the base class are also member variables of the derived class. Similarly, the member functions of the base class (unless redefined) are also member functions of the derived class. (Remember Rule 1 when accessing a member of the base class in the derived class.)

The next sections describe two important issues related to inheritance. The first issue is the redefinition of the member functions of the base class in the derived class. While discussing

this issue, we also address how to access the `private` (data) members of the base class in the derived class. The second key inheritance issue is related to the constructor. The constructor of a derived class cannot *directly* access the `private` member variables of the base class. Thus, we need to ensure that the `private` member variables that are inherited from the base class are initialized when a constructor of the derived class executes.

Redefining (Overriding) Member Functions of the Base Class

Suppose that a `class derivedClass` is derived from the `class baseClass`. Further assume that both `derivedClass` and `baseClass` have some member variables. It then follows that the member variables of the `class derivedClass` are its own member variables, together with the member variables of `baseClass`. Suppose that `baseClass` contains a function, `print`, that prints the values of the member variables of `baseClass`. Now `derivedClass` contains member variables in addition to the member variables inherited from `baseClass`. Suppose that you want to include a function that prints the member variables of `derivedClass`. You can give any name to this function. However, in the `class derivedClass`, you can also name this function as `print` (the same name used by `baseClass`). This is called redefining (or overriding) the member function of the base class. Next, we illustrate how to redefine the member functions of a base class with the help of an example.

NOTE

To redefine a `public` member function of a base class in the derived class, the corresponding function in the derived class must have the same name, number, and types of parameters. In other words, the name of the function being redefined in the derived class must have the same name and the same set of parameters. If the corresponding functions in the base class and the derived class have the same name but different sets of parameters, this is function overloading in the derived class, which is also allowed.

Consider the definition of the following class:

```
//*****************************************************************
// Author: D.S. Malik
//
// class rectangleType
// This class specifies the members to implement the properties
// of a rectangle.
//*****************************************************************

class rectangleType
{
public:
    void setDimension(double l, double w);
      //Function to set the length and width of the rectangle.
      //Postcondition: length = l; width = w;

    double getLength() const;
      //Function to return the length of the rectangle.
      //Postcondition: The value of length is returned.
```

```
    double getWidth() const;
      //Function to return the width of the rectangle.
      //Postcondition: The value of width is returned.

    double area() const;
      //Function to return the area of the rectangle.
      //Postcondition: The area of the rectangle is calculated
      //    and returned.

    double perimeter() const;
      //Function to return the perimeter of the rectangle.
      //Postcondition: The perimeter of the rectangle is
      //    calculated and returned.

    void print() const;
      //Function to output the length and width of the rectangle.

    rectangleType();
      //default constructor
      //Postcondition: length = 0; width = 0;

    rectangleType(double l, double w);
      //constructor with parameters
      //Postcondition: length = l; width = w;

private:
    double length;
    double width;
};
```

Figure 2-2 shows the UML class diagram of the `class rectangleType`.

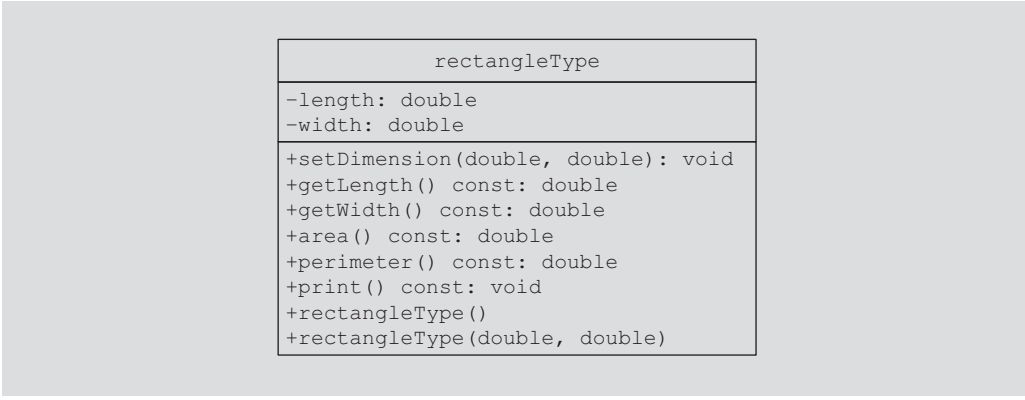


FIGURE 2-2 UML class diagram of the `class rectangleType`

Suppose that the definitions of the member functions of the `class rectangleType` are as follows:

```
void rectangleType::setDimension(double l, double w)
{
    if (l >= 0)
        length = l;
    else
        length = 0;

    if (w >= 0)
        width = w;
    else
        width = 0;
}

double rectangleType::getLength() const
{
    return length;
}

double rectangleType::getWidth() const
{
    return width;
}

double rectangleType::area() const
{
    return length * width;
}

double rectangleType::perimeter() const
{
     return 2 * (length + width);
}

void rectangleType::print() const
{
    cout << "Length = "  << length
         << "; Width = " << width;
}

rectangleType::rectangleType(double l, double w)
{
    setDimension(l, w);
}

rectangleType::rectangleType()
{
    length = 0;
    width = 0;
}
```

Now consider the definition of the following `class boxType`, derived from the `class rectangleType`:

```
//*************************************************************
// Author: D.S. Malik
//
// class boxType
// This class is derived from the class rectangleType and it
// specifies the members to implement the properties of a box.
//*************************************************************

class boxType: public rectangleType
{
public:
    void setDimension(double l, double w, double h);
      //Function to set the length, width, and height of the box.
      //Postcondition: length = l; width = w; height = h;

    double getHeight() const;
      //Function to return the height of the box.
      //Postcondition: The value of height is returned.

    double area() const;
      //Function to return the surface area of the box.
      //Postcondition: The surface area of the box is
      //    calculated and returned.

    double volume() const;
      //Function to return the volume of the box.
      //Postcondition: The volume of the box is calculated and
      //    returned.

    void print() const;
      //Function to output the length, width, and height of a box.

    boxType();
      //Default constructor
      //Postcondition: length = 0; width = 0; height = 0;

    boxType(double l, double w, double h);
      //Constructor with parameters
      //Postcondition: length = l; width = w; height = h;

private:
    double height;
};
```

Figure 2-3 shows the UML class diagram of the **class boxType** and the inheritance hierarchy.

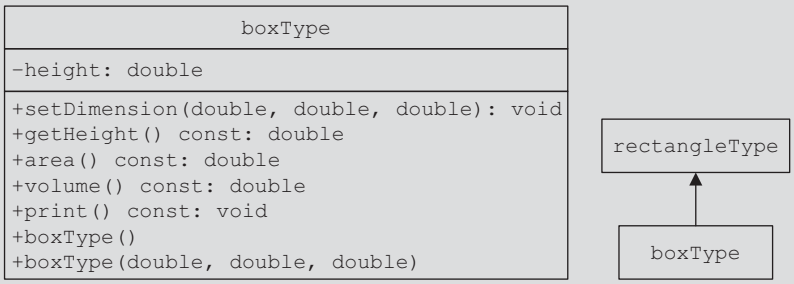


FIGURE 2-3 UML class diagram of the `class boxType` and the inheritance hierarchy

From the definition of the **class boxType**, it is clear that the **class boxType** is derived from the **class rectangleType**, and that it is a **public** inheritance. Therefore, all **public** members of the **class rectangleType** are **public** members of the **class boxType**. The **class boxType** also overrides (redefines) the functions **print** and **area**.

In general, while writing the definitions of the member functions of a derived class to specify a call to a **public** member function of the base class, we do the following:

- If the derived class overrides a **public** member function of the base class, then to specify a call to that **public** member function of the base class, you use the name of the base class, followed by the scope resolution operator, **::**, followed by the function name with the appropriate parameter list.
- If the derived class does not override a **public** member function of the base class, you may specify a call to that **public** member function by using the name of the function and the appropriate parameter list. (See the following note for member functions of the base class that are overloaded in the derived class.)

NOTE If a derived class *overloads* a **public** member function of the base class, then while writing the definition of a member function of the derived class, to specify a call to that (overloaded) member function of the base class, you might need (depending on the compiler) to use the name of the base class followed by the scope resolution operator, **::**, followed by the function name with the appropriate parameter list. For example, the **class boxType** overloads the member function **setDimension** of the **class rectangleType**. (See the definition of the function **setDimension** of the **class boxType** given later in this section.)

Next, let us write the definition of the member function `print` of the `class boxType`. The `class boxType` has three member variables: `length`, `width`, and `height`. The member function `print` of the `class boxType` prints the values of these member variables. To write the definition of the function `print` of the `class boxType`, keep in mind the following:

- The member variables `length` and `width` are `private` members of the `class rectangleType`, and so cannot be directly accessed in the `class boxType`. Therefore, when writing the definition of the function `print` of the `class boxType`, we cannot access `length` and `width` directly.
- The member variables `length` and `width` of the `class rectangleType` are accessible in the class `boxType` through the `public` member functions of the `class rectangleType`. Therefore, when writing the definition of the member function `print` of the `class boxType`, we first call the member function `print` of the `class rectangleType` to print the values of `length` and `width`. After printing the values of `length` and `width`, we output the values of `height`.

To call the member function `print` of `rectangleType` in the definition of the member function `print` of `boxType`, we must use the following statement:

```
rectangleType::print();
```

This statement ensures that we call the member function `print` of the base `class rectangleType`, not of the `class boxType`.

The definition of the member function `print` of the `class boxType` is:

```
void boxType::print() const
{
    rectangleType::print();
    cout << "; Height = " << height;
}
```

Let us write the definitions of the remaining member functions of the `class boxType`.

The definition of the function `setDimension` is as follows:

```
void boxType::setDimension(double l, double w, double h)
{
    rectangleType::setDimension(l, w);

    if (h >= 0)
        height = h;
    else
        height = 0;
}
```

Notice that in the preceding definition of the function `setDimension`, a call to the member function `setDimension` of the `class rectangleType` is preceded by the name of the class and the scope resolution operator, even though the `class boxType` overloads—not overrides—the function `setDimension`.

The definition of the function `getHeight` is as follows:

```
double boxType::getHeight() const
{
    return height;
}
```

The member function `area` of the `class boxType` determines the surface area of a box. To determine the surface area of a box, we need to access the length and width of the box, which are declared as `private` members of the `class rectangleType`. Therefore, we use the member functions `getLength` and `getWidth` of the `class rectangleType` to retrieve the length and width, respectively. Because the `class boxType` does not contain any member functions that have the names `getLength` or `getWidth`, we call these member functions of the `class rectangleType` without using the name of the base class.

```
double boxType::area() const
{
    return 2 * (getLength() * getWidth()
                + getLength() * height
                + getWidth() * height);
}
```

The member function `volume` of the `class boxType` determines the volume of a box. To determine the volume of a box, you multiply the length, width, and height of the box, or multiply the area of the base of the box by its height. Let us write the definition of the member function `volume` by using the second alternative. To do this, you can use the member function `area` of the `class rectangleType` to determine the area of the base. Because the `class boxType` overrides the member function `area`, to specify a call to the member function `area` of the `class rectangleType`, we use the name of the base class and the scope resolution operator, as shown in the following definition:

```
double boxType::volume() const
{
    return rectangleType::area() * height;
}
```

In the next section, we discuss how to specify a call to the constructor of the base class when writing the definition of a constructor of the derived class.

Constructors of Derived and Base Classes

A derived class can have its own `private` member variables, and so a derived class can explicitly include its own constructors. A constructor typically serves to initialize the member variables. When we declare a derived class object, this object inherits the members of the base class, but the derived class object cannot directly access the `private` (data) members of the base class. The same is true for the member functions of a derived class. That is, the member functions of a derived class cannot directly access the `private` members of the base class.

As a consequence, the constructors of a derived class can (directly) initialize only the (`public` data) members inherited from the base class of the derived class. Thus, when a derived class object is declared, it must also automatically execute one of the constructors of the base class. Because constructors cannot be called like other functions, the execution of a derived class constructor must trigger the execution of one of the base class constructors. This is, in fact, what happens. To make this explicit, a call to the base class constructor is specified in the *heading of the definition* of a derived class constructor.

In the preceding section, we defined the `class rectangleType` and derived the `class boxType` from it. Moreover, we illustrated how to override a member function of the `class rectangleType`. Let us now discuss how to write the definitions of the constructors of the `class boxType`.

The `class rectangleType` has two constructors and two member variables. The `class boxType` has three member variables: `length`, `width`, and `height`. The member variables `length` and `width` are inherited from the `class rectangleType`.

First, let us write the definition of the default constructor of the `class boxType`. Recall that, if a class contains the default constructor and no values are specified when the object is declared, the default constructor executes and initializes the object. Because the `class rectangleType` contains the default constructor, when writing the definition of the default constructor of the `class boxType`, we do not specify any constructor of the base class.

```
boxType::boxType()
{
    height = 0.0;
}
```

Next, we discuss how to write the definitions of constructors with parameters. To trigger the execution of a constructor (with parameters) of the base class, you specify the name of a constructor of the base class with the parameters in the heading of the definition of the constructor of the derived class.

Consider the following definition of the constructor with parameters of the `class boxType`:

```
boxType::boxType(double l, double w, double h)
         : rectangleType(l, w)
{
    if (h >= 0)
        height = h;
    else
        height = 0;
}
```

In this definition, we specify the constructor of `rectangleType` with two parameters. When this constructor of `boxType` executes, it triggers the execution of the constructor with two parameters of type `double` of the `class rectangleType`.

Consider the following statements:

```
rectangleType myRectangle(5.0, 3.0);  //Line 1
boxType myBox(6.0, 5.0, 4.0);         //Line 2
```

The statement in Line 1 creates the `rectangleType` object `myRectangle`. Thus, the object `myRectangle` has two member variables: `length` and `width`. The statement in Line 2 creates the `boxType` object `myBox`. Thus, the object `myBox` has three member variables: `length`, `width`, and `height`. See Figure 2-4.

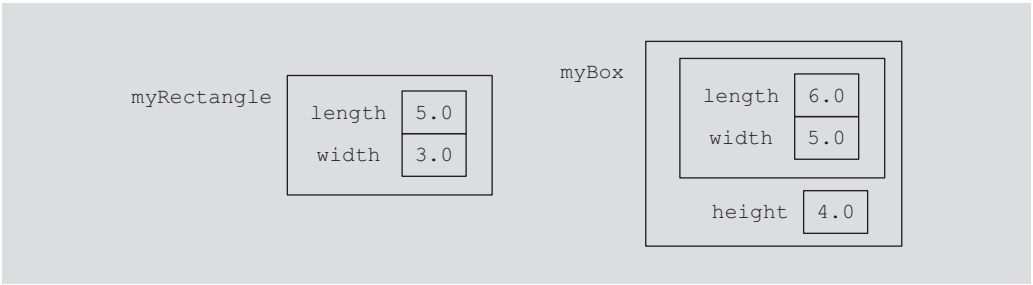


FIGURE 2-4 Objects `myRectangle` and `myBox`

Consider the following statements:

```
myRectangle.print();      //Line 3
cout << endl;             //Line 4
myBox.print();            //Line 5
cout << endl;             //Line 6
```

In the statement in Line 3, the member function `print` of the `class rectangleType` is executed. In the statement in Line 5, the function `print` associated with the `class boxType` is executed. Recall that, if a derived class overrides a member function of the base class, the redefinition applies only to the objects of the derived class. Thus, the output of the statement in Line 3 is:

```
Length = 5.0; Width = 3.0
```

The output of the statement in Line 5 is:

```
Length = 6.0; Width = 5.0; Height = 4.0
```

NOTE

(**Constructors with default parameters and the inheritance hierarchy**) Recall that a class can have a constructor with default parameters. Therefore, a derived class can also have a constructor with default parameters. For example, suppose that the definition of the `class rectangleType` is as given next. (To save space, these definitions have no documentation.)

```
class rectangleType
{
public:
    void setDimension(double l, double w);
    double getLength() const;
    double getWidth() const;
    double area() const;
    double perimeter()const;
    void print() const;
    rectangleType(double l = 0, double w = 0);
      //Constructor with default parameters

private:
    double length;
    double width;
};
```

Suppose the definition of the constructor is:

```
rectangleType::rectangleType(double l, double w)
{
    setDimension(l, w);
}
```

Now suppose that the definition of the `class boxType` is as follows:

```
class boxType: public rectangleType
{
public:
    void setDimension(double l, double w, double h);
    double getHeight()const;
    double area() const;
    double volume() const;
    void print() const;
    boxType(double l = 0, double w = 0, double h = 0);
      //Constructor with default parameters

private:
    double height;
};
```

You can write the definition of the constructor of the `class boxType` as follows:

```
boxType::boxType(double l, double w, double h)
       : rectangleType(l, w)
{
    if (h >= 0)
        height = h;
    else
        height = 0;
}
```

Notice that this definition also takes care of the default constructor of the `class boxType`.

NOTE

Suppose that a base `class`, `baseClass`, has `private` member variables and constructors. Further suppose that the `class derivedClass` is derived from `baseClass`, and `derivedClass` has no member variables. Therefore, the member variables of `derivedClass` are the ones inherited from `baseClass`. A constructor cannot be called like other functions, and the member variables of `baseClass` cannot be directly accessed by the member functions of `derivedClass`. To guarantee the initialization of the inherited member variables of an object of type `derivedClass`, even though `derivedClass` has no member variables, it must have the appropriate constructors. A constructor (with parameters) of `derivedClass` merely issues a call to a constructor (with parameters) of `baseClass`. Therefore, when you write the definition of the constructor (with parameters) of `derivedClass`, the heading of the definition of the constructor contains a call to an appropriate constructor (with parameters) of `baseClass`, and the body of the constructor is empty—that is, it contains only the opening and closing braces.

EXAMPLE 2-2

Suppose that you want to define a class to group the attributes of an employee. There are both full-time employees and part-time employees. Part-time employees are paid based on the number of hours worked and an hourly rate. Suppose that you want to define a class to keep track of a part-time employee's information such as `name`, `pay rate`, and `hours worked`. You can then print the employee's name together with his or her wages. Because every employee is a person, and Example 1-12 (Chapter 1) defined the `class personType` to store the first name and the last name together with the necessary operations on `name`, we can define a `class partTimeEmployee` based on the `class personType`. You can also redefine the `print` function to print the appropriate information.

```
//*****************************************************************
// Author: D.S. Malik
//
// class partTimeEmployee
// This class is derived from the class personType and it
// specifies the members to implement the properties of a
// part-time employee.
//*****************************************************************

class partTimeEmployee: public personType
{
public:
    void print() const;
      //Function to output the first name, last name, and
      //the wages.
      //Postcondition: Outputs: firstName lastName wages are $$$$.$$
```

```
    double calculatePay() const;
      //Function to calculate and return the wages.
      //Postcondition: Pay is calculated and returned.

    void setNameRateHours(string first, string last,
                          double rate, double hours);
      //Function to set the first name, last name, payRate,
      //and hoursWorked according to the parameters.
      //Postcondition: firstName = first; lastName = last;
      //    payRate = rate; hoursWorked = hours

    partTimeEmployee(string first = "", string last = "",
                     double rate = 0, double hours = 0);
      //Constructor with parameters
      //Sets the first name, last name, payRate, and hoursWorked
      //according to the parameters. If no value is specified,
      //the default values are assumed.
      //Postcondition: firstName = first; lastName = last;
      //    payRate = rate; hoursWorked = hours

private:
    double payRate;     //variable to store the pay rate
    double hoursWorked; //variable to store the hours worked
};
```

Figure 2-5 shows the UML class diagram of the **class partTimeEmployee** and the inheritance hierarchy.

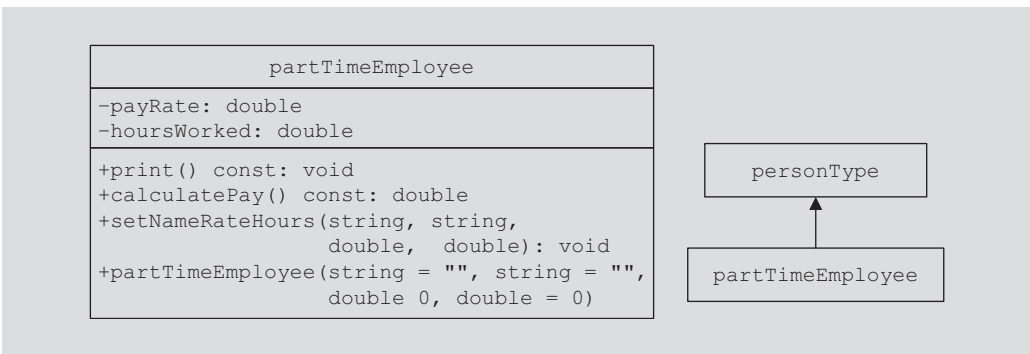


FIGURE 2-5 UML class diagram of the `class partTimeEmployee` and inheritance hierarchy

The definitions of the member functions of the **class partTimeEmployee** are as follows:

```
void partTimeEmployee::print() const
{
    personType::print();  //print the name of the employee
    cout << "'s wages are: $" << calculatePay() << endl;
}
```

```
double partTimeEmployee::calculatePay() const
{
    return (payRate * hoursWorked);
}

void partTimeEmployee::setNameRateHours(string first,
                         string last, double rate, double hours)
{
    personType::setName(first, last);
    payRate = rate;
    hoursWorked = hours;
}

      //Constructor
partTimeEmployee::partTimeEmployee(string first, string last,
                                   double rate, double hours)
      : personType(first, last)
{
    payRate = rate;
    hoursWorked = hours;
}
```

Header File of a Derived Class

The previous section explained how to derive new classes from previously defined classes. To define new classes, you create new header files. The base classes are already defined, and header files contain their definitions. Thus, to create new classes based on the previously defined classes, the header files of the new classes contain commands that tell the computer where to look for the definitions of the base classes.

Suppose that the definition of the `class personType` is placed in the header file `personType.h`. To create the definition of the `class partTimeEmployee`, the header file—say, `partTimeEmployee.h`—must contain the preprocessor directive:

```
#include "personType.h"
```

before the definition of the `class partTimeEmployee`. To be specific, the header file `partTimeEmployee.h` is as follows:

```
//Header file partTimeEmployee

#include "personType.h"

//*************************************************************
// Author: D.S. Malik
//
// class partTimeEmployee
// This class is derived from the class personType and it
// specifies the members to implement the properties of a
// part-time employee.
//*************************************************************
```

```
class partTimeEmployee: public personType
{
public:
    void print() const;
      //Function to output the first name, last name, and
      //the wages.
      //Postcondition: Outputs: firstName lastName wages are $$$$.$$

    double calculatePay() const;
      //Function to calculate and return the wages.
      //Postcondition: Pay is calculated and returned.

    void setNameRateHours(string first, string last,
                          double rate, double hours);
      //Function to set the first name, last name, payRate,
      //and hoursWorked according to the parameters.
      //Postcondition: firstName = first; lastName = last;
      //    payRate = rate; hoursWorked = hours

    partTimeEmployee(string first = "", string last = "",
                     double rate = 0, double hours = 0);
      //Constructor with parameters
      //Sets the first name, last name, payRate, and hoursWorked
      //according to the parameters. If no value is specified,
      //the default values are assumed.
      //Postcondition: firstName = first; lastName = last;
      //    payRate = rate; hoursWorked = hours

private:
    double payRate;     //variable to store the pay rate
    double hoursWorked; //variable to store the hours worked
};
```

The definitions of the member functions can be placed in a separate file (whose extension is `.cpp`). Recall that to include a system-provided header file, such as `iostream`, in a user program, you enclose the header file between angular brackets; to include a user-defined header file in a program, you enclose the header file between double quotation marks.

Multiple Inclusions of a Header File

The previous section discussed how to create the header file of a derived class. To include a header file in a program, you use the preprocessor command `include`. Recall that before a program is compiled, the preprocessor first processes the program. Consider the following header file:

```
//Header file test.h

const int ONE = 1;
const int TWO = 2;
```

Suppose that the header file `testA.h` includes the file `test.h` to use the identifiers `ONE` and `TWO`. To be specific, suppose that the header file `testA.h` looks like:

```
//Header file testA.h

#include "test.h"
.
.
.
```

Now consider the following program code:

```
//Program headerTest.cpp

#include "test.h"
#include "testA.h"
.
.
.
```

When the program `headerTest.cpp` is compiled, it is first processed by the preprocessor. The preprocessor includes first the header file `test.h` and then the header file `testA.h`. When the header file `testA.h` is included, because it contains the preprocessor directive `#include "test.h"`, the header file `test.h` is included twice in the program. The second inclusion of the header file `test.h` results in compile-time errors, such as the identifier `ONE` already being declared. This problem occurs because the first inclusion of the header file `test.h` has already defined the variables `ONE` and `TWO`. To avoid multiple inclusion of a file in a program, we use certain preprocessor commands in the header file. Let us first rewrite the header file `test.h` using these preprocessor commands, and then explain the meaning of these commands.

```
//Header file test.h

#ifndef H_test
#define H_test
const int ONE = 1;
const int TWO = 2;
#endif
```

a. `#ifndef H_test` means "if not defined `H_test`"
b. `#define H_test` means "define `H_test`"
c. `#endif` means "`end if`"

Here `H_test` is a preprocessor identifier.

The effect of these commands is as follows: If the identifier `H_test` is not defined, we must define the identifier `H_test` and let the remaining statements between `#define` and `#endif` pass through the compiler. If the header file `test.h` is included the second time in the program, the statement `#ifndef` fails and all the statements until `#endif` are skipped. In fact, all header files are written using similar preprocessor commands.

Protected Members of a Class

The **private** members of a class are **private** to the class and cannot be directly accessed outside the class. Only member functions of that class can access the **private** members. As discussed previously, the derived class cannot access **private** members of a class. However, it is sometimes necessary for a derived class to access a **private** member of a base class. If you make a **private** member become **public**, anyone can access that member. Recall that the members of a class are classified into three categories: **public**, **private**, and **protected**. So, for a base class to give access to a member to its derived class and still prevent its direct access outside the class, you must declare that member under the member access specifier **protected**. Thus, the accessibility of a **protected** member of a class is in between **public** and **private**. A derived class can directly access the **protected** member of a base class.

To summarize, if a derived class needs to access a member of a base class, that member of the base class should be declared under the member access specifier **protected**.

Inheritance as public, protected, or private

Suppose **class B** is derived from **class A**. Then **B** cannot directly access the **private** members of **A**. That is, the **private** members of **A** are hidden to **B**. What about the **public** and **protected** members of **A**? This section gives the rules that generally apply when accessing the members of a base class.

Consider the following statement:

```
class B: memberAccessSpecifier A
{
  .
  .
  .
};
```

In this statement, **memberAccessSpecifier** is either **public**, **protected**, or **private**.

1. If **memberAccessSpecifier** is **public**—that is, the inheritance is **public**—then
   a. The **public** members of **A** are **public** members of **B**. They can be directly accessed in **class B**.
   b. The **protected** members of **A** are **protected** members of **B**. They can be directly accessed by the member functions (and **friend** functions) of **B**.
   c. The **private** members of **A** are hidden to **B**. They can be accessed by the member functions (and **friend** functions) of **B** through the **public** or **protected** members of **A**.

2. If `memberAccessSpecifier` is `protected`—that is, the inheritance is `protected`—then
   a. The `public` members of `A` are `protected` members of `B`. They can be accessed by the member functions (and `friend` functions) of `B`.
   b. The `protected` members of `A` are `protected` members of `B`. They can be accessed by the member functions (and `friend` functions) of `B`.
   c. The `private` members of `A` are hidden to `B`. They can be accessed by the member functions (and `friend` functions) of `B` through the `public` or `protected` members of `A`.
3. If `memberAccessSpecifier` is `private`—that is, the inheritance is `private`—then
   a. The `public` members of `A` are `private` members of `B`. They can be accessed by the member functions (and `friend` functions) of `B`.
   b. The `protected` members of `A` are `private` members of `B`. They can be accessed by the member functions (and `friend` functions) of `B`.
   c. The `private` members of `A` are hidden to `B`. They can be accessed by the member functions (and `friend` functions) of `B` through the `public` or `protected` members of `A`.

NOTE The section, "`friend` Functions of Classes" (located later in this chapter) describes the `friend` functions.

Composition

Composition is another way to relate two classes. In composition, one or more members of a class are objects of another class type. Composition is a "has-a" relationship; for example, "every person has a date of birth."

Example 1-12, in Chapter 1, defined a class called `personType`. The `class personType` stores a person's first name and last name. Suppose we want to keep track of additional information for a person, such as a personal ID (for example, a Social Security number) and a date of birth. Because every person has a personal ID and a date of birth, we can define a new class, called `personalInfoType`, in which one of the members is an object of type `personType`. We can declare additional members to store the personal ID and date of birth for the `class personalInfoType`.

First, we define another `class`, `dateType`, to store only a person's date of birth, and then construct the `class personalInfoType` from the classes `personType` and `dateType`. This way, we can demonstrate how to define a new class using two classes.

To define the `class dateType`, we need three data members to store the month, day number, and year. Some of the operations that need to be performed on a date are to set the date and to print the date. The following statements define the `class dateType`:

```
//*************************************************************
// Author: D.S. Malik
//
// class dateType
// This class specifies the members to implement a date.
//*************************************************************

class dateType
{
public:
    void setDate(int month, int day, int year);
      //Function to set the date.
      //The member variables dMonth, dDay, and dYear are set
      //according to the parameters.
      //Postcondition: dMonth = month; dDay = day; dYear = year

    int getDay() const;
      //Function to return the day.
      //Postcondition: The value of dDay is returned.

    int getMonth() const;
      //Function to return the month.
      //Postcondition: The value of dMonth is returned.

    int getYear() const;
      //Function to return the year.
      //Postcondition: The value of dYear is returned.

    void printDate() const;
      //Function to output the date in the form mm-dd-yyyy.

    dateType(int month = 1, int day = 1, int year = 1900);
      //Constructor to set the date
      //The member variables dMonth, dDay, and dYear are set
      //according to the parameters.
      //Postcondition: dMonth = month; dDay = day; dYear = year. If
      //    no values are specified, the default values are used to
      //    initialize the member variables.

private:
    int dMonth; //variable to store the month
    int dDay;   //variable to store the day
    int dYear;  //variable to store the year
};
```

Figure 2-6 shows the UML class diagram of the **class dateType**.

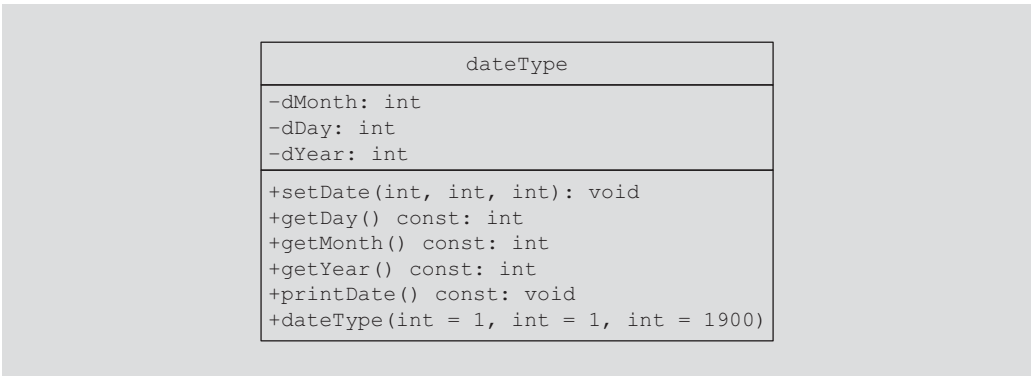


FIGURE 2-6 UML class diagram of the `class dateType`

The definitions of the member functions of the **class dateType** are as follows:

```
void dateType::setDate(int month, int day, int year)
{
    dMonth = month;
    dDay = day;
    dYear = year;
}
```

The definition of the function **setDate**, before storing the date into the data members, does not check whether the date is valid. That is, it does not confirm whether **month** is between **1** and **12**, **year** is greater than **0**, and **day** is valid (for example, for January, **day** should be between **1** and **31**). In Programming Exercise 2 at the end of this chapter, you are asked to rewrite the definition of the function **setDate** so that the date is validated before storing it in the data members.

The definitions of the remaining member functions are as follows:

```
int dateType::getDay() const
{
    return dDay;
}

int dateType::getMonth() const
{
    return dMonth;
}

int dateType::getYear() const
{
    return dYear;
}
```

```
void dateType::printDate() const
{
    cout << dMonth << "-" << dDay << "-" << dYear;
}

    //Constructor with parameters
dateType::dateType(int month, int day, int year)
{
    setDate(month, day, year);
}
```

Because the constructor uses the function `setDate` before storing the date into the data members, the constructor also does not check whether the date is valid. In Programming Exercise 2 at the end of this chapter, when you rewrite the definition of the function `setDate` to validate the date, and the constructor uses the function `setDate`, the date set by the constructor will also be validated.

Next, we give the definition of the `class personalInfoType`:

```
//***********************************************************
// Author: D.S. Malik
//
// class personalInfo
// This class specifies the members to implement a person's
// personal information.
//***********************************************************

class personalInfoType
{
public:
    void setPersonalInfo(string first, string last, int month,
                         int day, int year, int ID);
      //Function to set the personal information.
      //The member variables are set according to the
      //parameters.
      //Postcondition: firstName = first; lastName = last;
      //    dMonth = month; dDay = day; dYear = year;
      //    personID = ID;

    void printPersonalInfo () const;
      //Function to print the personal information.

    personalInfoType(string first = "", string last = "",
                     int month = 1, int day = 1, int year = 1900,
                     int ID = 0);
      //Constructor
      //The member variables are set according to the parameters.
      //Postcondition: firstName = first; lastName = last;
      //    dMonth = month; dDay = day; dYear = year;
      //    personID = ID;
      //    If no values are specified, the default values are
      //    used to initialize the member variables.
```

```
private:
    personType name;
    dateType bDay;
    int personID;
};
```


Figure 2-7 shows the UML class diagram of the **class personalInfoType** and composition (aggregation).

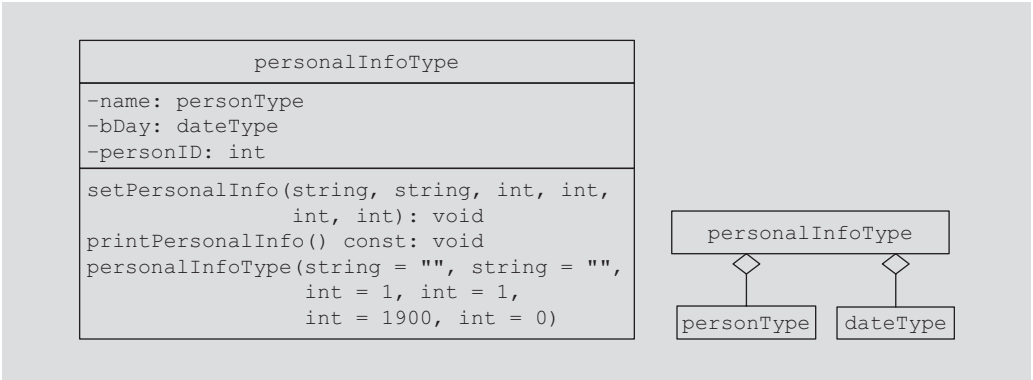


FIGURE 2-7 UML class diagram of the `class personalInfoType` and composition (aggregation)

Before we give the definition of the member functions of the **class personalInfoType**, let us discuss how the constructors of the objects **bDay** and **name** are invoked.

Recall that a class constructor is automatically executed when a class object enters its scope. Suppose that we have the following statement:

```
personalInfoType student;
```

When the object **student** enters its scope, the objects **bDay** and **name**, which are members of **student**, also enter their scopes; as a result, one of their constructors is executed. We therefore need to know how to pass arguments to the constructors of the member objects (that is, **bDay** and **name**). Recall that constructors do not have a type and so cannot be called like other functions. The arguments to the constructor of a member object (such as **bDay**) are specified in the heading part of the definition of the constructor of the class. The following statements illustrate how to pass arguments to the constructors of the member objects:

```
personalInfoType::personalInfoType(string first, string last, int month,
                                   int day, int year, int ID)
        : name(first, last), bDay(month, day, year)
{
    .
    .
    .
}
```

Member objects of a class are constructed (that is, initialized) in the order they are declared (not in the order they are listed in the constructor's member initialization list) and before the enclosing class objects are constructed. Thus, in our case, the object `name` is initialized first, then `bDay`, and, finally, `student`.

The definitions of the member functions of the `class personalInfoType` are as follows:

```
void personalInfoType::setPersonalInfo(string first, string last,
                           int month, int day, int year, int ID)
{
    name.setName(first,last);
    bDay.setDate(month,day,year);
    personID = ID;
}

void personalInfoType::printPersonalInfo() const
{
    name.print();
    cout << "'s date of birth is ";
    bDay.printDate();
    cout << endl;
    cout << "and personal ID is " << personID;
}

personalInfoType::personalInfoType(string first, string last,
                           int month, int day, int year, int ID)
         : name(first, last), bDay(month, day, year)
{
    personID = ID;
}
```

In the case of inheritance, use the class name to invoke the base class's constructor. In the case of composition, use the member object name to invoke its own constructor.

Polymorphism: Operator and Function Overloading

In Chapter 1, you learned how classes in C++ are used to combine data and operations on that data in a single entity. The ability to combine data and operations is called **encapsulation**. It is the first principle of object-oriented design (OOD). Chapter 1 defined the abstract data type (ADT) and described how classes in C++ implement ADTs. The first section of this chapter discussed how new classes can be derived from existing classes through the mechanism of inheritance. Inheritance, the second principle of OOD, encourages code reuse.

The remainder of this chapter discusses the third principle of OOD—polymorphism. First we discuss polymorphism via **operator overloading**, and then via **templates**. Templates enable the programmer to write generic codes for related functions and classes. We will simplify function overloading through the use of templates, called **function templates**.

Operator Overloading

This section describes how operators are loaded in C++. But first let us see why you would want to overload operators.

Why Operator Overloading Is Needed

Chapter 1 defined and implemented the `class clockType`. It also showed how you can use the `class clockType` to represent the time of day in a program. Let us review some of the characteristics of the `class clockType`.

Consider the following statements:

```
clockType myClock(8,23,34);
clockType yourClock(4,5,30);
```

The first statement declares `myClock` to be an object of type `clockType` and initializes the data members `hr`, `min`, and `sec` of `myClock` to `8`, `23`, and `34`, respectively. The second statement declares `yourClock` to be an object of type `clockType` and initializes the data members `hr`, `min`, and `sec` of `yourClock` to `4`, `5`, and `30`, respectively.

Now consider the following statements:

```
myClock.printTime();
myClock.incrementSeconds();
if (myClock.equalTime(yourClock))
.
.
.
```

The first statement prints the value of `myClock` in the form `hr:min:sec`. The second statement increments the value of `myClock` by one second. The third statement checks whether the value of `myClock` is the same as the value of `yourClock`.

These statements do their job. However, if we can use the insertion operator `<<` to output the value of `myClock`, the increment operator `++` to increment the value of `myClock` by one second, and relational operators for comparison, we can enhance the flexibility of C++ considerably and can improve code readability. More specifically, we prefer to use the following statements instead of the previous ones:

```
cout << myClock;
myClock++;
if (myClock == yourClock)
.
.
.
```

Recall that the only built-in operations on classes are the assignment operator and the member selection operator. Therefore, other operators cannot be directly applied to class objects. However, C++ allows the programmer to extend the definitions of most of the operators so that operators such as relational operators, arithmetic operators, insertion

operators for data output, and extraction operators for data input can be applied to classes. In C++ terminology, this is called **operator overloading**. In addition to operator overloading, this chapter discusses function overloading.

Operator Overloading

Recall how the arithmetic operator / works. If both operands of / are integers, the result is an integer; otherwise, the result is a floating-point number. Similarly, the stream insertion operator, <<, and the stream extraction operator, >>, are overloaded. The operator << is used as both a stream insertion operator and a left shift operator. The operator >> is used as both a stream extraction operator and a right shift operator. These are examples of operator overloading.

Other examples of overloaded operators are + and -. The results of + and - are different for integer arithmetic, floating-point arithmetic, and pointer arithmetic.

C++ allows the user to overload most of the operators so that the operators can work effectively in a specific application. It does not allow the user to create new operators. Most of the existing operators can be overloaded to manipulate class objects.

To overload an operator, you must write functions (that is, the header and body). The name of the function that overloads an operator is the reserved word `operator` followed by the operator to be overloaded. For example, the name of the function to overload the operator >= is

```
operator>=
```

Operator function: The function that overloads an operator.

Syntax for Operator Functions

The result of an operation is a value; therefore, the operator function is a value-returning function.

The syntax of the heading for an operator function is as follows:

```
returnType operator operatorSymbol(arguments)
```

In C++, `operator` is a reserved word.

Operator overloading provides the same concise expressions for user-defined data types as it does for built-in data types. To overload an operator for a class, you do the following:

1. Include the statement to declare the function to overload the operator (that is, the operator function) in the definition of the class.
2. Write the definition of the operator function.

Certain rules must be followed when you include an operator function in a class definition. These rules are described in the section "Operator Functions as Member Functions and Nonmember Functions," later in this chapter.

Overloading an Operator: Some Restrictions

When overloading an operator, keep the following in mind:

- You cannot change the precedence of an operator.
- The associativity cannot be changed. (For example, the associativity of the arithmetic operator + is from left to right and it cannot be changed.)
- You cannot use default arguments with an overloaded operator.
- You cannot change the number of arguments that an operator takes.
- You cannot create new operators. Only existing operators can be overloaded. The operators that cannot be overloaded are

  `.  .*  ::  ?:  sizeof`
- The meaning of how an operator works with built-in types, such as `int`, remains the same.
- Operators can be overloaded either for objects of the user-defined type, or for a combination of objects of the user-defined type and objects of the built-in type.

The Pointer `this`

A member function of a class can (directly) access the data members of that class for a given object. Sometimes it is necessary for a function member to refer to the object as a whole, rather than the object's individual data members. How do you refer to the object as a whole (that is, as a single unit) in the definition of the member function, especially when the object is not passed as a parameter? Every object of a class maintains a (hidden) pointer to itself, and the name of this pointer is **`this`**. In C++, **`this`** is a reserved word. The pointer **`this`** is available for you to use. When an object invokes a member function, the member function references the pointer **`this`** of the object. For example, suppose that `test` is a class and has a member function called `funcOne`. Further suppose that the definition of `funcOne` looks like the following:

```
test test::funcOne()
{
   .
   .
   .
   return *this;
}
```

If **`x`** and **`y`** are objects of type `test`, the statement

```
y = x.funcOne();
```

copies the value of the object **`x`** into the object **`y`**; that is, the data members of **`x`** are copied into the corresponding data members of **`y`**. When the object **`x`** invokes the function `funcOne`, the pointer `this` in the definition of the member function `funcOne` refers to the object **`x`**, and so `this` means the address of **`x`** and **`*this`** means the value of **`x`**.

The following example illustrates how the pointer **this** works.

EXAMPLE 2-3

In Example 1-12 (in Chapter 1), we designed a class to implement a person's name in a program. Here we extend the definition of the **class personType** to individually set a person's first name and last name, and then return the entire object. The extended definition of the **class personType** is as follows:

```
//***********************************************************
// Author: D.S. Malik
//
// class personType
// This class specifies the members to implement a name.
//***********************************************************

class personType
{
public:
    void print() const;
      //Function to output the first name and last name in
      //the form firstName lastName

    void setName(string first, string last);
      //Function to set firstName and lastName according to the
      //parameters.
      //Postcondition: firstName = first; lastName = last

    personType& setFirstName(string first);
      //Function to set the first name.
      //Postcondition: firstName = first
      //     After setting the first name, a reference to the
      //     object, that is, the address of the object, is
      //     returned.

    personType& setLastName(string last);
      //Function to set the last name.
      //Postcondition: lastName = last
      //     After setting the last name, a reference to the object,
      //     that is, the address of the object, is returned.

    string getFirstName() const;
      //Function to return the first name.
      //Postcondition: The value of firstName is returned.

    string getLastName() const;
      //Function to return the last name.
      //Postcondition: The value of lastName is returned.

    personType(string first = "", string last = "");
      //Constructor
      //Sets firstName and lastName according to the parameters.
      //Postcondition: firstName = first; lastName = last
```

```
private:
    string firstName; //variable to store the first name
    string lastName;  //variable to store the last name
};
```


Notice that in this definition of the **class** **personType**, we replace the default constructor and the constructor with parameters by one constructor with default parameters.

The definitions of the functions **print**, **setTime**, **getFirstName**, **getLastName**, and the constructor is the same as before (see Example 1-12). The definitions of the functions **setFirstName** and **setLastName** are as follows:

```
personType& personType::setLastName(string last)
{
    lastName = last;

    return *this;
}

personType& personType::setFirstName(string first)
{
    firstName = first;

    return *this;
}
```

The following program shows how to use the **class** **personType**. (We assume that the definition of the **class** **personType** is in the file **personType.h**.)

```
//*****************************************************************
// Author: D.S. Malik
// Test Program: class personType
//*****************************************************************

#include <iostream>                                          //Line 1
#include <string>                                            //Line 2
#include "personType.h"                                      //Line 3

using namespace std;                                         //Line 4

int main()                                                   //Line 5
{                                                            //Line 6
    personType student1("Lisa", "Smith");                    //Line 7
    personType student2;                                     //Line 8
    personType student3;                                     //Line 9

    cout << "Line 10 -- Student 1: ";                        //Line 10
    student1.print();                                        //Line 11
    cout << endl;                                            //Line 12

    student2.setFirstName("Shelly").setLastName("Malik");    //Line 13

    cout << "Line 14 -- Student 2: ";                        //Line 14
    student2.print();                                        //Line 15
    cout << endl;                                            //Line 16
```

```
    student3.setFirstName("Cindy");                          //Line 17

    cout << "Line 18 -- Student 3: ";                        //Line 18
    student3.print();                                        //Line 19
    cout << endl;                                            //Line 20

    student3.setLastName("Tomek");                           //Line 21

    cout << "Line 22 -- Student 3: ";                        //Line 22
    student3.print();                                        //Line 23
    cout << endl;                                            //Line 24

    return 0;                                                //Line 25
}                                                            //Line 26
```

Sample Run:

```
Line 10 -- Student 1: Lisa Smith
Line 14 -- Student 2: Shelly Malik
Line 18 -- Student 3: Cindy
Line 22 -- Student 3: Cindy Tomek
```

The statements in Lines 7, 8, and 9 declare and initialize the objects `student1`, `student2`, and `student3`, respectively. The objects `student2` and `student3` are initialized to empty strings. The statement in Line 11 outputs the value of `student1` (see Line 10 in the sample run, which contains the output of Lines 10, 11, and 12). The statement in Line 13 works as follows. In the statement

```
student2.setFirstName("Shelly").setLastName("Malik");
```

first the expression

```
student2.setFirstName("Shelly")
```

is executed because the associativity of the dot operator is from left to right. This expression sets the first name to `"Shelly"` and returns a reference to the object, which is `student2`. Thus, the next expression executed is

```
student2.setLastName("Malik")
```

which sets the last name of `student2` to `"Malik"`. The statement in Line 15 outputs the value of `student2`. The statement in Line 17 sets the first name of the object `student3` to `"Cindy"`, and ignores the value returned. The statement in Line 19 outputs the value of `student3`. Notice the output in Line 18. The output shows only the first name, not the last name, because we have not yet set the last name of `student3`. The last name of `student3` is still empty, which was set by the statement in Line 9 when `student3` was declared. Next, the statement in Line 21 sets the last name of `student3`, and the statement in Line 23 outputs the value of `student3`.

Friend Functions of Classes

A **friend function** of a class is a nonmember function of the class, but has access to all the members (`public` or non-`public`) of the class. To make a function as a friend of a class, the reserved word `friend` precedes the function prototype (in the class definition). The word `friend` appears only in the function prototype in the class definition, not in the definition of the friend function.

Consider the following statements:

```
class classIllusFriend
{
    friend void two(/*parameters*/);
      .
      .
      .
};
```

In the definition of the `class classIllusFriend`, `two` is declared as a `friend` of the `class classIllusFriend`. That is, it is a nonmember function of the `class classIllusFriend`. When you write the definition of the function `two`, any object of type `classIllusFriend`—which is either a local variable of `two` or a formal parameter of `two`—can access its `private` members within the definition of the function `two`. (Example 2-4 illustrates this concept.) Moreover, because a `friend` function is not a member of a class, its declaration can be placed within the `private`, `protected`, or `public` part of the class. However, they are typically placed before any member function declaration.

DEFINITION OF A `friend` FUNCTION

When writing the definition of a `friend` function, the name of the class and the scope resolution operator do not precede the name of the `friend` function in the function heading. Also, recall that the word `friend` does not appear in the heading of the `friend` function's definition. Thus, the definition of the function `two` in the previous `class classIllusFriend` is as follows:

```
void friendFunc(/*parameters*/)
{
    .
    .
    .
}
```

Of course, we will place the definition of the `friend` function in the implementation file.

The next section illustrates the difference between a member function and a nonmember function (`friend` function), when we overload some of the operators for a specific class.

The following example shows how a `friend` function accesses the `private` members of a class.

EXAMPLE 2-4

Consider the following class:

```
class classIllusFriend
{
    friend void friendFunc(classIllusFriend cIFObject);

public:
    void print();
    void setx(int a);

private:
    int x;
};
```

In the definition of the `class classIllusFriend`, `friendFunc` is declared as a `friend` function. Suppose that the definitions of the member functions of the `class classIllusFriend` are as follows:

```
void classIllusFriend::print()
{
    cout << "In class classIllusFriend: x = " << x << endl;
}

void classIllusFriend::setx(int a)
{
    x = a;
}
```

Consider the following definition of the function `friendFunc`:

```
void friendFunc(classIllusFriend cIFObject)          //Line 1
{                                                    //Line 2
    classIllusFriend localTwoObject;                 //Line 3

    localTwoObject.x = 45;                           //Line 4

    localTwoObject.print();                          //Line 5

    cout << "Line 6: In friendFunc accessing "
         << "private member variable " << "x = "
         << localTwoObject.x
         << endl;                                    //Line 6

    cIFObject.x = 88;                                //Line 7

    cIFObject.print();                               //Line 8

    cout << "Line 9: In friendFunc accessing "
         << "private member variable " << "x = "
         << cIFObject.x << endl;                     //Line 9
}                                                    //Line 10
```

The function `friendFunc` contains a formal parameter `cIFObject` and a local variable `localTwoObject`, both of type `classIllusFriend`. In the statement in Line 4, the object `localTwoObject` accesses its `private` member variable `x` and sets its value to `45`. If `friendFunc` is not declared as a `friend` function of the `class classIllusFriend`, this statement would result in a syntax error because an object cannot directly access its `private` members. Similarly, in the statement in Line 7, the formal parameter `cIFObject` accesses its `private` member variable `x` and sets its value to `88`. Once again, this statement would result in a syntax error if `friendFunc` is not declared a `friend` function of the `class classIllusFriend`. The statement in Line 6 outputs the value of the `private` member variable `x` of `localTwoObject` by directly accessing `x`. Similarly, the statement in Line 9 outputs the value of `x` of `cIFObject` by directly accessing it. The function `friendFunc` also prints the value of `x` by using the function `print` (see the statements in Lines 6 and 9).

Now consider the definition of the following function `main`:

```
int main()                                                  //Line 11
{                                                           //Line 12
    classIllusFriend aObject;                               //Line 13

    aObject.setx(32);                                       //Line 14

    cout << "Line 15: aObject.x: ";                         //Line 15
    aObject.print();                                        //Line 16
    cout << endl;                                           //Line 17

    cout << "*~*~*~* Testing friendFunc *~*~*~*"
         << endl << endl;                                   //Line 18

    friendFunc(aObject);                                    //Line 19

    return 0;                                               //Line 20
}                                                           //Line 21
```

Sample Run:

```
Line 15: aObject.x: In class classIllusFriend: x = 32

*~*~*~* Testing friendFunc *~*~*~*

In class classIllusFriend: x = 45
Line 6: In friendFunc accessing private member variable x = 45
In class classIllusFriend: x = 88
Line 9: In friendFunc accessing private member variable x = 88
```

For the most part, the output is self-explanatory. The statement in Line 19 calls the function `friendFunc` (a `friend` function of the `class classIllusFriend`) and passes the object `aObject` as an actual parameter. Notice that the function `friendFunc` generates the four lines of output.

Operator Functions as Member Functions and Nonmember Functions

Earlier in this chapter we stated that certain rules must be followed when you include an operator function in the definition of a class. This section describes these rules.

Most operator functions can be either member functions or nonmember functions—that is, `friend` functions of a class. To make an operator function be a member or nonmember function of a class, keep the following in mind:

1. The function that overloads any of the operators `()`, `[]`, `->`, or `=` for a class must be declared as a member of the class.
2. Suppose that an operator `op` is overloaded for a class—say, `opOverClass`. (Here, `op` stands for an operator that can be overloaded, such as `+` or `>>`.)
   a. If the leftmost operand of `op` is an object of a different type (that is, not of type `opOverClass`), the function that overloads the operator `op` for `opOverClass` must be a nonmember—that is, a friend of the `class opOverClass`.
   b. If the operator function that overloads the operator `op` for the `class opOverClass` is a member of the `class opOverClass`, then when applying `op` on objects of type `opOverClass`, the leftmost operand of `op` must be of type `opOverClass`.

You must follow these rules when including an operator function in a class definition.

You will see later in this chapter that functions that overload the insertion operator, `<<`, and the extraction operator, `>>`, for a class must be nonmembers—that is, they must be `friend` functions of the class.

Except for certain operators noted previously, operators can be overloaded either as member functions or as nonmember functions. The following discussion shows the difference between these two types of functions.

To facilitate our discussion of operator overloading, we will use the `class rectangleType`, defined earlier in this chapter. Also, suppose that you have the following statements:

```
rectangleType myRectangle;
rectangleType yourRectangle;
rectangleType tempRect;
```

That is, `myRectangle`, `yourRectangle`, and `tempRect` are objects of type `rectangleType`.

C++ consists of both binary and unary operators. It also has a ternary operator, which *cannot* be overloaded. The next few sections discuss how to overload various binary and unary operators.

Overloading Binary Operators

Suppose that # represents a binary operator (arithmetic, such as +; or relational, such as ==) that is to be overloaded for the `class rectangleType`. This operator can be overloaded as either a member function of the class or as a **`friend`** function. We describe both ways to overload this operator.

OVERLOADING THE BINARY OPERATORS AS MEMBER FUNCTIONS

Suppose that # is overloaded as a member function of the `class rectangleType`. The name of the function to overload # for the `class rectangleType` is

```
operator#
```

Because `myRectangle` and `yourRectangle` are objects of type `rectangleType`, you can perform the following operation:

```
myRectangle # yourRectangle
```

The compiler translates this expression into the following expression:

```
myRectangle.operator#(yourRectangle)
```

This expression clearly shows that the function `operator#` has only one parameter, which is `yourRectangle`.

Because `operator#` is a member of the `class rectangleType` and `myRectangle` is an object of type `rectangleType`, in the previous statement, `operator#` has direct access to the `private` members of the object `myRectangle`. Thus, the first parameter of `operator#` is the object that is invoking the function `operator#`, and the second parameter is passed as a parameter to this function.

GENERAL SYNTAX TO OVERLOAD THE BINARY (ARITHMETIC OR RELATIONAL) OPERATORS AS MEMBER FUNCTIONS

This section describes the general form of the functions to overload the binary operators as member functions of a class.

Function Prototype (to be included in the definition of the class):

```
returnType operator#(const className&) const;
```

where # stands for the binary operator, arithmetic or relational, to be overloaded; `returnType` is the type of value returned by the function; and `className` is the name of the class for which the operator is being overloaded.

Function Definition:

```
returnType className::operator#
                      (const className& otherObject) const
{
    //algorithm to perform the operation

    return value;
}
```

NOTE The return type of the function that overloads a relational operator is **bool**.

EXAMPLE 2-5

Let us overload +, *, ==, and != for the **class rectangleType**. These operators are overloaded as member functions.

```
class rectangleType
{
public:
    void setDimension(double l, double w);
    double getLength() const;
    double getWidth() const;
    double area() const;
    double perimeter() const;
    void print() const;

    rectangleType operator+(const rectangleType&) const;
      //Overload the operator +
    rectangleType operator*(const rectangleType&) const;
      //Overload the operator *

    bool operator==(const rectangleType&) const;
      //Overload the operator ==
    bool operator!=(const rectangleType&) const;
      //Overload the operator !=

    rectangleType();
    rectangleType(double l, double w);

private:
    double length;
    double width;
};
```

The definition of the function operator+ is as follows:

```
rectangleType rectangleType::operator+
                        (const rectangleType& rectangle) const
{
    rectangleType tempRect;

    tempRect.length = length + rectangle.length;
    tempRect.width = width + rectangle.width;

    return tempRect;
}
```

Notice that `operator+` adds the corresponding lengths and widths of the two rectangles. The definition of the function `operator*` is as follows:

```
rectangleType rectangleType::operator*
                        (const rectangleType& rectangle) const
{
    rectangleType tempRect;

    tempRect.length = length * rectangle.length;
    tempRect.width = width * rectangle.width;

    return tempRect;
}
```

Notice that `operator*` multiplies the corresponding lengths and widths of the two rectangles.

Two rectangles are equal if their lengths and widths are equal. Therefore, the definition of the function to overload the operator `==` is as follows:

```
bool rectangleType::operator==
                       (const rectangleType& rectangle) const
{
    return (length == rectangle.length &&
            width == rectangle.width);
}
```

Two rectangles are not equal if either their lengths are not equal or their widths are not equal. Therefore, the definition of the function to overload the operator `!=` is as follows:

```
bool rectangleType::operator!=
                        (const rectangleType& rectangle) const
{
    return (length != rectangle.length ||
            width != rectangle.width);
}
```

OVERLOADING THE BINARY OPERATORS (ARITHMETIC OR RELATIONAL) AS NONMEMBER FUNCTIONS

Suppose that `#` represents the binary operator (arithmetic or relational) that is to be overloaded as a *nonmember* function of the `class rectangleType`.

Further suppose that the following operation is to be performed:

```
myRectangle # yourRectangle
```

In this case, the expression is compiled as follows:

```
operator#(myRectangle, yourRectangle)
```

Here, we see that the function `operator#` has two parameters. This expression also clearly shows that the function `operator#` is neither a member of the object `myRectangle` nor a member of the object `yourRectangle`. Both the objects, `myRectangle` and `yourRectangle`, are passed as parameters to the function `operator#`.

To include the operator function `operator#` as a nonmember function of the class in the definition of the class, the reserved word `friend` must appear before the function heading. Also, the function `operator#` must have two parameters.

GENERAL SYNTAX TO OVERLOAD THE BINARY (ARITHMETIC OR RELATIONAL) OPERATORS AS NONMEMBER FUNCTIONS

This section describes the general form of the functions that overload binary operators as nonmember functions of a class.

Function Prototype (to be included in the definition of the class):

```
friend returnType operator#(const className&, const className&);
```

where `#` stands for the binary operator to be overloaded, `returnType` is the type of value returned by the function, and `className` is the name of the class for which the operator is being overloaded.

Function Definition:

```
returnType operator#(const className& firstObject,
                     const className& secondObject)
{
      //algorithm to perform the operation

    return value;
}
```

Overloading the Stream Insertion (<<) and Extraction (>>) Operators

The operator function that overloads the insertion operator, `<<`, or the extraction operator, `>>`, for a class must be a nonmember function of that class for the following reason.

Consider the following expression:

```
cout << myRectangle;
```

In this expression, the leftmost operand of `<<` (that is, `cout`) is an `ostream` object, not an object of type `rectangleType`. Because the leftmost operand of `<<` is not an object of type `rectangleType`, the operator function that overloads the insertion operator for `rectangleType` must be a nonmember function of the `class rectangleType`.

Similarly, the operator function that overloads the stream extraction operator for `rectangleType` must be a nonmember function of the `class rectangleType`.

OVERLOADING THE STREAM INSERTION OPERATOR (<<)

The general syntax to overload the stream insertion operator, <<, for a class is described next.

Function Prototype (to be included in the definition of the class):

```
friend ostream& operator<<(ostream&, const className&);
```

Function Definition:

```
ostream& operator<<(ostream& osObject, const className& cObject)
{
      //local declaration, if any
      //Output the members of cObject.
      //osObject << . . .

      //Return the stream object.
    return osObject;
}
```

In this function definition:

- Both parameters are reference parameters.
- The first parameter—that is, `osObject`—is a reference to an `ostream` object.
- The second parameter is a `const` reference to a particular class.
- The function return type is a reference to an `ostream` object.

OVERLOADING THE STREAM EXTRACTION OPERATOR (>>)

The general syntax to overload the stream extraction operator, >>, for a class is described next.

Function Prototype (to be included in the definition of the class):

```
friend istream& operator>>(istream&, className&);
```

Function Definition:

```
istream& operator>>(istream& isObject, className& cObject)
{
      //local declaration, if any
      //Read the data into cObject.
      //isObject >> . . .

      //Return the stream object.
    return isObject;
}
```

We note the following in this function definition.

- Both parameters are reference parameters.
- The first parameter—that is, `isObject`—is a reference to an `istream` object.

- The second parameter is usually a reference to a particular class. The data read will be stored in the object.
- The function return type is a reference to an `istream` object.

Example 2-6 shows how the stream insertion and extraction operators are overloaded for the `class rectangleType`. We also show how to overload arithmetic and relational operators as member functions of the class.

EXAMPLE 2-6

The definition of the `class rectangleType` and the definitions of the operator functions are as follows:

```
#include <iostream>

using namespace std;

class rectangleType
{
      //Overload the stream insertion and extraction operators
    friend ostream& operator<< (ostream&, const rectangleType &);
    friend istream& operator>> (istream&, rectangleType &);

public:
    void setDimension(double l, double w);
    double getLength() const;
    double getWidth() const;
    double area() const;
    double perimeter() const;
    void print() const;

    rectangleType operator+(const rectangleType&) const;
      //Overload the operator +
    rectangleType operator*(const rectangleType&) const;
      //Overload the operator *

    bool operator==(const rectangleType&) const;
      //Overload the operator ==
    bool operator!=(const rectangleType&) const;
      //Overload the operator !=

    rectangleType();
    rectangleType(double l, double w);

private:
    double length;
    double width;
};

//The definitions of the functions operator+, operator*, operator==,
//operator!=, and the constructor are the same as in Example 2-5.
```

```
ostream& operator<< (ostream& osObject,
                     const rectangleType& rectangle)
{
    osObject << "Length = "  << rectangle.length
             << "; Width = " << rectangle.width;

    return osObject;
}

istream& operator>> (istream& isObject,
                     rectangleType& rectangle)
{
    isObject >> rectangle.length >> rectangle.width;

    return isObject;
}
```

Consider the following program. (We assume that the definition of the `class rectangleType` is in the header file `rectangleType.h`.)

```
//*****************************************************************
// Author: D.S. Malik
//
// This program shows how to use the modified class rectangleType.
//*****************************************************************

#include <iostream>                                          //Line 1

#include "rectangleType.h"                                   //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    rectangleType myRectangle(23, 45);                       //Line 6
    rectangleType yourRectangle;                             //Line 7

    cout << "Line 8: myRectangle: " << myRectangle
         << endl;                                            //Line 8

    cout << "Line 9: Enter the length and width "
         << "of a rectangle: ";                              //Line 9
    cin >> yourRectangle;                                    //Line 10
    cout << endl;                                            //Line 11

    cout << "Line 12: yourRectangle: "
         << yourRectangle << endl;                           //Line 12

    cout << "Line 13: myRectangle + yourRectangle: "
         << myRectangle + yourRectangle << endl;             //Line 13
    cout << "Line 14: myRectangle * yourRectangle: "
         << myRectangle * yourRectangle << endl;             //Line 14

    return 0;                                                //Line 15
}                                                            //Line 16
```

Sample Run: In this sample run, the user input is shaded.

```
Line 8: myRectangle: Length = 23; Width = 45
Line 9: Enter the length and width of a rectangle: 32 15

Line 12: yourRectangle: Length = 32; Width = 15
Line 13: myRectangle + yourRectangle: Length = 55; Width = 60
Line 14: myRectangle * yourRectangle: Length = 736; Width = 675
```

The statements in Lines 6 and 7 declare and initialize `myRectangle` and `yourRectangle` to be objects of type `rectangleType`. The statement in Line 8 outputs the value of `myRectangle` using `cout` and the insertion operator. The statement in Line 10 inputs the data into `yourRectangle` using `cin` and the extraction operator. The statement in Line 12 outputs the value of `yourRectangle` using `cout` and the insertion operator. The `cout` statement in Line 13 adds the lengths and widths of `myRectangle` and `yourRectangle` and outputs the result. Similarly, the `cout` statement in Line 14 multiplies the lengths and widths of `myRectangle` and `yourRectangle` and outputs the result. The output shows that both the stream insertion and stream extraction operators were overloaded successfully.

OVERLOADING UNARY OPERATORS

The process of overloading unary operators is similar to the process of overloading binary operators. The only difference is that in the case of unary operators, the operator has only one argument; in the case of binary operators, the operator has two operands. Therefore, to overload a unary operator for a class we do the following.

- If the operator function is a member of the class, it has no parameters.
- If the operator function is a nonmember—that is, a `friend` function of the class—it has one parameter.

Operator Overloading: Member Versus Nonmember

The preceding sections discussed and illustrated how to overload operators. Certain operators must be overloaded as member functions of the class, and some must be overloaded as nonmember (`friend`) functions. What about the ones that can be overloaded as either member functions or nonmember functions? For example, the binary arithmetic operator `+` can be overloaded as a member function or a nonmember function. If you overload `+` as a member function, the operator `+` has direct access to the data members of one of the objects, and you need to pass only one object as a parameter. On the other hand, if you overload `+` as a nonmember function, you must pass both objects as parameters. Therefore, overloading `+` as a nonmember could require additional memory and computer time to make a local copy of the data. Thus, for efficiency purposes, wherever possible, you should overload operators as member functions.

PROGRAMMING EXAMPLE: Complex Numbers

A number of the form $a + ib$, where $i^2 = -1$, and a and b are real numbers, is called a complex number. We call a the real part and b the imaginary part of $a + ib$. Complex numbers can also be represented as ordered pairs (a, b). The addition and multiplication of complex numbers is defined by the following rules:

$(a + ib) + (c + id) = (a + c) + i(b + d)$

$(a + ib) * (c + id) = (ac - bd) + i(ad + bc)$

Using the ordered pair notation, these rules are written as follows:

$(a, b) + (c, d) = ((a + c), (b + d))$

$(a, b) * (c, d) = ((ac - bd), (ad + bc))$

C++ has no built-in data type that allows us to manipulate complex numbers. In this example, we will construct a data type, `complexType`, that can be used to process complex numbers. We will overload the stream insertion and stream extraction operators for easy input and output. We will also overload the operators `+` and `*` to perform addition and multiplication of complex numbers. If x and y are complex numbers, we can evaluate expressions such as $x + y$ and $x * y$.

```
#ifndef H_complexNumber
#define H_complexNumber

//*****************************************************************
// Author: D.S. Malik
// class complexType.h
// This class specifies the members to implement a complex number.
//*****************************************************************

#include <iostream>
using namespace std;

class complexType
{
      //Overload the stream insertion and extraction operators
    friend ostream& operator<<(ostream&, const complexType&);
    friend istream& operator>>(istream&, complexType&);

public:
    void setComplex(const double& real, const double& imag);
      //Function to set the complex numbers according to
      //the parameters.
      //Postcondition: realPart = real; imaginaryPart = imag;

    void getComplex(double& real, double& imag) const;
      //Function to retrieve the complex number.
      //Postcondition: real = realPart; imag = imaginaryPart;
```

```
    complexType(double real = 0, double imag = 0);
      //Constructor
      //Initializes the complex number according to the parameters.
      //Postcondition: realPart = real; imaginaryPart = imag;

    complexType operator+
                    (const complexType& otherComplex) const;
      //Overload the operator +

    complexType operator*
                    (const complexType& otherComplex) const;
      //Overload the operator *

    bool operator== (const complexType& otherComplex) const;
      //Overload the operator ==

private:
    double realPart;       //variable to store the real part
    double imaginaryPart;  //variable to store the imaginary part
};

#endif
```

Next, we write the definitions of the functions to implement various operations of the `class complexType`.

The definitions of most of these functions are straightforward. We discuss only the definitions of the functions to overload the stream insertion operator, `<<`, and the stream extraction operator, `>>`.

To output a complex number in the form:

```
(a, b)
```

where `a` is the real part and `b` is the imaginary part, clearly the algorithm is as follows:

a. Output the left parenthesis, `(`.
b. Output the real part.
c. Output the comma.
d. Output the imaginary part.
e. Output the right parenthesis, `)`.

Therefore, the definition of the function `operator<<` is as follows:

```
ostream& operator<<(ostream& osObject, const complexType& complex)
{
    osObject << "(";                    //Step a
    osObject << complex.realPart;       //Step b
    osObject << ", ";                   //Step c
    osObject << complex.imaginaryPart;  //Step d
    osObject << ")";                    //Step e

    return osObject;    //return the ostream object
}
```

Next, we discuss the definition of the function to overload the stream extraction operator, `>>`.

The input is of the form:

```
(3, 5)
```

In this input, the real part of the complex number is `3` and the imaginary part is `5`. Clearly, the algorithm to read this complex number is as follows:

a. Read and discard the left parenthesis.
b. Read and store the real part.
c. Read and discard the comma.
d. Read and store the imaginary part.
e. Read and discard the right parenthesis.

Following these steps, the definition of the function `operator>>` is as follows:

```
istream& operator>>(istream& isObject, complexType& complex)
{
    char ch;

    isObject >> ch;                     //Step a
    isObject >> complex.realPart;       //Step b
    isObject >> ch;                     //Step c
    isObject >> complex.imaginaryPart;  //Step d
    isObject >> ch;                     //Step e

    return isObject;    //return the istream object
}
```

The definitions of the other functions are as follows:

```
bool complexType::operator==
                    (const complexType& otherComplex) const
{
    return (realPart == otherComplex.realPart &&
            imaginaryPart == otherComplex.imaginaryPart);
}
```

```
    //Constructor
complexType::complexType(double real, double imag)
{
    realPart = real;
    imaginaryPart = imag;
}

    //Function to set the complex number after the object
    //has been declared.
void complexType::setComplex(const double& real,
                             const double& imag)
{
    realPart = real;
    imaginaryPart = imag;
}

void complexType::getComplex(double& real, double& imag) const
{
    real = realPart;
    imag = imaginaryPart;
}

       //overload the operator +
complexType complexType::operator+
                        (const complexType& otherComplex) const
{
    complexType temp;

    temp.realPart = realPart + otherComplex.realPart;
    temp.imaginaryPart = imaginaryPart
                         + otherComplex.imaginaryPart;

       return temp;
}

    //overload the operator *
complexType complexType::operator*
                           (const complexType& otherComplex) const
{
    complexType temp;

    temp.realPart = (realPart * otherComplex.realPart) -
                (imaginaryPart * otherComplex.imaginaryPart);
    temp.imaginaryPart = (realPart * otherComplex.imaginaryPart)
                  + (imaginaryPart * otherComplex.realPart);
    return temp;
}
```

The following program illustrates the use of the `class complexType`:

```
//***********************************************************
// Author: D.S. Malik
//
// This program shows how to use the class complexType.
//***********************************************************

#include <iostream>                                    //Line 1
#include "complexType.h"                               //Line 2

using namespace std;                                   //Line 3

int main()                                             //Line 4
{                                                      //Line 5
    complexType num1(23, 34);                          //Line 6
    complexType num2;                                  //Line 7
    complexType num3;                                  //Line 8

    cout << "Line 9: Num1 = " << num1 << endl;         //Line 9
    cout << "Line 10: Num2 = " << num2 << endl;        //Line 10

    cout << "Line 11: Enter the complex number "
         << "in the form (a, b): ";                    //Line 11
    cin >> num2;                                       //Line 12
    cout << endl;                                      //Line 13

    cout << "Line 14: New value of num2 = "
         << num2 << endl;                              //Line 14

    num3 = num1 + num2;                                //Line 15

    cout << "Line 16: Num3 = " << num3 << endl;        //Line 16

    cout << "Line 17: " << num1 << " + " << num2
         << " = " << num1 + num2 << endl;              //Line 17

    cout << "Line 18: " << num1 << " * " << num2
         << " = " << num1 * num2 << endl;              //Line 18

    return 0;                                          //Line 19
}                                                      //Line 20
```

Sample Run: In this sample run, the user input is shaded.

```
Line 9: Num1 = (23, 34)
Line 10: Num2 = (0, 0)
Line 11: Enter the complex number in the form (a, b): (3, 4)

Line 14: New value of num2 = (3, 4)
Line 16: Num3 = (26, 38)
Line 17: (23, 34) + (3, 4) = (26, 38)
Line 18: (23, 34) * (3, 4) = (-67, 194)
```

Function Overloading

The previous section discussed operator overloading. Operator overloading provides the programmer with the same concise notation for user-defined data types as the operator has with built-in types. The types of arguments used with an operator determine the action to take.

Similar to operator overloading, C++ allows the programmer to overload a function name. Recall that a class can have more than one constructor, but all constructors of a class have the same name, which is the name of the class. This case is an example of overloading a function.

Overloading a function refers to the creation of several functions with the same name. However, if several functions have the same name, every function must have a different set of parameters. The types of parameters determine which function to execute.

Suppose you need to write a function that determines the larger of two items. Both items can be integers, floating-point numbers, characters, or strings. You could write several functions as follows:

```
int largerInt(int x, int y);
char largerChar(char first, char second);
double largerDouble(double u, double v);
string largerString(string first, string second);
```

The function `largerInt` determines the larger of the two integers, the function `largerChar` determines the larger of the two characters, and so on. These functions all perform similar operations. Instead of giving different names to these functions, you can use the same name—say, `larger`—for each function; that is, you can overload the function `larger`. Thus, you can write the previous function prototypes simply as

```
int larger(int x, int y);
char larger(char first, char second);
double larger(double u, double v);
string larger(string first, string second);
```

If the call is `larger(5,3)`, for example, the first function executes. If the call is `larger('A', '9')`, the second function executes, and so on.

For function overloading to work, we must give the definition of each function. The next section teaches you how to overload functions with a single code segment and leave the job of generating code for separate functions to the compiler.

Templates

Templates are very powerful features of C++. By using templates, you can write a single code segment for a set of related functions, called a **function template**, and for related classes, called a **class template**. The syntax we use for templates is as follows:

```
template <class Type>
declaration;
```

where Type is the type of data, and declaration is either a function declaration or a class declaration. In C++, template is a reserved word. The word class in the heading refers to any user-defined type or built-in type. Type is referred to as a formal parameter to the template.

Just as variables are parameters to functions, types (that is, data types) are parameters to templates.

Function Templates

In the section, "Function Overloading" (located earlier in this chapter), when we introduced function overloading, the function larger was overloaded to find the larger of two integers, characters, floating-point numbers, or strings. To implement the function larger, we need to write four function definitions for the data type: one for int, one for char, one for double, and one for string. However, the body of each function is similar. C++ simplifies the process of overloading functions by providing function templates.

The syntax of the function template is as follows:

```
template <class Type>
function definition;
```

where Type is referred to as a formal parameter of the template. It is used to specify the type of parameters to the function and the return type of the function, and to declare variables within the function.

The statements

```
template <class Type>
Type larger(Type x, Type y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```

define a function template larger, which returns the larger of two items. In the function heading, the type of the formal parameters x and y is Type, which will be specified by the type of the actual parameters when the function is called. The statement

```
cout << larger(5, 6) << endl;
```

is a call to the function template larger. Because 5 and 6 are of type int, the data type int is substituted for Type and the compiler generates the appropriate code.

If we omit the body of the function in the function template definition, the function template, as usual, is the prototype.

The following example illustrates the use of function templates.

EXAMPLE 2-7

This example uses the function template `larger` to determine the larger of the two items.

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to write and use a template in a
// program.
//*****************************************************************

#include <iostream>                                              //Line 1
#include <string>                                                //Line 2

using namespace std;                                             //Line 3

template <class Type>                                            //Line 4
Type larger(Type x, Type y);                                     //Line 5

int main()                                                       //Line 6
{                                                                //Line 7
    cout << "Line 8: Larger of 5 and 6 = "
         << larger(5, 6) << endl;                                //Line 8

    cout << "Line 9: Larger of A and B = "
         << larger('A','B') << endl;                             //Line 9

    cout << "Line 10: Larger of 5.6 and 3.2 = "
         << larger(5.6, 3.2) << endl;                            //Line 10

    string str1 = "Hello";                                       //Line 11
    string str2 = "Happy";                                       //Line 12

    cout << "Line 13: Larger of " << str1 << " and "
         << str2 << " = " << larger(str1, str2) << endl;         //Line 13

    return 0;                                                    //Line 14
}                                                                //Line 15

template <class Type>
Type larger(Type x, Type y)
{
    if (x >= y)
        return x;
    else
        return y;
}
```

Output

```
Line 8: Larger of 5 and 6 = 6
Line 9: Larger of A and B = B
Line 10: Larger of 5.6 and 3.2 = 5.6
Line 13: Larger of Hello and Happy = Hello
```


Class Templates

Like function templates, class templates are used to write a single code segment for a set of related classes. For example, in Chapter 1, we defined a list as an ADT; our list element type there was **int**. If the list element type changes from **int** to, say, **char**, **double**, or **string**, we need to write separate classes for each element type. For the most part, the operations on the list and the algorithms to implement those operations remain the same. Using class templates, we can create a generic **class listType**, and the compiler can generate the appropriate source code for a specific implementation.

The syntax we use for a class template is as follows:

```
template <class Type>
class declaration
```

Class templates are called **parameterized types** because, based on the parameter type, a specific class is generated. For example, if the template parameter type is **int**, we can generate a list to process integers; if the parameter type is **string**, we can generate a list to process strings.

A class template for the ADT **listType** is defined as follows:

```
template <class elemType>
class listType
{
public:
    bool isEmpty();
    bool isFull();
    void search(const elemType& searchItem, bool& found);
    void insert(const elemType& newElement);
    void remove(const elemType& removeElement);
    void destroyList();
    void printList();

    listType();

private:
    elemType list[100];  //array to hold the list elements
    int length;          //variable to store the number
                         //of elements in the list
};
```

This definition of the class template `listType` is a generic definition and includes only the basic operations on a list. To derive a specific list from this list and to add or rewrite the operations, we declare the array containing the list elements and the length of the list as **`protected`**.

Next, we describe a specific list. Suppose that you want to create a list to process integer data. The statement

```
listType<int> intList;          //Line 1
```

declares `intList` to be a list of `100` components, with each component being of type **`int`**. Similarly, the statement

```
listType<string> stringList;     //Line 2
```

declares `stringList` to be a list of `100` components, with each component being of type `string`.

In the statements in Lines 1 and 2, `listType<int>` and `listType<string>` are referred to as **template instantiations** or **instantiations** of the class template `listType<elemType>`, where `elemType` is the class parameter in the template header. A template instantiation can be created with either a built-in or user-defined type.

The function members of a class template are considered function templates. Thus, when giving the definitions of function members of a class template, we must follow the definition of the function template. For example, the definition of the member `insert` of the **`class`** `listType` is as follows:

```
template<class elemType>
void listType<elemType>::insert(const elemType& newElement)
{
  .
  .
  .
}
```

In the heading of the member function's definition, `elemType` specifies the data type of the list elements.

Header File and Implementation File of a Class Template

Until now, we have placed the definition of the class (in the specification file) and the definition of the member functions (in the implementation file) in separate files. The object code was generated from the implementation file (independently of any client code) and linked with the client code. This strategy does not work with class templates. Passing parameters to a function has an effect at run time, whereas passing a parameter to a class template has an effect at compile time. Because the actual parameter to a class is specified in the client code, and because the compiler cannot instantiate a function template without the actual parameter to the template, we can no longer compile the implementation file independently of the client code.

This problem has several possible solutions. We could put the class definition and the definitions of the function templates directly in the client code, or we could put the class definition and the definitions of the function templates together in the same header file. Another alternative is to put the class definition and the definitions of the functions in separate files (as usual), but include a directive to the implementation file at the end of the header file (that is, the specification file). In either case, the function definitions and the client code are compiled together. For illustrative purposes, we will put the class definition and the function definitions in the same header file.

QUICK REVIEW

1. Inheritance and composition are meaningful ways to relate two or more classes.
2. Inheritance is an "is a" relationship.
3. Composition is a "has a" relationship.
4. In single inheritance, the derived class is derived from only one existing class, called the base class.
5. In multiple inheritance, a derived class is derived from more than one base class.
6. The `private` members of a base class are `private` to the base class. The derived class cannot directly access them.
7. The `public` members of a base class can be inherited either as `public`, `protected`, or `private` by the derived class.
8. A derived class can redefine the function members of a base class, but this redefinition applies only to the objects of the derived class.
9. A call to a base class's constructor is specified in the heading of the definition of the derived class's constructor.
10. When initializing the object of a derived class, the constructor of the base class is executed first.
11. Review the inheritance rules given in this chapter.
12. In composition, a member of a class is an object of another class.
13. In composition, a call to the constructor of the member objects is specified in the heading of the definition of the class's constructor.
14. The three basic principles of OOD are encapsulation, inheritance, and polymorphism.
15. An operator that has different meanings with different data types is said to be overloaded.
16. In C++, `<<` is used as a stream insertion operator and as a left shift operator. Similarly, `>>` is used as a stream extraction operator and as a right shift operator. Both are examples of operator overloading.

17. The function that overloads an operator is called an operator function.
18. The syntax of the heading of the operator function is

    ```
    returnType operator operatorSymbol(parameters)
    ```
19. In C++, `operator` is a reserved word.
20. Operator functions are value-returning functions.
21. Except for the assignment operator and the member selection operator, to use an operator on class objects, that operator must be overloaded.
22. Operator overloading provides the same concise notation for user-defined data types as is available with built-in data types.
23. When an operator is overloaded, its precedence cannot be changed, its associativity cannot be changed, default arguments cannot be used, the number of arguments that the operator takes cannot be changed, and the meaning of how an operator works with built-in data types remains the same.
24. It is not possible to create new operators. Only existing operators can be overloaded.
25. Most C++ operators can be overloaded.
26. The operators that cannot be overloaded are `.`, `.*`, `::`, `?:`, and `sizeof`.
27. The pointer `this` refers to the object as a whole.
28. The operator function that overloads the operators `()`, `[]`, `->`, or `=` must be a member of a class.
29. A `friend` function is a nonmember of a class.
30. The heading of a `friend` function is preceded by the word `friend`.
31. In C++, `friend` is a reserved word.
32. If an operator function is a member of a class, the leftmost operand of the operator must be a class object (or a reference to a class object) of that operator's class.
33. The binary operator function as a member of a class has only one parameter; as a nonmember of a class, it has two parameters.
34. The operator functions that overload the stream insertion operator, `<<`, and the stream extraction operator, `>>`, for a class must be `friend` functions of that class.
35. In C++, a function name can be overloaded.
36. Every instance of an overloaded function has different sets of parameters.
37. In C++, `template` is a reserved word.
38. Using templates, you can write a single code segment for a set of related functions—called the function template.
39. Using templates, you can write a single code segment for a set of related classes—called the class template.

40. A syntax of a template is

```
template <class elemType>
declaration;
```

where `elemType` is a user-defined identifier, which is used to pass types (that is, data types) as parameters, and `declaration` is either a function or a class. The word `class` in the heading refers to any user-defined data type or built-in data type.

41. Class templates are called parameterized types.

42. In a class template, the parameter `elemType` specifies how a generic class template is to be customized to form a specific class.

43. Suppose `cType` is a class template and `func` is a member function of `cType`. The heading of the function definition of `func` is

```
template <class elemType >
funcType cType<elemType>::func(formal parameters)
```

where `funcType` is the type of the function, such as `void`.

44. Suppose `cType` is a class template, which can take `int` as a parameter. The statement

```
cType<int> x;
```

declares `x` to be an object of type `cType`, and the type passed to the `class cType` is `int`.

EXERCISES

1. Mark the following statements as true or false.
   a. The constructor of a derived class specifies a call to the constructor of the base class in the heading of the function definition.
   b. The constructor of a derived class specifies a call to the constructor of the base class using the name of the class.
   c. Suppose that `x` and `y` are classes, one of the data members of `x` is an object of type `y`, and both classes have constructors. The constructor of `x` specifies a call to the constructor of `y` by using the object name of type `y`.
   d. A derived class must have a constructor.
   e. In C++, all operators can be overloaded for user-defined data types.
   f. In C++, operators cannot be redefined for built-in types.
   g. The function that overloads an operator is called the operator function.
   h. C++ allows users to create their own operators.
   i. The precedence of an operator cannot be changed, but its associativity can be changed.

j. Every instance of an overloaded function has the same number of parameters.

k. It is not necessary to overload relational operators for classes that have only `int` data members.

l. The member function of a `class` template is a function template.

m. When writing the definition of a `friend` function, the keyword `friend` must appear in the function heading.

n. The function heading of the operator function to overload the pre-increment operator `(++)` and the postincrement operator `(++)` is the same because both operators have the same symbols.

2. Draw a class hierarchy in which several classes are derived from a single base class.

3. Suppose that a `class employeeType` is derived from the `class personType` (see Example 1-12, in Chapter 1). Give examples of data and function members that can be added to the `class employeeType`.

4. Explain the difference between the `private` and `protected` members of a class.

5. Consider the following class definition:

```
class aClass
{
public:
    void print() const;
    void set(int, int);
    aClass();
    aClass(int, int);

private:
    int u;
    int v;
};
```

What is wrong with the following `class` definitions?

a.
```
class bClass public aClass
{
public:
    void print();
    void set(int, int, int);
private:
    int z;
};
```

b.
```
class cClass: public aClass
{
public:
    void print();
    int sum();
    cClass();
    cClass(int);
}
```

6. Consider the following statements:

```
class yClass
{
public:
    void one();
    void two(int, int);
    yClass();
private:
    int a;
    int b;
};

class xClass: public yClass
{
public:
    void one();
    xClass();
private:
    int z;
};

yClass y;
xClass x;
```

   a. The **private** members of **yClass** are **public** members of **xClass**. True or False?

   b. Mark the following statements as valid or invalid. If a statement is invalid, explain why.

      i.
```
void yClass::one()
{
    cout << a + b << endl;
}
```

      ii.
```
y.a = 15;
x.b = 30;
```

      iii.
```
void xClass::one()
{
    a = 10;
    b = 15;
    z = 30;
    cout << a + b + z << endl;
}
```

      iv.
```
cout << y.a << " " << y.b << " " << x.z << endl;
```

7. Assume the declaration of Exercise 6.

   a. Write the definition of the default constructor of **yClass** so that the **private** data members of **yClass** are initialized to 0.

   b. Write the definition of the default constructor of **xClass** so that the **private** data members of **xClass** are initialized to 0.

c. Write the definition of the member function `two` of `yClass` so that the **`private`** data member `a` is initialized to the value of the first parameter of `two`, and the **`private`** data member `b` is initialized to the value of the second parameter of `two`.

8. What is wrong with the following code?

```
class classA
{
protected:
    void setX(int a);              //Line 1
     //Postcondition: x = a;       //Line 2
private:                           //Line 3
    int x;                         //Line 4
};
.
.
.
int main()
{
    classA aObject;                //Line 5

    aObject.setX(4);               //Line 6
    return 0;                      //Line 7
}
```

9. Consider the following code:

```
class one
{
public:
    void print() const;
      //Outputs the values of x and y
protected:
    void setData(int u, int v);
      //Postcondition: x = u; y = v;
private:
    int x;
    int y;
};

class two: public one
{
public:
    void setData(int a, int b, int c);
      //Postcondition: x = a; y = b; z = c;
    void print() const;
     //Outputs the values of x, y, and z
private:
    int z;
};
```

a. Write the definition of the function `setData` of the `class two`.

b. Write the definition of the function `print` of the `class two`.

10. What is the output of the following C++ program?

```
#include <iostream>
#include <string>

using namespace std;

class baseClass
{
public:
    void print() const;

    baseClass(string s = " ", int a = 0);
      //Postcondition: str = s; x = a
protected:
    int x;

private:
    string str;
};

class derivedClass: public baseClass
{
public:
    void print() const;

    derivedClass(string s = "", int a = 0, int b = 0);
     //Postcondition: str = s; x = a; y = b

private:
    int y;
};

int main()
{
    baseClass baseObject("This is base class", 2);
    derivedClass derivedObject("DDDDDD", 3, 7);

    baseObject.print();
    derivedObject.print();

    return 0;
}

void baseClass::print() const
{
    cout << x << " " << str << endl;
}

baseClass::baseClass(string s, int a)
{
    str = s;
    x = a;
}
```

```
void derivedClass::print() const
{
    cout << "Derived class: " << y << endl;
    baseClass::print();
}

derivedClass::derivedClass(string s, int a, int b)
             :baseClass("Hello Base", a + b)
{
    y = b;
}
```

11. What is the output of the following program?

```
#include <iostream>

using namespace std;

class baseClass
{
public:
    void print()const;

    int getX();

    baseClass(int a = 0);

protected:
    int x;
};

class derivedClass: public baseClass
{
public:
    void print()const;

    int getResult();

    derivedClass(int a = 0, int b = 0);

private:
    int y;
};


int main()
{
    baseClass baseObject(7);
    derivedClass derivedObject(3,8);

    baseObject.print();
    derivedObject.print();
```

```
    cout << "**** " << baseObject.getX() << endl;
    cout << "#### " << derivedObject.getResult() << endl;

    return 0;
}

void baseClass::print() const
{
    cout << "In base: x = " << x << endl;
}

baseClass::baseClass(int a)
{
    x = a;
}

int baseClass::getX()
{
    return x;
}

void derivedClass::print() const
{
    cout << "In derived: x = " << x << ", y = " << y
         << ", x + y = " << x + y << endl;
}

int derivedClass::getResult()
{
    return x + y;
}

derivedClass::derivedClass(int a, int b)
              :baseClass(a)
{
    y = b;
}
```

12. What is a `friend` function?
13. Suppose that the operator `<<` is to be overloaded for a user-defined `class mystery`. Why must `<<` be overloaded as a `friend` function?
14. Suppose that the binary operator `+` is overloaded as a member function for a `class strange`. How many parameters does the function `operator+` have?
15. Consider the following declaration:

```
class strange
{
.
.
.
};
```

a. Write a statement that shows the declaration in the `class strange` to overload the operator `>>`.

b. Write a statement that shows the declaration in the `class strange` to overload the binary operator `+` as a member function.

c. Write a statement that shows the declaration in the `class strange` to overload the operator `==` as a member function.

d. Write a statement that shows the declaration in the `class strange` to overload the postincrement operator `++` as a member function.

16. Assume the declaration of Exercise 15.

a. Write a statement that shows the declaration in the `class strange` to overload the binary operator `+` as a **`friend`** function.

b. Write a statement that shows the declaration in the `class strange` to overload the operator `==` as a **`friend`** function.

c. Write a statement that shows the declaration in the `class strange` to overload the postincrement operator `++` as a **`friend`** function.

17. Find the error(s) in the following code:

```
class mystery                              //Line 1
{
    ...
    bool operator <= (mystery);            //Line 2
    ...
};

bool mystery::<=(mystery rightObj)     //Line 3
{
    ...
}
```

18. Find the error(s) in the following code:

```
class mystery                                      //Line 1
{
    ...
    bool operator <= (mystery, mystery);     //Line 2
    ...
};
```

19. Find the error(s) in the following code:

```
class mystery                               //Line 1
{
...
friend operator+ (mystery);                 //Line 2
  //Overload the binary operator +
...
};
```

20. How many parameters are required to overload the preincrement operator for a class as a member function?

21. How many parameters are required to overload the preincrement operator for a class as a **friend** function?
22. How many parameters are required to overload the postincrement operator for a class as a member function?
23. How many parameters are required to overload the postincrement operator for a class as a **friend** function?
24. Find the error(s) in the following code:

```
template <class type>     //Line 1
class strange             //Line 2
{
    ...
};

strange<int> s1;          //Line 3
strange<type> s2;         //Line 4
```

25. Consider the following declaration:

```
template <class type>
class strange
{
      ...
private:
    Type a;
    Type b;
};
```

    a. Write a statement that declares `sObj` to be an object of type `strange` such that the `private` data members `a` and `b` are of type `int`.
    b. Write a statement that shows the declaration in the `class strange` to overload the operator `==` as a member function.
    c. Assume that two objects of type `strange` are equal if their corresponding data members are equal. Write the definition of the function `operator==` for the `class strange`, which is overloaded as a member function.

26. Consider the definition of the following function template:

```
template <class Type>
Type surprise(Type x, Type y)
{
    return x + y ;
}
```

What is the output of the following statements?

    a. `cout << surprise(5, 7) << endl;`
    b.
```
string str1 = "Sunny";
string str2 = " Day";
cout << surprise(str1, str2) << endl;
```

27. Consider the definition of the following function template:

```
Template <class Type>
Type funcExp(Type list[], int size)
{
    Type x = list[0];
    Type y = list[size - 1];

    for (int j = 1; j < (size - 1)/2; j++)
    {
        if (x < list[j])
            x = list[j];
        if (y > list[size - 1 -j])
            y = list[size - 1 -j];
    }

    return x + y;
}
```

Further suppose that you have the following declarations:

```
int list[10] = {5,3,2,10,4,19,45,13,61,11};
string strList[] = {"One", "Hello", "Four", "Three", "How", "Six"};
```

What is the output of the following statements?

a. `cout << funcExp(list, 10) << endl;`

b. `cout << funcExp(strList, 6) << endl;`

28. Write the definition of the function template that swaps the contents of two variables.

PROGRAMMING EXERCISES

1. In Chapter 1, the `class clockType` was designed to implement the time of day in a program. Certain applications, in addition to hours, minutes, and seconds, might require you to store the time zone. Derive the **`class extClockType`** from the `class clockType` by adding a data member to store the time zone. Add the necessary member functions and constructors to make the class functional. Also, write the definitions of the member functions and the constructors. Finally, write a test program to test your class.
2. In this chapter, the `class dateType` was designed to implement the date in a program, but the member function `setDate` and the constructor do not check whether the date is valid before storing the date in the data members. Rewrite the definitions of the function `setDate` and the constructor so that the values for the month, day, and year are checked before storing the date into the data members. Add a function member, `isLeapYear`, to check whether a year is a leap year. Moreover, write a test program to test your class.

3. A point in the x-y plane is represented by its x-coordinate and y-coordinate. Design a class, `pointType`, that can store and process a point in the x-y plane. You should then perform operations on the point, such as showing the point, setting the coordinates of the point, printing the coordinates of the point, returning the x-coordinate, and returning the y-coordinate. Also, write a test program to test the various operations on the point.
4. Every circle has a center and a radius. Given the radius, we can determine the circle's area and circumference. Given the center, we can determine its position in the x-y plane. The center of a circle is a point in the x-y plane. Design a `class`, `circleType`, that can store the radius and center of the circle. Because the center is a point in the x-y plane and you designed the class to capture the properties of a point in Programming Exercise 3, you must derive the `class circleType` from the `class pointType`. You should be able to perform the usual operations on a circle, such as setting the radius, printing the radius, calculating and printing the area and circumference, and carrying out the usual operations on the center.
5. Every cylinder has a base and height, where the base is a circle. Design a class, `cylinderType`, that can capture the properties of a cylinder and perform the usual operations on a cylinder. Derive this class from the `class circleType` designed in Programming Exercise 4. Some of the operations that can be performed on a cylinder are as follows: Calculate and print the volume, calculate and print the surface area, set the height, set the radius of the base, and set the center of the base.
6. In Programming Exercise 2, the `class dateType` was designed and implemented to keep track of a date, but it has very limited operations. Redefine the `class dateType` so that it can perform the following operations on a date in addition to the operations already defined:
   a. Set the month.
   b. Set the day.
   c. Set the year.
   d. Return the month.
   e. Return the day.
   f. Return the year.
   g. Test whether the year is a leap year.
   h. Return the number of days in the month. For example, if the date is 3-12-2011, the number of days to be returned is 31 because there are 31 days in March.
   i. Return the number of days passed in the year. For example, if the date is 3-18-2011, the number of days passed in the year is 77. Note that the number of days returned also includes the current day.

j. Return the number of days remaining in the year. For example, if the date is 3-18-2011, the number of days remaining in the year is 288.

k. Calculate the new date by adding a fixed number of days to the date. For example, if the date is 3-18-2011 and the days to be added are 25, the new date is 4-12-2011.

7. Write the definitions of the functions to implement the operations defined for the `class dateType` in Programming Exercise 6.

8. The `class dateType` defined in Programming Exercise 6 prints the date in numerical form. Some applications might require the date to be printed in another form, such as March 24, 2003. Derive the `class extDateType` so that the date can be printed in either form.

   Add a data member to the `class extDateType` so that the month can also be stored in string form. Add a function member to output the month in the string format followed by the year—for example, in the form March 2003.

   Write the definitions of the functions to implement the operations for the `class extDateType`.

9. Using the `class`es `extDateType` (Programming Exercise 8) and `dayType` (Chapter 1, Programming Exercise 2), design the `class calendarType` so that, given the month and the year, we can print the calendar for that month. To print a monthly calendar, you must know the first day of the month and the number of days in that month. Thus, you must store the first day of the month, which is of the form `dayType`, and the month and the year of the calendar. Clearly, the month and the year can be stored in an object of the form `extDateType` by setting the day component of the date to 1, and the month and year as specified by the user. Thus, the `class calendarType` has two data members: an object of type `dayType` and an object of type `extDateType`.

   Design the `class calendarType` so that the program can print a calendar for any month starting January 1, 1500. Note that the day for January 1 of the year 1500 is a Monday. To calculate the first day of a month, you can add the appropriate days to Monday of January 1, 1500.

   For the `class calendarType`, include the following operations:

   a. Determine the first day of the month for which the calendar will be printed. Call this operation `firstDayOfMonth`.

   b. Set the month.

   c. Set the year.

   d. Return the month.

   e. Return the year.

   f. Print the calendar for the particular month.

   g. Add the appropriate constructors to initialize the data members.

10. a. Write the definitions of the member functions of the `class calendarType` (designed in Programming Exercise 8) to implement the operations of the `class calendarType`.

    b. Write a test program to print the calendar for either a particular month or a particular year. For example, the calendar for September 2011 is as follows:

```
                September 2011
Sun     Mon     Tue     Wed     Thu     Fri     Sat
                                  1       2       3
  4       5       6       7       8       9      10
 11      12      13      14      15      16      17
 18      19      20      21      22      23      24
 25      26      27      28      29      30
```

11. In Chapter 1, the `class clockType` was designed and implemented to implement the time of day in a program. This chapter discussed how to overload various operators. Redesign the `class clockType` by overloading the following operators: the stream insertion `<<` and stream extraction `>>` operators for input and output, the pre- and postincrement increment operators to increment the time by one second, and the relational operators to compare the two times. Also write a test program to test various operations of the `class` clockType.

12. a. Extend the definition of the `class complexType` so that it performs the subtraction and division operations. Overload the operators subtraction and division for this class as member functions.

    If (a, b) and (c, d) are complex numbers,

    $(a, b) - (c, d) = (a - c, b - d)$,

    If (c, d) is nonzero,

    $(a, b) / (c, d) = ((ac + bd) / (c^2 + d^2), (-ad + bc) / (c^2 + d^2))$

    b. Write the definitions of the functions to overload the operators – and / as defined in part a.

    c. Write a test program that tests the various operations on the `class complexType`. Format your answer with two decimal places.

13. a. Rewrite the definition of the `class complexType` so that the arithmetic and relational operators are overloaded as nonmember functions.

    b. Write the definitions of the member functions of the `class complexType` as designed in part a.

    c. Write a test program that tests the various operations on the `class complexType` as designed in parts a and b. Format your answer with two decimal places.

14. Let $a + ib$ be a complex number. The conjugate of $a + ib$ is $a - ib$ and the absolute value of $a + ib$ is $\sqrt{a^2 + b^2}$.Extend the definition of the `class complexType` of the Programming Example, Complex Numbers by overloading the operators ~ and `!` as member functions so that ~ returns the conjugate of a complex number and `!` returns the absolute value. Write the definitions of these operator functions.

15. Redo Programming Exercise 13 so that the operators ~ and `!` are overloaded as nonmember functions.

16. In C++, the largest `int` value is `2147483647`. So an integer larger than this cannot be stored and processed as an integer. Similarly, if the sum or product of two positive integers is greater than `2147483647`, the result will be incorrect. One way to store and manipulate large integers is to store each individual digit of the number in an array. Design the `class largeIntegers` so that an object of this class can store an integer up to 100 digits long. Overload the operators + and – to add and subtract, respectively, the values of two objects of this class. (In the Programming Exercises in Chapter 3, we will overload the multiplication operator.) Overload the assignment operator to copy the value of a large integer into another large integer. Overload the stream extraction and insertion operators for easy input and output. Your program must contain appropriate constructors to initialize objects of the `class largeIntegers`. (*Hint:* Read numbers as strings and store the digits of the number in the reverse order. Add instance variables to store the number of digits and the sign of the number.)

17. The roots of the quadratic equation $ax^2 + bx + c = 0$, $a \neq 0$ are given by the following formula:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

In this formula, the term $b^2 - 4ac$ is called the **discriminant**. If $b^2 - 4ac = 0$, the equation has a single (repeated) root. If $b^2 - 4ac > 0$, the equation has two real roots. If $b^2 - 4ac < 0$, the equation has two complex roots. Design and implement the `class quadraticEq` so that an object of this class can store the coefficients of a quadratic equation. Overload the operators + and – to add and subtract, respectively, the corresponding coefficients of two quadratic equations. Overload the relational operators `==` and `!=` to determine if two quadratic equations are the same. Add appropriate constructors to initialize objects. Overload the stream extraction and insertion operator for easy input and output. Also, include function members to determine and output the type and the roots of the equation. Write a program to test your class.

18. Programming Exercise 6 in Chapter 1 describes how to design the `class lineType` to implement a line. Redo this programming exercise so that the `class lineType`:

a. Overloads the stream insertion operator, `<<`, for easy output.
b. Overloads the stream extraction operator, `>>`, for easy input. (The line $ax + by = c$ is input as (a, b, c).
c. Overloads the assignment operator to copy a line into another line.
d. Overloads the unary operator `+` as a member function, so that it returns `true` if a line is vertical; `false` otherwise.
e. Overloads the unary operator `-` as a member function, so that it returns `true` if a line is horizontal; `false` otherwise.
f. Overloads the operator `==` as a member function, so that it returns `true` if two lines are equal; `false` otherwise.
g. Overloads the operator `||` as a member function, so that it returns `true` if two lines are parallel; `false` otherwise.
h. Overloads the operator `&&` as a member function, so that it returns `true` if two lines are perpendicular; `false` otherwise.

Write a program to test your class.

19. Rational fractions are of the form a / b, where a and b are integers and $b \neq 0$. In this exercise, by "fractions" we mean rational fractions. Suppose a / b and c / d are fractions. Arithmetic operations on fractions are defined by the following rules:

$a / b + c / d = (ad + bc) / bd$

$a / b - c / d = (ad - bc)/bd$

$a / b \times c / d = ac / bd$

$(a / b) / (c / d) = ad / bc$, where $c / d \neq 0$.

Fractions are compared as follows: a / b *op* c / d if ad *op* bc, where *op* is any of the relational operations. For example, $a / b < c / d$ if $ad < bc$.

Design a `class`—say, `fractionType`—that performs the arithmetic and relational operations on fractions. Overload the arithmetic and relational operators so that the appropriate symbols can be used to perform the operation. Also, overload the stream insertion and stream extraction operators for easy input and output.

a. Write a C++ program that, using the `class fractionType`, performs operations on fractions.
b. Among other things, test the following: Suppose `x`, `y`, and `z` are objects of type `fractionType`. If the input is `2/3`, the statement

```
cin >> x;
```

should store `2/3` in `x`. The statement

```
cout << x + y << endl;
```

should output the value of `x + y` in fraction form. The statement

```
z = x + y;
```

should store the sum of `x` and `y` in `z` in fraction form. Your answer need not be in the lowest terms.

20. a. In Programming Exercise 1 in Chapter 1, we defined a `class romanType` to implement Roman numerals in a program. In that exercise, we also implemented a function, `romanToDecimal`, to convert a Roman numeral into its equivalent decimal number.

Modify the definition of the `class romanType` so that the data members are declared as `protected`. Use the `class string` to manipulate the strings. Furthermore, overload the stream insertion and stream extraction operators for easy input and output. The stream insertion operator outputs the Roman numeral in the Roman format.

Also, include a member function, `decimalToRoman`, that converts the decimal number (the decimal number must be a positive integer) to an equivalent Roman numeral format. Write the definition of the member function `decimalToRoman`.

For simplicity, we assume that only the letter I can appear in front of another letter and that it appears only in front of the letters `V` and `X`. For example, `4` is represented as `IV`, `9` is represented as `IX`, `39` is represented as `XXXIX`, and 49 is represented as `XXXXIX`. Also, `40` will be represented as `XXXX`, `190` will be represented as `CLXXXX`, and so on.

b. Derive a `class extRomanType` from the `class romanType` to do the following. In the `class extRomanType`, overload the arithmetic operators `+`, `-`, `*`, and `/` so that arithmetic operations can be performed on Roman numerals. Also, overload the pre- and postincrement and decrement operators as member functions of the `class extRomanType`.

To add (subtract, multiply, or divide) Roman numerals, add (subtract, multiply, or divide, respectively) their decimal representations and then convert the result to the Roman numeral format. For subtraction, if the first number is smaller than the second number, output a message saying that, "`Because the first number is smaller than the second, the numbers cannot be subtracted`". Similarly, for division, the numerator must be larger than the denominator. Use similar conventions for the increment and decrement operators.

c. Write the definitions of the functions to overload the operators described in part b.

d. Write a program to test your `class extRomanType`.



CHAPTER 3

Pointers and Array-Based Lists

IN THIS CHAPTER, YOU WILL:

- Learn about the pointer data type and pointer variables
- Explore how to declare and manipulate pointer variables
- Learn about the address of operator and dereferencing operator
- Discover dynamic variables
- Examine how to use the `new` and `delete` operators to manipulate dynamic variables
- Learn about pointer arithmetic
- Discover dynamic arrays
- Become aware of the shallow and deep copies of data
- Discover the peculiarities of classes with pointer data members
- Explore how dynamic arrays are used to process lists
- Learn about virtual functions
- Become aware of abstract classes

The data types in C++ are classified into three categories: simple, structured, and pointers. Until now, you have worked with only the first two data types. This chapter discusses the third data type: the pointer data type. You first learn how to declare pointer variables (or pointers, for short) and manipulate the data to which they point. Later, you use these concepts when you study dynamic arrays and linked lists. Linked lists are discussed in Chapter 5.

The Pointer Data Type and Pointer Variables

The values belonging to pointer data types are the memory addresses of your computer. However, there is no name associated with the pointer data type in C++. Because the domain, (that is, the values of a pointer data type), consists of addresses (memory locations), a pointer variable is a variable whose content is an address, that is, a memory location.

Pointer variable: A variable whose content is an address (that is, a memory address).

Declaring Pointer Variables

The value of a pointer variable is an address. That is, the value refers to another memory space. The data is typically stored in this memory space. Therefore, when you declare a pointer variable, you also specify the data type of the value to be stored in the memory location to which the pointer variable points.

In C++, you declare a pointer variable by using the asterisk symbol (`*`) between the data type and the variable name. The general syntax to declare a pointer variable is as follows:

```
dataType *identifier;
```

As an example, consider the following statements:

```
int *p;
char *ch;
```

In these statements, both `p` and `ch` are pointer variables. The content of `p` (when properly assigned) points to a memory location of type `int`, and the content of `ch` points to a memory location of type `char`. Usually `p` is called a pointer variable of type `int`, and `ch` is called a pointer variable of type `char`.

Before discussing how pointers work, let us make the following observations. The following statements that declare `p` to be a pointer variable of type `int` are equivalent:

```
int *p;
int*  p;
int * p;
```

Thus, the character * can appear anywhere between the data type name and the variable name.

Now consider the following statement:

```
int* p, q;
```

In this statement, only p is a pointer variable, not q. Here q is an int variable. To avoid confusion, we prefer to attach the character * to the variable name. So the preceding statement is written as follows:

```
int *p, q;
```

Of course, the statement

```
int *p, *q;
```

declares both p and q to be pointer variables of type int.

Now that you know how to declare pointers, next we discuss how to make a pointer point to a memory space and how to manipulate the data stored in these memory locations.

Because the value of a pointer is a memory address, a pointer can store the address of a memory space of the designated type. For example, if p is a pointer of type int, p can store the address of any memory space of type int. C++ provides two operators—the address of operator (&) and the dereferencing operator (*)—to work with pointers. The next two sections describe these operators.

Address of Operator (&)

In C++, the ampersand, &, called the **address of operator**, is a unary operator that returns the address of its operand. For example, given the statements

```
int x;
int *p;
```

the statement

```
p = &x;
```

assigns the address of x to p. That is, x and the value of p refer to the same memory location.

Dereferencing Operator (*)

The previous chapters used the asterisk character, *, as the binary multiplication operator. C++ also uses * as a unary operator. When *, commonly referred to as the **dereferencing operator** or **indirection operator**, is used as a unary operator, * refers to the

object to which the operand of the * (that is, the pointer) points. For example, given the statements

```
int x = 25;
int *p;
p = &x;   //store the address of x in p
```

the statement

```
cout << *p << endl;
```

prints the value stored in the memory space to which **p** points, which is the value of **x**. Also, the statement

```
*p = 55;
```

stores 55 in the memory location to which **p** points—that is, 55 is stored in **x**.

Example 3-1 shows how a pointer variable works.

EXAMPLE 3-1

Let us consider the following statements:

```
int *p;
int num;
```

In these statements, **p** is a pointer variable of type **int** and **num** is a variable of type **int**. Let us assume that memory location 1200 is allocated for **p** and memory location 1800 is allocated for **num**. (See Figure 3-1.)

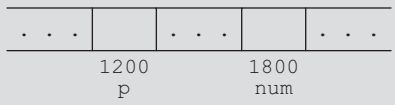
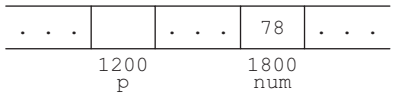
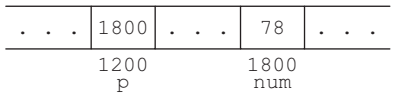
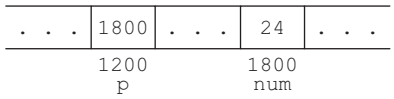


FIGURE 3-1 Variables p and num

Consider the following statements:

1. `num = 78;`
2. `p = #`
3. `*p = 24;`

The following shows the values of the variables after the execution of each statement.

After statement	Values of the variables	Explanation
1		The statement `num = 78;` stores `78` into `num`.
2		The statement `p = #` stores the address of `num`, which is `1800`, into `p`.
3		The statement `*p = 24;` stores `24` into the memory location to which `p` points. Because the value of `p` is `1800`, statement 3 stores `24` into memory location `1800`. Note that the value of `num` is also changed.

Let us summarize the preceding discussion.

1. A declaration such as `int *p;` allocates memory for `p` only, not for `*p`. Later, you learn how to allocate memory for `*p`.
2. The content of `p` points only to a memory location of type `int`.
3. `&p`, `p`, and `*p` all have different meanings.
4. `&p` means the address of `p`—that is, `1200` (as shown in Figure 3-1).
5. `p` means the content of `p`, which is `1800`, after the statement `p = #` executes.
6. `*p` means the content of the memory location to which `p` points. Note that the value of `*p` is `78` after the statement `p = #` executes; the value of `*p` is `24` after the statement `*p = 24;` executes.

The program in Example 3-2 further illustrates how a pointer variable works.

EXAMPLE 3-2

```
//*********************************************************
// Author: D.S. Malik
//
// This program illustrates how a pointer variable works.
//*********************************************************

#include <iostream>                                    //Line 1

using namespace std;                                   //Line 2
```

```
int main()                                                  //Line 3
{                                                           //Line 4
    int *p;                                                 //Line 5
    int num1 = 5;                                           //Line 6
    int num2 = 8;                                           //Line 7

    p = &num1; //store the address of num1 into p; Line 8

    cout << "Line 9: &num1 = " << &num1
         << ", p = " << p << endl;                          //Line 9
    cout << "Line 10: num1 = " << num1
         << ", *p = " << *p << endl;                        //Line 10

    *p = 10;                                                //Line 11
    cout << "Line 12: num1 = " << num1
         << ", *p = " << *p << endl << endl;                //Line 12

    p = &num2; //store the address of num2 into p; Line 13

    cout << "Line 14: &num2 = " << &num2
         << ", p = " << p << endl;                          //Line 14
    cout << "Line 15: num2 = " << num2
         << ", *p = " << *p << endl;                        //Line 15

    *p = 2 * (*p);                                          //Line 16
    cout << "Line 17: num2 = " << num2
         << ", *p = " << *p << endl;                        //Line 17

    return 0;                                               //Line 18
}                                                           //Line 19
```

Sample Run:

```
Line 9: &num1 = 0012FF54, p = 0012FF54
Line 10: num1 = 5, *p = 5
Line 12: num1 = 10, *p = 10

Line 14: &num2 = 0012FF48, p = 0012FF48
Line 15: num2 = 8, *p = 8
Line 17: num2 = 16, *p = 16
```

For the most part, the preceding output is straightforward. Let us look at some of these statements. The statement in Line 8 stores the address of `num1` into `p`. The statement in Line 9 outputs the value of `&num1`, the address of `num1`, and the value of `p`. (Note that the values output by Line 9 are machine dependent. When you execute this program on your computer, you are likely to get different values of `&num1` and `p`.) The statement in Line 10 outputs the value of `num1` and `*p`. Because `p` points to the memory location of `num1`, `*p` outputs the value of this memory location, that is, of `num1`. The statement in Line 11 changes the value of `*p` to `10`. Because `p` points to the memory location `num1`, the value of `num1` is also changed. The statement in Line 12 outputs the value of `num1` and `*p`.

The statement in Line 13 stores the address of `num2` into `p`. So after the execution of this statement, `p` points to `num2`. So, any change that `*p` makes immediately changes the value

of num2. The statement in Line 14 outputs the address of num2 and the value of p. This statement in Line 16 multiplies the value of *p, which is the value of num2, by 2 and stores the new value into *p. This statement also changes the value of num2. The statement in Line 17 outputs the value of num2 and *p.

Pointers and Classes

Consider the following statements:

```
string *str;
str = new string;
*str = "Sunny Day";
```

The first statement declares str to be a pointer variable of type string. The second statement allocates memory of type string and stores the address of the allocated memory in str. The third statement stores the string "Sunny Day" in the memory to which str points. Now suppose that you want to use the string function length to find the length of the string "Sunny Day". The statement (*str).length() returns the length of the string. Note the parentheses around *str. The expression (*str).length() is a mixture of pointer dereferencing and the class component selection. In C++, the dot operator, ., has a higher precedence than the dereferencing operator, *. Let us elaborate on this a little more. In the expression (*str).length(), the operator * evaluates first, so the expression *str evaluates first. Because str is a pointer variable of type string, *str refers to a memory space of type string. Therefore, in the expression (*str).length(), the function length of the class string executes. Now consider the expression *str.length(). Let us see how this expression gets evaluated. Because . has a higher precedence than *, the expression str.length() evaluates first. The expression str.length() would result in a syntax error because str is *not* a string object, so it cannot use the function length of the class string.

As you can see, in the expression (*str).length(), the parentheses around *str are important. However, typos are unavoidable. Therefore, to simplify the accessing of class or struct components via a pointer, C++ provides another operator, called the **member access operator arrow**, ->. The operator -> consists of two consecutive symbols: a hyphen and the "greater than" symbol.

The syntax for accessing a class (struct) member using the operator -> is as follows:

```
pointerVariableName->classMemberName
```

Thus, the expression

```
(*str).length()
```

is equivalent to the expression

```
str->length()
```

Accessing `class` (`struct`) components via pointers using the operator `->` thus eliminates the use both of the parentheses and of the dereferencing operator. Because typos are unavoidable and missing parentheses can result in either an abnormal program termination or erroneous results, when accessing `class` (`struct`) components via pointers, this book uses the arrow notation.

Initializing Pointer Variables

Because C++ does not automatically initialize variables, pointer variables must be initialized if you do not want them to point to anything. Pointer variables are initialized using the constant value `0`, called the **null pointer**. Thus, the statement `p = 0;` stores the null pointer in `p`; that is, `p` points to nothing. Some programmers use the named constant `NULL` to initialize pointer variables. The following two statements are equivalent:

```
p = NULL;
p = 0;
```

The number `0` is the only number that can be directly assigned to a pointer variable.

Dynamic Variables

In the previous sections, you learned how to declare pointer variables, how to store the address of a variable into a pointer variable of the same type as the variable, and how to manipulate data using pointers. However, you learned how to use pointers to manipulate data only into memory spaces that were created using other variables. In other words, the pointers manipulated data into existing memory spaces. So what is the benefit to using pointers? You can access these memory spaces by working with the variables that were used to create them. In this section, you learn about the power behind pointers. In particular, you learn how to allocate and deallocate memory during program execution using pointers.

Variables that are created during program execution are called **dynamic variables**. With the help of pointers, C++ creates dynamic variables. C++ provides two operators, `new` and `delete`, to create and destroy dynamic variables, respectively. When a program requires a new variable, the operator `new` is used. When a program no longer needs a dynamic variable, the operator `delete` is used.

In C++, `new` and `delete` are reserved words.

Operator `new`

The operator `new` has two forms: one to allocate a single variable, and another to allocate an array of variables. The syntax to use the operator `new` is as follows:

```
new dataType;            //to allocate a single variable
new dataType[intExp];    //to allocate an array of variables
```

where `intExp` is any expression evaluating to a positive integer.

The operator **new** allocates memory (a variable) of the designated type and returns a pointer to it—that is, the address of this allocated memory. Moreover, the allocated memory is uninitialized.

Consider the following declaration:

```
int *p;
char *q;
int x;
```


The statement

```
p = &x;
```

stores the address of **x** in **p**. However, no new memory is allocated. On the other hand, consider the following statement:

```
p = new int;
```

This statement creates a variable during program execution somewhere in memory, and stores the address of the allocated memory in **p**. The allocated memory is accessed via pointer dereferencing—namely, ***p**. Similarly, the statement

```
q = new char[16];
```

creates an array of 16 components of type **char** and stores the base address of the array in **q**.

Because a dynamic variable is unnamed, it cannot be accessed directly. It is accessed indirectly by the pointer returned by **new**. The following statements illustrate this concept:

```
int *p;   //p is a pointer of type int

p = new int;  //allocates memory of type int and stores the address
              //of the allocated memory in p
*p = 28;      //stores 28 in the allocated memory
```

NOTE

The operator **new** allocates memory space of a specific type and returns the (starting) address of the allocated memory space. However, if the operator **new** is unable to allocate the required memory space (for example, there is not enough memory space), the program might terminate with an error message.

Operator delete

Suppose you have the following declaration:

```
int *p;
```

This statement declares **p** to be a pointer variable of type **int**. Next, consider the following statements:

```
p = new int;      //Line 1
*p = 54;          //Line 2
p = new int;      //Line 3
*p = 73;          //Line 4
```

Let us see the effect of these statements. The statement in Line 1 allocates memory space of type **int** and stores the address of the allocated memory space into **p**. Suppose that the address of allocated memory space is **1500**. Then, the value of **p** after the execution of this statement is **1500**. (See Figure 3-2.)

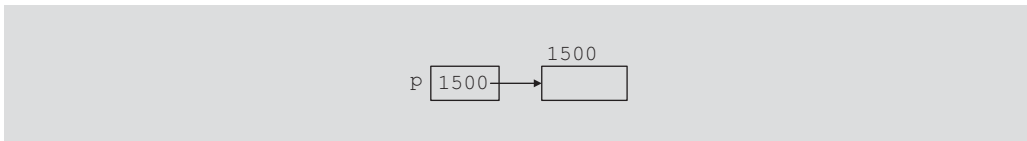


FIGURE 3-2 p after the execution of `p = new int;`

In Figure 3-2, the number **1500** on top of the box indicates the address of the memory space. The statement in Line 2 stores **54** into the memory space to which **p** points, which is **1500**. In other words, after execution of the statement in Line 2, the value stored into memory space at location **1500** is **54**. (See Figure 3-3.)

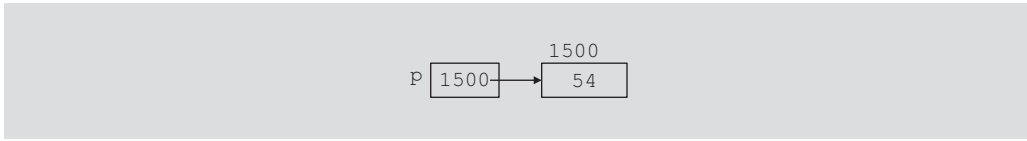


FIGURE 3-3 p and *p after the execution of `*p = 54;`

Next, the statement in Line 3 executes, which allocates a memory space of type **int** and stores the address of the allocated memory space into **p**. Suppose the address of this allocated memory space is **1800**. It follows that the value of **p** is now **1800**. (See Figure 3-4.)

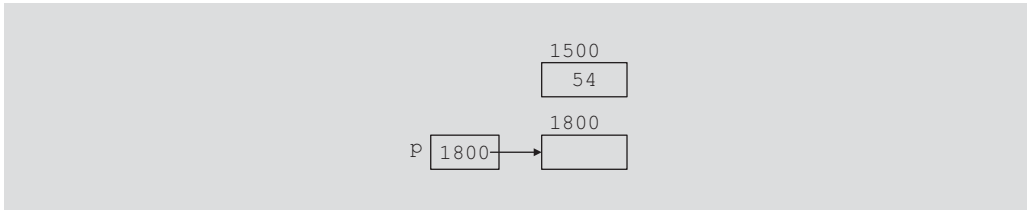


FIGURE 3-4 p after the execution of `p = new int;`

The statement in Line 4 stores **73** into the memory space to which **p** points, which is **1800**. In other words, after execution of the statement in Line 4, the value stored into the memory space at location **1800** is **73**. (See Figure 3-5.)

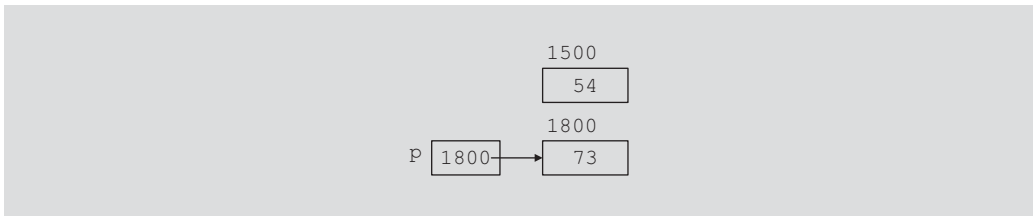


FIGURE 3-5 p after the execution of *p = 73;

Now the obvious question is, what happened to the memory space 1500, to which p was pointing, before the execution of the statement in Line 3? After execution of the statement in Line 3, p points to the new memory space at location 1800. The previous memory space at location 1500 is now inaccessible. In addition, the memory space 1500 remains marked as allocated. In other words, it cannot be reallocated. This is called **memory leak**. That is, there is an unused memory space that cannot be allocated.

Imagine what would happen if you execute statements such as Line 1 a few thousand times, or a few million times. There will be a good amount of memory leak. The program might then run out of memory space for data manipulation, and eventually result in an abnormal termination of the program.

The question at hand is how to *avoid* memory leak. When a dynamic variable is no longer needed, it can be destroyed; that is, its memory can be deallocated. The C++ operator **delete** is used to destroy dynamic variables, so that its memory space can be allocated again when needed. The syntax to use the operator **delete** has the following two forms:

```
delete pointerVariable; //to deallocate a single dynamic variable
delete [] pointerVariable; //to deallocate a dynamic array
```

Thus, given the declarations of the previous section, the statements

```
delete p;
delete str;
```

deallocate the memory spaces to which the pointers **p** and **str** point.

Suppose **p** is a pointer variable, as declared previously. Note that an expression such as

```
delete p;
```

only marks as deallocated the memory spaces to which these pointer variables point. Depending on a particular system, after these statements execute, these pointer variables might still contain the addresses of the deallocated memory spaces. In this case, we say that these pointers are **dangling**. Therefore, if you later access the memory spaces via these pointers without properly initializing them, depending on a particular system, either the program will access a wrong memory space, which might result in corrupting data, or the program will terminate with an error message. One way to avoid this pitfall is to set these pointers to **NULL** after the **delete** operation.

The program in the following example illustrates how to allocate dynamic memory and how to manipulate data into that dynamic memory.

EXAMPLE 3-3

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to allocate dynamic memory
// using a pointer variable and how to manipulate data into
// that memory location.
//***********************************************************

#include <iostream>                                    //Line 1

using namespace std;                                   //Line 2

int main()                                             //Line 3
{                                                      //Line 4
    int *p;                                            //Line 5
    int *q;                                            //Line 6

    p = new int;                                       //Line 7
    *p = 34;                                           //Line 8
    cout << "Line 9: p = " << p
         << ", *p = " << *p << endl;                   //Line 9

    q = p;                                             //Line 10
    cout << "Line 11: q = " << q
         << ", *q = " << *q << endl;                   //Line 11

    *q = 45;                                           //Line 12
    cout << "Line 13: p = " << p
         << ", *p = " << *p << endl;                   //Line 13
    cout << "Line 14: q = " << q
         << ", *q = " << *q << endl;                   //Line 14

    p = new int;                                       //Line 15
    *p = 18;                                           //Line 16
    cout << "Line 17: p = " << p
         << ", *p = " << *p << endl;                   //Line 17
    cout << "Line 18: q = " << q
         << ", *q = " << *q << endl;                   //Line 18

    delete q;                                          //Line 19
    q = NULL;                                          //Line 20
    q = new int;                                       //Line 21
    *q = 62;                                           //Line 22
    cout << "Line 23: p = " << p
         << ", *p = " << *p << endl;                   //Line 23
    cout << "Line 24: q = " << q
         << ", *q = " << *q << endl;                   //Line 24

    return 0;                                          //Line 25
}                                                      //Line 26
```

Sample Run:

```
Line 9: p = 00355620, *p = 34
Line 11: q = 00355620, *q = 34
Line 13: p = 00355620, *p = 45
Line 14: q = 00355620, *q = 45
Line 17: p = 003556C8, *p = 18
Line 18: q = 00355620, *q = 45
Line 23: p = 003556C8, *p = 18
Line 24: q = 00355620, *q = 62
```


The statements in Lines 5 and 6 declare **p** and **q** to be pointer variables of type **int**. The statement in Line 7 allocates memory of type **int** and stores the address of the allocated memory into **p**. (See Figure 3-6.)

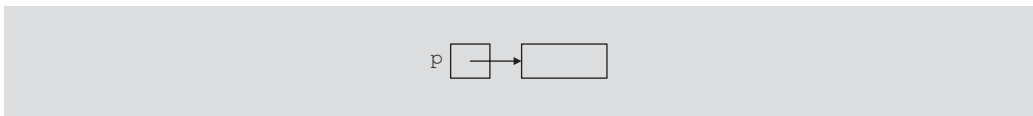


FIGURE 3-6 Pointer p and the memory space to which it points

The box indicates the allocated memory (in this case, of type **int**), and **p** together with the arrow indicates that **p** points to the allocated memory. The statement in Line 8 stores **34** into the memory location to which **p** points. (See Figure 3-7.)

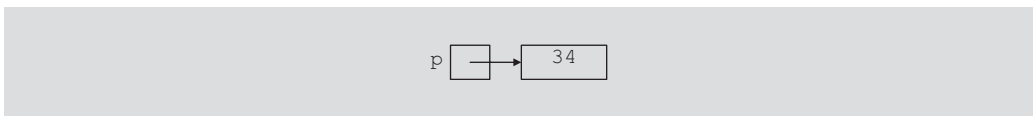


FIGURE 3-7 Pointer p and the value of the memory location to which p points

The statement in Line 9 outputs the value of **p** and ***p**. (Note that the values of **p** and **q** shown in the sample run are machine dependent. When you execute this program, you are likely to get different values of **p** and **q**.)

The statement in Line 10 copies the value of **p** into **q**. (See Figure 3-8.)

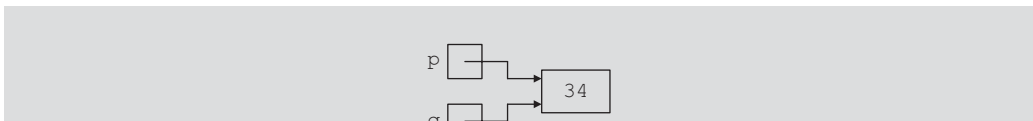


FIGURE 3-8 Pointers p and q and the memory space to which they point after the execution of the statement in Line 10

After the execution of the statement in Line 10, **p** and **q** both point to the same memory location. So any changes made into that memory location by **q** immediately change the value of ***p**. The statement in Line 11 outputs the value of **q** and ***q**. The statement in Line 12 stores **45** into the memory location to which **q** points. (See Figure 3-9.)

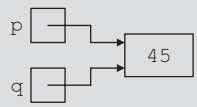


FIGURE 3-9 Pointers p and q and the memory space to which they point after the execution of the statement in Line 12

The statements in Lines 13 and 14 output the values of **p**, ***p**, **q**, and ***q**.

The statement in Line 15 allocates memory space of type **int** and stores the address of that memory into **p**. (See Figure 3-10.)

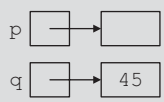


FIGURE 3-10 Pointers p and q and the memory space to which they point after the execution of the statement in Line 15

The statement in Line 16 stores **18** into the memory location to which **p** points. (See Figure 3-11.)

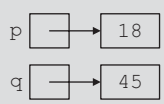


FIGURE 3-11 Pointers p and q and the memory space to which they point after the execution of the statement in Line 16

The statements in Lines 17 and 18 output the values of **p**, ***p**, **q**, and ***q**.

The statement in Line 19 deallocates the memory space to which **q** points and the statement in Line 20 sets the value of **q** to **NULL**. After the execution of the statement in Line 20, **q** does not point to any memory location. (See Figure 3-12.)

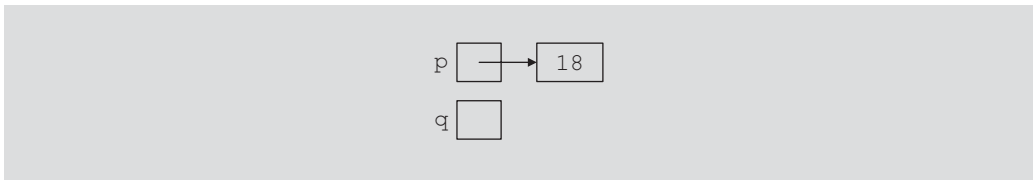


FIGURE 3-12 Pointers p and q and the memory space to which they point after the execution of the statement in Line 20

The statement in Line 21 allocates a memory space of type **int** and stores the address of that memory space into **q**. The statement in Line 22 stores **62** in the memory space to which **q** points. (See Figure 3-13.)

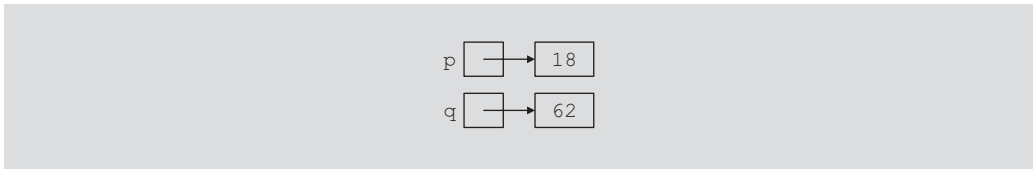


FIGURE 3-13 Pointers p and q and the memory space to which they point after the execution of the statement in Line 22

The statements in Lines 23 and 24 output the values of **p**, ***p**, **q**, and ***q**.

In the preceding program, omit statements in Lines 19 and 20, rerun the program, and note how the last output statements change.

Operations on Pointer Variables

The operations that are allowed on pointer variables are the assignment and relational operations and some limited arithmetic operations. The value of one pointer variable can be assigned to another pointer variable of the same type. Two pointer variables of the same type can be compared for equality, and so on. Integer values can be added and subtracted from a pointer variable. The value of one pointer variable can be subtracted from another pointer variable.

For example, suppose that we have the following statements:

```
int *p, *q;
```

The statement

```
p = q;
```

copies the value of **q** into **p**. After this statement executes, both **p** and **q** point to the same memory location. Any changes made to ***p** automatically change the value of ***q**, and vice versa.

The expression

```
p == q
```

evaluates to **true** if p and q have the same value—that is, if they point to the same memory location. Similarly, the expression

```
p != q
```

evaluates to true if p and q point to different memory locations.

The arithmetic operations that are allowed differ from the arithmetic operations on numbers. First, let us use the following statements to explain the increment and decrement operations on pointer variables:

```
int *p;
double *q;
char *chPtr;
```

Suppose that the size of the memory allocated for an int variable is 4 bytes, a double variable is 8 bytes, and a char variable is 1 byte.

The statement

```
p++;
```
or `p = p + 1;`

increments the value of p by 4 bytes because p is a pointer of type **int**. Similarly, the statements

```
q++;
chPtr++;
```

increment the value of q by 8 bytes and the value of chPtr by 1 byte, respectively.

The increment operator increments the value of a pointer variable by the size of the memory to which it is pointing. Similarly, the decrement operator decrements the value of a pointer variable by the size of the memory to which it is pointing.

Moreover, the statement

```
p = p + 2;
```

increments the value of p by 8 bytes.

Thus, when an integer is added to a pointer variable, the value of the pointer variable is incremented by the integer times the size of the memory to which the pointer is pointing. Similarly, when an integer is subtracted from a pointer variable, the value of the pointer variable is decremented by the integer times the size of the memory to which the pointer is pointing.

NOTE

Pointer arithmetic can be quite dangerous. Using pointer arithmetic, the program can accidentally access the memory locations of other variables and change their content without warning. The programmer is then left to try to find out what went wrong. If a pointer variable tries to access either the memory spaces of other variables or an illegal memory space, some systems might terminate the program with an appropriate error message. Always exercise extra care when doing pointer arithmetic.

Dynamic Arrays

The arrays used earlier are called static arrays because their size was fixed at compile time. One of the limitations of a static array is that every time you execute the program, the size of the array is fixed, so it might not be possible to use the same array to process different data sets of the same type. One way to handle this limitation is to declare an array that is large enough to process a variety of data sets. However, if the array is big and the data set is small, such a declaration would result in memory waste. On the other hand, it would be helpful if, during program execution, you could prompt the user to enter the size of the array and then create an array of the appropriate size. This approach is especially helpful if you cannot even guess the array size. In this section, you learn how to create arrays during program execution and how to process such arrays.

An array created during the execution of a program is called a **dynamic array**. To create a dynamic array, we use the second form of the **new** operator.

The statement

```
int *p;
```

declares p to be a pointer variable of type **int**. The statement

```
p = new int[10];
```

allocates 10 contiguous memory locations, each of type **int**, and stores the address of the first memory location into p. In other words, the operator **new** creates an array of 10 components of type **int**, it returns the base address of the array, and the assignment operator stores the base address of the array into p. Thus, the statement

```
*p = 25;
```

stores 25 into the first memory location, and the statements

```
p++;       //p points to the next array component
*p = 35;
```

store 35 into the second memory location. Thus, by using the increment and decrement operations, you can access the components of the array. Of course, after performing a few increment operations, it is possible to lose track of the first array component. C++ allows us to use array notation to access these memory locations. For example, the statements

```
p[0] = 25;
p[1] = 35;
```

store 25 and 35 into the first and second array components, respectively. That is, p[0] refers to the first array component, p[1] refers to the second array component, and so on. In general, p[i] refers to the (i + 1)th array component. After the preceding statements execute, p still points to the first array component.

The following **for** loop initializes each array component to 0:

```
for (int j = 0; j < 10; j++)
    p[j] = 0;
```

When the array notation is used to process the array to which `p` points, `p` stays fixed at the first memory location. Note that `p` is a dynamic array, created during program execution.

EXAMPLE 3-4

The following program segment illustrates how to obtain a user's response to get the array size and create a dynamic array during program execution. Consider the following statements:

```
int *intList;                       //Line 1
int arraySize;                      //Line 2

cout << "Enter array size: ";       //Line 3
cin >> arraySize;                   //Line 4
cout << endl;                       //Line 5

intList = new int[arraySize];       //Line 6
```

The statement in Line 1 declares `intList` to be a pointer of type `int`, and the statement in Line 2 declares `arraySize` to be an `int` variable. The statement in Line 3 prompts the user to enter the size of the array, and the statement in Line 4 inputs the array size into the variable `arraySize`. The statement in Line 6 creates an array of the size specified by `arraySize`, and the base address of the array is stored in `intList`. From this point on, you can treat `intList` just like any other array. For example, you can use the array notation to process the elements of `intList` and pass `intList` as a parameter to the function.

Array Name: A Constant Pointer

The statement

```
int list[5];
```

declares `list` to be an array of five components. Recall that `list` itself is a variable and the value stored in `list` is the base address of the array—that is, it is the address of the first array component. Suppose the address of the first array component is `1000`. Figure 3-14 shows `list` and the array `list`.

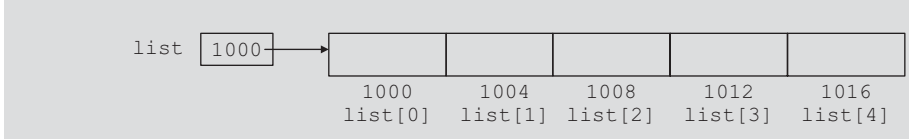


FIGURE 3-14 `list` and array `list`

Because the value of `list`, which is `1000`, is a memory address, `list` is a pointer variable. However, the value stored in `list`, which is `1000`, *cannot be altered during program execution.* That is, the value of `list` is *constant.* Therefore, the increment and decrement operations cannot be applied to `list`. In fact, any attempt to use the increment or decrement operations on `list` results in a compile-time error.

Notice that here we are *only* saying that the value of `list` cannot be changed. However, the data in the array `list` can be manipulated as usual. For example, the statement `list[0] = 25;` stores `25` into the first array component. Similarly, the statement `list[2] = 78;` stores `78` into the third component of `list`. (See Figure 3-15.)

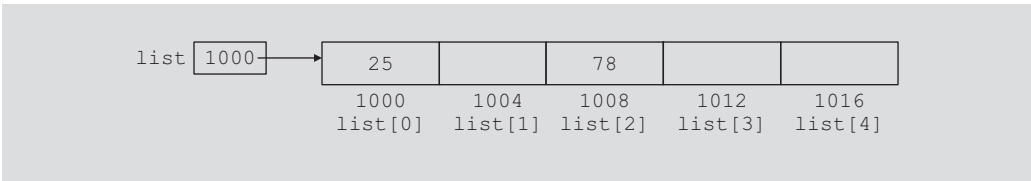


FIGURE 3-15 Array `list` after the execution of the statements `list[0] = 25;` and `list[2] = 78;`

If `p` is a pointer variable of type `int`, then the statement

```
p = list;
```

copies the value of `list`, which is `1000`, the base address of the array, into `p`. We are allowed to perform increment and decrement operations on `p`.

An *array name* is a *constant pointer.*

Functions and Pointers

A pointer variable can be passed as a parameter to a function either by value or by reference. To declare a pointer as a value parameter in a function heading, you use the same mechanism as you use to declare a variable. To make a formal parameter be a reference parameter, you use `&` when you declare the formal parameter in the function heading. Therefore, to declare a formal parameter as a reference parameter, you must use `&`. Between the data type name and the identifier name, you must include `*` to make the identifier a pointer and `&` to make it a reference parameter. The obvious question is: In what order should `&` and `*` appear between the data type name and the identifier to declare a pointer as a reference parameter? In C++, to make a pointer a reference parameter in a function heading, `*` appears before the `&` between the data type name and the identifier. The following example illustrates this concept:

```
void example(int* &p, double *q)
{
    .
    .
    .
}
```

In this example, both `p` and `q` are pointers. The parameter `p` is a reference parameter; the parameter `q` is a value parameter.

Pointers and Function Return Values

In C++, the return type of a function can be a pointer. For example, the return type of the function

```
int* testExp(...)
{
    .
    .
    .
}
```

is a pointer of type `int`.

Dynamic Two-Dimensional Arrays

The beginning of this section discussed how to create dynamic one-dimensional arrays. You can also create dynamic multidimensional arrays. In this section, we discuss how to create dynamic two-dimensional arrays. Dynamic multidimensional arrays are created similarly.

There are various ways you can create dynamic two-dimensional arrays. One way is as follows. Consider the statement:

```
int *board[4];
```

This statement declares `board` to be an array of four pointers wherein each pointer is of type `int`. Because `board[0]`, `board[1]`, `board[2]`, and `board[3]` are pointers, you can now use these pointers to create the rows of `board`. Suppose that each row of board has six columns. Then the following `for` loop creates the rows of `board`.

```
for (int row = 0; row < 4; row++)
    board[row] = new int[6];
```

Note that the expression `new int[6]` creates an array of six components of type `int` and returns the base address of the array. The assignment statement then stores the returned address into `board[row]`. It follows that after the execution of the previous `for` loop, `board` is a two-dimensional array of 4 rows and 6 columns.

In the previous `for` loop, if you replace the number 6 with the number 10, the loop will create a two-dimensional array of 4 rows and 10 columns. In other words, the number of columns of `board` can be specified during execution. However, the way `board` is declared, the number of rows is fixed. So in reality, `board` is not a true dynamic two-dimensional array.

Next, consider the following statement:

```
int **board;
```

This statement declares `board` to be a pointer to a pointer. In other words, `board` and `*board` are pointers. Now `board` can store the address of a pointer or an array of pointers of type `int`, and `*board` can store the address of an `int` memory space or an array of `int` values.

Suppose that you want `board` to be an array of 10 rows and 15 columns. To accomplish this, first we create an array of 10 pointers of type `int` and assign the address of that array to `board`. The following statement accomplishes this:

```
board = new int* [10];
```

Next we create the columns of `board`. The following `for` loop accomplishes this:

```
for (int row = 0; row < 10; row++)
    board[row] = new int[15];
```


To access the components of `board`, you can use the array subscripting notation. For example, see the next example. Note that the number of rows and the number of columns of `board` can be specified during program execution. The program in Example 3-5 further explains how to create two-dimensional arrays.

EXAMPLE 3-5

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use two-dimensional dynamic
// arrays.
//***********************************************************

#include <iostream>                                          //Line 1
#include <iomanip>                                           //Line 2

using namespace std;                                         //Line 3

void fill(int **p, int rowSize, int columnSize);             //Line 4
void print(int **p, int rowSize, int columnSize);            //Line 5

int main()                                                   //Line 6
{                                                            //Line 7
    int **board;                                             //Line 8

    int rows;                                                //Line 9
    int columns;                                             //Line 10

    cout << "Line 11: Enter the number of rows "
         <<"and columns: ";                                  //Line 11
    cin >> rows >> columns;                                  //Line 12
    cout << endl;                                            //Line 13

        //Create the rows of board
    board = new int* [rows];                                 //Line 14

        //Create the columns of board
    for (int row = 0; row < rows; row++)                     //Line 15
        board[row] = new int[columns];                       //Line 16
```

```
       //Insert elements into board
    fill(board, rows, columns);                          //Line 17

    cout << "Line 18: Board:" << endl;                   //Line 18

       //Output the elements of board
    print(board, rows, columns);                         //Line 19

    return 0;                                            //Line 20
}                                                        //Line 21

void fill(int **p, int rowSize, int columnSize)
{
    for (int row = 0; row < rowSize; row++)
    {
        cout << "Enter " << columnSize << " number(s) for row "
             << "number " << row << ": ";
        for (int col = 0; col < columnSize; col++)
            cin >> p[row][col];
        cout << endl;
    }
}

void print(int **p, int rowSize, int columnSize)
{
    for (int row = 0; row < rowSize; row++)
    {
         for (int col = 0; col < columnSize; col++)
            cout << setw(5) << p[row][col];
         cout << endl;
    }
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 11: Enter the number of rows and columns: 3 4

Enter 4 number(s) for row number 0: 1 2 3 4

Enter 4 number(s) for row number 1: 5 6 7 8

Enter 4 number(s) for row number 2: 9 10 11 12

Line 18: Board:
    1    2    3    4
    5    6    7    8
    9   10   11   12
```

The preceding program contains the functions `fill` and `print`. The function `fill` prompts the user to enter the elements of a two-dimensional array of type `int`. The function `print` outputs the elements of a two-dimensional array of type `int`.

For the most part, the preceding output is self-explanatory. Let us look at the statements in the function `main`. The statement in Line 8 declares `board` to be a pointer to a pointer of type `int`. The statements in Lines 9 and 10 declare `int` variables `rows` and `columns`. The statement in Line 11 prompts the user to input the number of rows and number of columns. The statement in Line 12 stores the number of rows in the variable `rows` and the number of columns in the variable `columns`. The statement in Line 14 creates the rows of `board` and the `for` loop in Lines 15 and 16 creates the columns of `board`. The statement in Line 17 used the function `fill` to fill the array `board` and the statement in Line 19 uses the function `print` to output the elements of `board`.

Shallow Vs. Deep Copy and Pointers

In an earlier section, we discussed pointer arithmetic and explained that if we are not careful, one pointer might access the data of another (completely unrelated) pointer. This event might result in unsuspected or erroneous results. Here, we discuss another peculiarity of pointers. To facilitate the discussion, we will use diagrams to show pointers and their related memory.

Suppose that you have the following declarations:

```
int *first;
int *second;
```

Further suppose that `first` points to an `int` array, as shown in Figure 3-16.

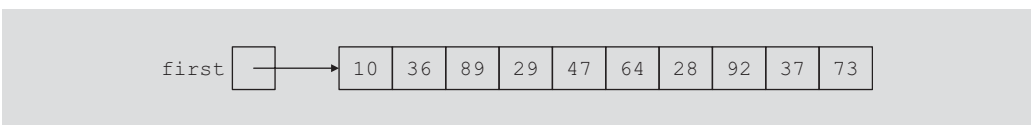


FIGURE 3-16 Pointer `first` and its array

Next, consider the following statement:

```
second = first;          //Line A
```

This statement copies the value of `first` into `second`. After this statement executes, both `first` and `second` point to the same array, as shown in Figure 3-17.

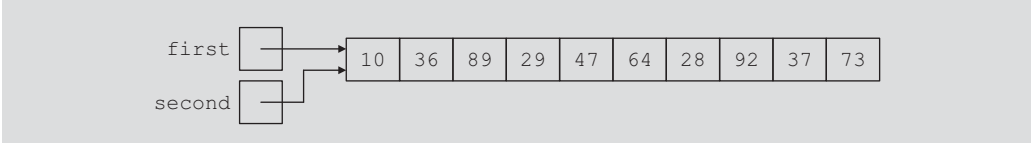


FIGURE 3-17 `first` and `second` after the statement `second = first;` executes

The statement `first[4] = 10;` not only changes the value of `first[4]`, it also changes the value of `second[4]` because they point to the same array.

Let us execute the following statement:

```
delete [] second;
```

After this statement executes, the array to which `second` points is deleted. This action results in Figure 3-18.

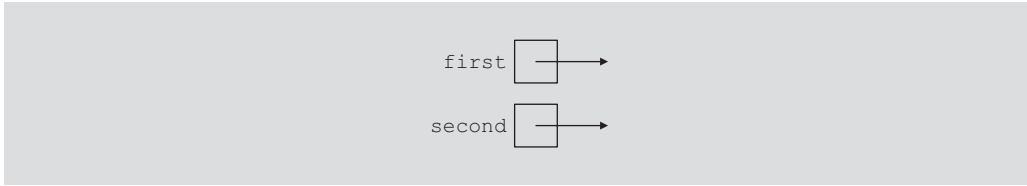


FIGURE 3-18 `first` and `second` after the statement `delete [] second;` executes

Because `first` and `second` pointed to the same array, after the statement

```
delete [] second;
```

executes, `first` becomes invalid, that is, `first` (as well as `second`) are now dangling pointers. Therefore, if the program later tries to access the memory to which `first` pointed, either the program will access the wrong memory or it will terminate in an error. This case is an example of a shallow copy. More formally, in a **shallow copy**, two or more pointers of the same type point to the same memory; that is, they point to the same data.

On the other hand, suppose that instead of the earlier statement, `second = first;` (in Line A), we have the following statements:

```
second = new int[10];

for (int j = 0; j < 10; j++)
    second[j] = first[j];
```

The first statement creates an array of 10 components of type `int`, and the base address of the array is stored in `second`. The second statement copies the array to which `first` points into the array to which `second` points. (See Figure 3-19.)

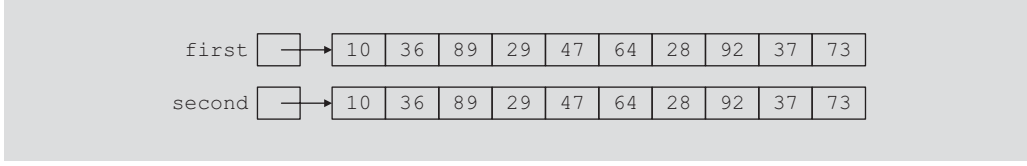


FIGURE 3-19 `first` and `second` both pointing to their own data

Both `first` and `second` now point to their own data. If `second` deletes its memory, there is no effect on `first`. This case is an example of a deep copy. More formally, in a **deep copy**, two or more pointers have their own data.

From the preceding discussion, it follows that you must know when to use a shallow copy and when to use a deep copy.

Classes and Pointers: Some Peculiarities

Because a class can have pointer member variables, this section discusses some peculiarities of such classes. To facilitate the discussion, we use the following class:

```
class pointerDataClass
{
public:
    .
    .
    .

private:
    int x;
    int lenP;
    int *p;
};
```

Also consider the following statements. (See Figure 3-20.)

```
pointerDataClass objectOne;
pointerDataClass objectTwo;
```

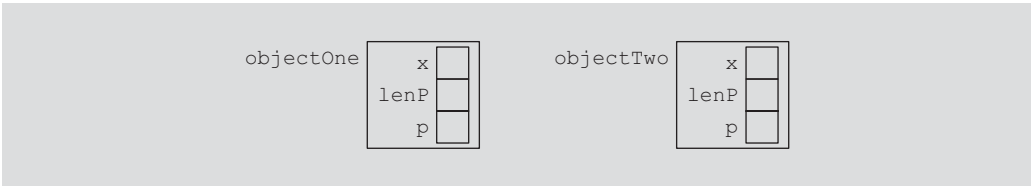


FIGURE 3-20 Objects `objectOne` and `objectTwo`

Destructor

The object `objectOne` has a pointer member variable `p`. Suppose that during program execution the pointer `p` creates a dynamic array. When `objectOne` goes out of scope, all the member variables of `objectOne` are destroyed. However, `p` created a dynamic array, and dynamic memory must be deallocated using the operator `delete`. Thus, if the pointer `p` does not use the `delete` operator to deallocate the dynamic array, the memory space of the dynamic array would stay marked as allocated, even though it cannot be

accessed. How do we ensure that when `p` is destroyed, the dynamic memory created by `p` is also destroyed? Suppose that `objectOne` is as shown in Figure 3-21.

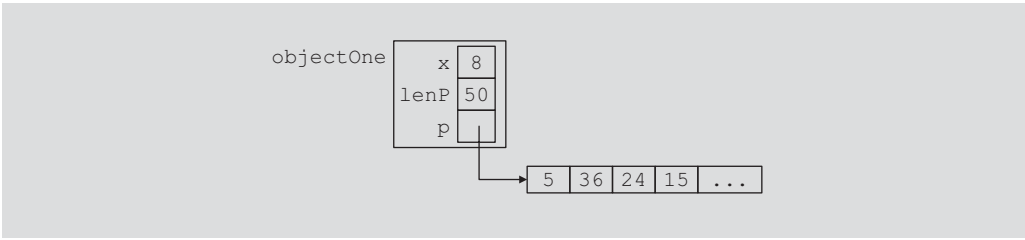


FIGURE 3-21 Object `objectOne` and its data

Recall that if a class has a destructor, the destructor automatically executes whenever a class object goes out of scope (see Chapter 1). Therefore, we can put the necessary code in the destructor to ensure that when `objectOne` goes out of scope, the memory created by the pointer `p` is deallocated. For example, the definition of the destructor for the `class pointerDataClass` is as follows:

```
pointerDataClass::~pointerDataClass()
{
    delete [] p;
}
```

Of course, you must include the destructor as a member of the class in its definition. Let us extend the definition of the `class pointerDataClass` by including the destructor. Moreover, the remainder of this section assumes that the definition of the destructor is as given previously—that is, the destructor deallocates the memory space pointed to by `p`.

```
class pointerDataClass
{
public:
    ~pointerDataClass();
      .
      .
      .

private:
    int x;
    int lenP;
    int *p;
};
```

NOTE For the destructor to work properly, the pointer `p` must have a valid value. If `p` is not properly initialized (that is, if the value of `p` is garbage) and the destructor executes, either the program terminates with an error message or the destructor deallocates an unrelated memory space. For this reason, you should exercise extra caution while working with pointers.

Assignment Operator

This section describes the limitations of the built-in assignment operators for classes with pointer member variables. Suppose that `objectOne` and `objectTwo` are as shown in Figure 3-22.

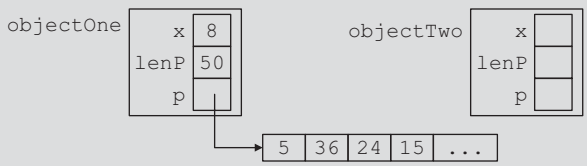


FIGURE 3-22 Objects `objectOne` and `objectTwo`

Recall that one of the built-in operations on classes is the assignment operator. For example, the statement:

```
objectTwo = objectOne;
```

copies the member variables of `objectOne` into `objectTwo`. That is, the value of `objectOne.x` is copied into `objectTwo.x`, and the value of `objectOne.p` is copied into `objectTwo.p`. Because `p` is a pointer, this memberwise copying of the data would lead to a shallow copying of the data. That is, both `objectTwo.p` and `objectOne.p` would point to the same memory space, as shown in Figure 3-23.

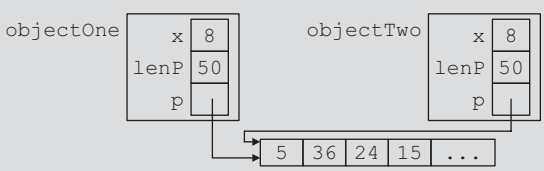


FIGURE 3-23 Objects `objectOne` and `objectTwo` after the statement `objectTwo = objectOne;` executes

Now, if `objectTwo.p` deallocates the memory space to which it points, `objectOne.p` would become invalid. This situation could very well happen, if the `class pointerDataClass` has a destructor that deallocates the memory space pointed to by `p` when an object of type `pointerDataClass` goes out of scope. It suggests that there must be a way to avoid this pitfall. To avoid this shallow copying of data for classes with a pointer member variable, C++ allows the programmer to extend the definition of the assignment operator. This process is called overloading the assignment operator. In the next section, we explain how to accomplish this task by using operator overloading. Once the assignment operator is properly overloaded, both `objectOne` and `objectTwo` have their own data, as shown in Figure 3-24.

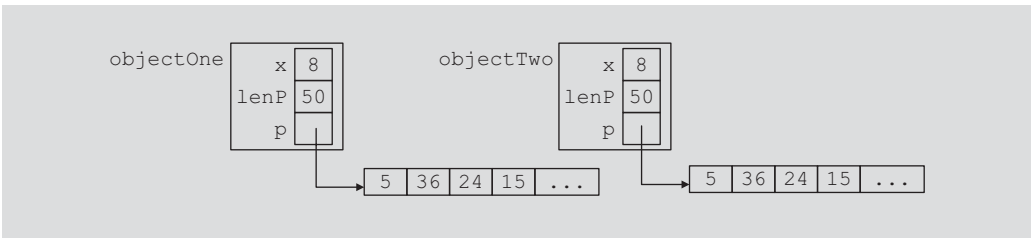


FIGURE 3-24 Objects `objectOne` and `objectTwo`

OVERLOADING THE ASSIGNMENT OPERATOR

Next we describe how to overload the assignment operator.

General Syntax to Overload the Assignment Operator = for a Class

Function Prototype (to be included in the definition of the class):

```
const className& operator=(const className&);
```

Function Definition:

```
const className& className::operator=(const className& rightObject)
{
         //local declaration, if any

      if (this != &rightObject)  //avoids self-assignment
      {
         //algorithm to copy rightObject into this object
      }

         //returns the object assigned
      return *this;
}
```

In the definition of the function `operator=`:

- There is only one formal parameter.
- The formal parameter is generally a `const` reference to a particular class.
- The return type of the function is a reference to a particular class.

Consider the statement

```
x = x;
```

Here, we are trying to copy the value of `x` into `x`; that is, this statement is a self-assignment. We must prevent such statements because they waste computer time.

The body of the function `operator=` does prevent such assignments. Let us see how.

Consider the `if` statement in the body of the operator function `operator=`:

```
if (this != &rightObject)  //avoids self-assignment
{
    //algorithm to copy rightObject into this object
}
```

Now the statement

```
x = x;
```

is compiled into the statement

```
x.operator=(x);
```

Because the function `operator=` is invoked by the object `x`, the pointer `this` in the body of the function `operator=` refers to the object `x`. Furthermore, because `x` is also a parameter to the function `operator=`, the formal parameter `rightObject` also refers to the object `x`. Therefore, in the expression

```
this != &rightObject
```

`this` means the address of `x`, and `&rightObject` also means the address of `x`. Thus, this expression will evaluate to `false` and, therefore, the body of the `if` statement will be skipped.

Notice that the return type of the function to overload the assignment operator is a reference. This is so that the statements such as `x = y = z`; can be executed, that is, the assignment operator can be used in a cascaded form.

In the section "Array-Based Lists," later in this chapter, we explicitly illustrate how to overload the assignment operator.

Copy Constructor

When declaring a class object, you can initialize it by using the value of an existing object of the same type. For example, consider the following statement:

```
pointerDataClass objectThree(objectOne);
```

The object `objectThree` is being declared and is also being initialized by using the value of `objectOne`. That is, the values of the member variables of `objectOne` are copied into the corresponding member variables of `objectThree`. This initialization is called the default memberwise initialization. The default memberwise initialization is due to the constructor, called the **copy constructor** (provided by the compiler). Just as in the case of the assignment operator, because the `class pointerDataClass` has pointer member variables, this default initialization would lead to a shallow copying of the data, as shown in Figure 3-25. (Assume that `objectOne` is given as before.)

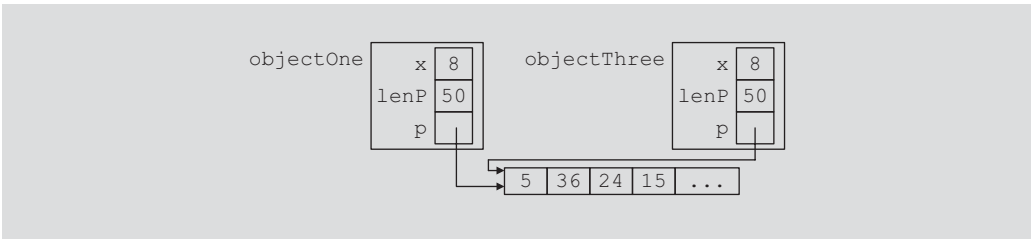


FIGURE 3-25 Objects `objectOne` and `objectThree`

Before describing how to overcome this deficiency, let us describe one more situation that could also lead to a shallow copying of the data. The solution to both these problems is the same.

Recall that, as parameters to a function, class objects can be passed either by reference or by value. Remember that the `class pointerDataClass` has the destructor, which deallocates the memory space pointed to by `p`. Suppose that `objectOne` is as shown in Figure 3-26.

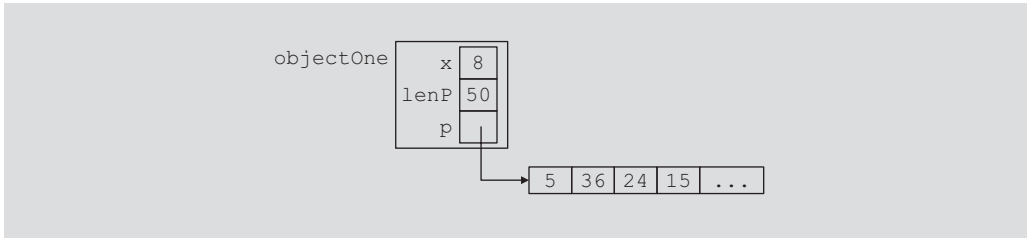


FIGURE 3-26 Object `objectOne`

Let us consider the following function prototype:

```
void destroyList(pointerDataClass paramObject);
```

The function `pointerDataClass` has a formal value parameter, `paramObject`. Now consider the following statement:

```
destroyList(objectOne);
```

In this statement, `objectOne` is passed as a parameter to the function `destroyList`. Because `paramObject` is a value parameter, the copy constructor copies the member variables of `objectOne` into the corresponding member variables of `paramObject`. Just as in the previous case, `paramObject.p` and `objectOne.p` would point to the same memory space, as shown in Figure 3-27.

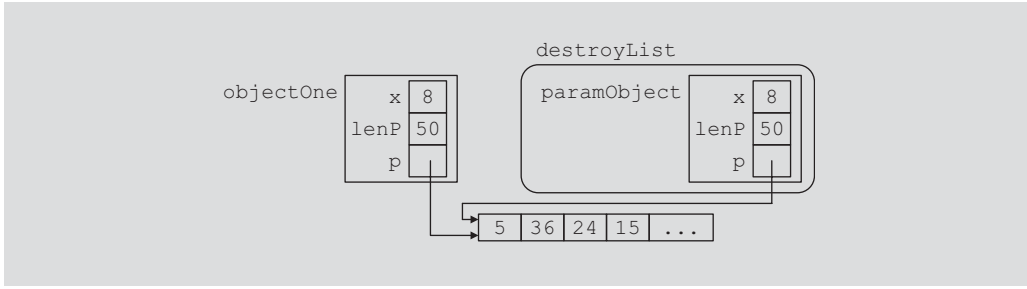


FIGURE 3-27 Pointer member variables of objects `objectOne` and `paramObject` pointing to the same array

Because `objectOne` is passed by value, the member variables of `paramObject` should have their own copy of the data. In particular, `paramObject.p` should have its own memory space. How do we ensure that this is, in fact, the case?

If a class has pointer member variables:

- During object declaration, the initialization of one object using the value of another object leads to a shallow copying of the data, if the default memberwise copying of data is allowed.
- If, as a parameter, an object is passed by value and the default memberwise copying of data is allowed, it leads to a shallow copying of the data.

In both cases, to force each object to have its own copy of the data, we must override the definition of the copy constructor provided by the compiler; that is, we must provide our own definition of the copy constructor. This is usually done by putting a statement that includes the copy constructor in the definition of the class, and then writing the definition of the copy constructor. Then, whenever the copy constructor needs to be executed, the system would execute the definition provided by us, not the one provided by the compiler. Therefore, for the `class pointerDataClass`, we can overcome this shallow copying problem by including the copy constructor in the `class pointerDataClass`.

The copy constructor automatically executes in three situations (the first two are described previously):

- When an object is declared and initialized by using the value of another object
- When, as a parameter, an object is passed by value
- When the return value of a function is an object

Therefore, once the copy constructor is properly defined for the `class pointerDataClass`, both `objectOne.p` and `objectThree.p` will have their own copies of the data. Similarly, `objectOne.p` and `paramObject.p` will have their own copies of the data, as shown in Figure 3-28.

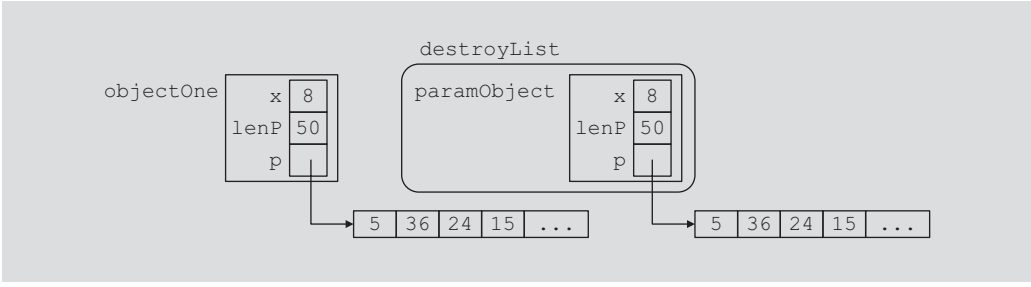


FIGURE 3-28 Pointer member variables of objects `objectOne` and `paramObject` with their own data

When the function `destroyList` exits, the formal parameter `paramObject` goes out of scope, and the destructor for the object `paramObject` deallocates the memory space pointed to by `paramObject.p`. However, this deallocation has no effect on `objectOne`.

The general syntax to include the copy constructor in the definition of a class is as follows:

```
className(const className& otherObject);
```

Notice that the formal parameter of the copy constructor is a constant reference parameter.

In the section, "Array-Based Lists," we explicitly illustrate how to include the copy constructor in a class and how it works.

For classes with pointer member variables, three things are normally done:

1. Include the destructor in the class.
2. Overload the assignment operator for the class.
3. Include the copy constructor.

Inheritance, Pointers, and Virtual Functions

Recall that, as a parameter, a class object can be passed either by value or by reference. Earlier chapters also said that the types of the actual and formal parameters must match. However, in the case of classes, C++ *allows the user to pass an object of a derived class to a formal parameter of the base class type.*

First, let us discuss the case when the formal parameter is either a reference parameter or a pointer. To be specific, let us consider the following classes:

```
class baseClass
{
public:
    void print();
    baseClass(int u = 0);

private:
    int x;
};

class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```

The `class baseClass` has three members. The `class derivedClass` is derived from the `class baseClass`, and has three members of its own. Both classes have a member function `print`. Suppose that the definitions of the member functions of both classes are as follows:

```
void baseClass::print()
{
    cout << "In baseClass x = " << x << endl;
}

baseClass::baseClass(int u)
{
    x = u;
}

void derivedClass::print()
{
    cout << "In derivedClass ***: ";
    baseClass::print();
    cout << "In derivedClass a = " << a << endl;
}

derivedClass::derivedClass(int u, int v)
               : baseClass(u)
{
    a = v;
}
```

Consider the following function in a user program (client code):

```
void callPrint(baseClass& p)
{
    p.print();
}
```

The function `callPrint` has a formal reference parameter `p` of type `baseClass`. You can call the function `callPrint` by using an object of either type `baseClass` or type `derivedClass` as a parameter. Moreover, the body of the function `callPrint` calls the member function `print`. Consider the following function `main`:

```
int main()                                      //Line 1
{                                               //Line 2
    baseClass one(5);                           //Line 3
    derivedClass two(3, 15);                    //Line 4

    one.print();                                //Line 5
    two.print();                                //Line 6

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;           //Line 7

    callPrint(one);                             //Line 8
    callPrint(two);                             //Line 9

    return 0;                                   //Line 10
}                                               //Line 11
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In baseClass x = 3
```

The statements in Lines 3 through 7 are quite straightforward. Let us look at the statements in Lines 8 and 9. The statement in Line 8 calls the function `callPrint` and passes the object `one` as the parameter; it generates the fifth line of the output. The statement in Line 9 also calls the function `callPrint`, but passes the object `two` as the parameter; it generates the sixth line of the output. The output generated by the statements in Lines 8 and 9 shows only the value of `x`, even though in these statements a different class object is passed as a parameter. (Because in Line 9 object `two` is passed as a parameter to the function `callPrint`, one would expect that the output generated by the statement in Line 9 should be similar to the output in the second and third lines of the output.) What actually occurred is that for both statements (Lines 8 and 9), the member function `print` of the `class baseClass` is executed. This is due to the fact that the binding of the member function `print`, in the body of the function `callPrint`, occurred at compile time. Because the formal parameter `p` of the function `callPrint` is of type `baseClass`, for the statement `p.print();`, the compiler associates the function `print` of the `class baseClass`. More specifically, in **compile-time binding**, the necessary code to call a specific function is generated by the compiler. (Compile-time binding is also known as **static binding**.)

For the statement in Line 9, the actual parameter is of type `derivedClass`. Thus, when the body of the function `callPrint` executes, logically the `print` function of object `two` should execute, which is not the case. So, during program execution, how does C++ correct this problem of making the call to the appropriate function? C++ corrects this problem by providing the mechanism of **virtual functions**. The binding of virtual functions occurs at program execution time, not at compile time. This kind of binding is called **run-time binding**. More formally, in run-time binding, the compiler does not generate the code to call a specific function. Instead, it generates enough information to enable the run-time system to generate the specific code for the appropriate function call. Run-time binding is also known as **dynamic binding**.

In C++, virtual functions are declared using the reserved word `virtual`. Let us redefine the previous classes using this feature:

```
class baseClass
{
public:
    virtual void print();       //virtual function
    baseClass(int u = 0);

private:
    int x;
};
```

```
class derivedClass: public baseClass
{
public:
    void print();
    derivedClass(int u = 0, int v = 0);

private:
    int a;
};
```


Note that we need to declare a `virtual` function only in the base `class`.

The definition of the member function `print` is the same as before. If we execute the previous program with these modifications, the output is as follows.

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
```

This output shows that for the statement in Line 9, the `print` function of `derivedClass` is executed (see the last two lines of the output).

The previous discussion also applies when a formal parameter is a pointer to a class, and a pointer of the derived class is passed as an actual parameter. To illustrate this feature, suppose we have the preceding classes. (We assume that the definition of the `class baseClass` is in the header file `baseClass.h`, and the definition of the `class derivedClass` is in the header file `derivedClass.h`.) Consider the following program:

```
//*****************************************************
// Author: D.S. Malik
//
// This program illustrates how virtual functions and
// pointer formal parameters work.
//*****************************************************

#include <iostream>                          //Line 1

#include "derivedClass.h"                    //Line 2

using namespace std;                         //Line 3

void callPrint(baseClass *p);                //Line 4

int main()                                   //Line 5
{                                            //Line 6
    baseClass *q;                            //Line 7
    derivedClass *r;                         //Line 8
```

```
    q = new baseClass(5);                          //Line 9
    r = new derivedClass(3, 15);                   //Line 10

    q->print();                                    //Line 11
    r->print();                                    //Line 12

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;              //Line 13

    callPrint(q);                                  //Line 14
    callPrint(r);                                  //Line 15

    return 0;                                      //Line 16
}                                                  //Line 17

void callPrint(baseClass *p)
{
    p->print();
}
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
```

The preceding examples show that if a formal parameter, say `p` of a class type, is either a reference parameter or a pointer and `p` uses a virtual function of the base class, we can effectively pass a derived class object as an actual parameter to `p`.

However, if `p` is a *value parameter*, then this mechanism of passing a derived class object as an actual parameter to `p` does not work, even if `p` uses a virtual function. Recall that, if a formal parameter is a value parameter, the value of the actual parameter is copied into the formal parameter. Therefore, if a formal parameter is of a `class` type, the member variables of the actual object are copied into the corresponding member variables of the formal parameter.

Suppose that we have the classes defined above—that is, `baseClass` and `derivedClass`. Consider the following function definition:

```
void callPrint(baseClass p)  //p is a value parameter
{
    p.print();
}
```

Further suppose that we have the following declaration:

```
derivedClass two;
```

The object two has two member variables, x and a. The member variable x is inherited from the base class. Consider the following function call:

```
callPrint(two);
```

In this statement, because the formal parameter p is a value parameter, the member variables of two are copied into the member variables of p. However, because p is an object of type baseClass, it has only one member variable. Consequently, only the member variable x of two will be copied into the member variable x of p. Also, the statement:

```
p.print();
```

in the body of the function will result in executing the member function print of the class baseClass.

The output of the following program further illustrates this concept. (As before, we assume that the definition of the class baseClass is in the header file baseClass.h, and the definition of the class derivedClass is in the header file derivedClass.h.)

```
//*****************************************************
// Author: D.S. Malik
//
// This program illustrates how virtual functions and a
// pointer variable of base class as a formal parameter
// work.
//*****************************************************

#include <iostream>                              //Line 1

#include "derivedClass.h"                        //Line 2

using namespace std;                             //Line 3

void callPrint(baseClass p);                     //Line 4

int main()                                       //Line 5
{                                                //Line 6
    baseClass one(5);                            //Line 7
    derivedClass two(3, 15);                     //Line 8

    one.print();                                 //Line 9
    two.print();                                 //Line 10

    cout << "*** Calling the function "
         << "callPrint  ***" << endl;            //Line 11

    callPrint(one);                              //Line 12
    callPrint(two);                              //Line 13

    return 0;                                    //Line 14
}                                                //Line 15
```

```
void callPrint(baseClass p)  //p is a value parameter
{
    p.print();
}
```

Sample Run:

```
In baseClass x = 5
In derivedClass ***: In baseClass x = 3
In derivedClass a = 15
*** Calling the function callPrint  ***
In baseClass x = 5
In baseClass x = 3
```

Look closely at the output of the statements in Lines 12 and 13 (the last two lines of output). In Line 13, because the formal parameter `p` is a value parameter, the member variables of `two` are copied into the corresponding member variables of `p`. However, because `p` is an object of type `baseClass`, it has only one member variable. Consequently, only the member variable `x` of `two` is copied into the member variable `x` of `p`. Moreover, the statement `p.print();` in the function `callPrint` executes the function `print` of the `class baseClass`, not the `class derivedClass`. Therefore, the last line of the output shows only the value of x (the member variable of `two`).

NOTE An object of the base class type cannot be passed to a formal parameter of the derived class type.

Classes and Virtual Destructors

One thing recommended for classes with pointer member variables is that these classes should have the destructor. The destructor is automatically executed when the class object goes out of scope. Thus, if the object creates dynamic objects, the destructor can be designed to deallocate the storage for them. If a derived class object is passed to a formal parameter of the base class type, the destructor of the base class executes regardless of whether the derived class object is passed by reference or by value. Logically, however, the destructor of the derived class should be executed when the derived class object goes out of scope.

To correct this problem, the destructor of the base class must be virtual. The **virtual destructor** of a base class automatically makes the destructor of a derived class virtual. When a derived class object is passed to a formal parameter of the base class type, then when the object goes out of scope, the destructor of the derived class executes. After executing the destructor of the derived class, the destructor of the base class executes. Therefore, when the derived class object is destroyed, the base class part (that is, the members inherited from the base class) of the derived class object is also destroyed.

If a base class contains virtual functions, make the destructor of the base class virtual.

Abstract Classes and Pure Virtual Functions

The preceding section discussed virtual functions. Other than enforcing run-time binding of functions, virtual functions also have another use, which is discussed in this section. Chapter 2 described the second principal of OOD—inheritance. Through inheritance we can derive new classes without designing them from scratch. The derived classes, in addition to inheriting the existing members of the base class, can add their own members and also redefine or override public and protected member functions of the base class. The base class can contain functions that you would want each derived class to implement. There are many scenarios when a class is desired to be served as a base class for a number of derived classes, however, the base class may contain certain functions that may not have meaningful definitions in the base class.

Let us consider the `class shape` given in Chapter 2. As noted in that chapter, from the `class shape` you can derive other `class`es such as `rectangle`, `circle`, `ellipse`, and so on. Some of the things common to every shape are its center, using the center to move a shape to a different location, and drawing the shape. Among others, we can include these in the `class shape`. For example, you could have the definition of the `class shape` similar to the following:

```
class shape
{
public:
    virtual void draw();
      //Function to draw the shape.

    virtual void move(double x, double y);
      //Function to move the shape at the position (x, y).
    .
    .
    .
};
```

Because the definitions of the functions `draw` and `move` are specific to a particular shape, each derived class can provide an appropriate definition of these functions. Note that we have made the functions `draw` and `move` `virtual` to enforce run-time binding of these functions.

The way the definition of the `class shape` is written when you write the definition of the functions of the `class shape`, you must also write the definitions of the functions `draw` and `move`. However, at this point there is no shape to draw or move. Therefore, these function bodies have no code. One way to handle this is to make the body of these functions empty. This solution would work, but it has another drawback. Once we write the definitions of the functions of the `class shape`, then we could create an object of this class. Because there is no shape to work with, we would like to prevent the user from creating objects of the `class shape`. It follows that we would like to do the following two things—to not include the definitions of the functions `draw` and `move`, and to prevent the user from creating objects of the `class shape`.

Because we do not want to include the definitions of the functions `draw` and `move` of the `class shape`, we must convert these functions to **pure virtual functions**. In this case, the prototypes of these functions are:

```
virtual void draw() = 0;
virtual void move(double x, double y) = 0;
```

Note the expression `= 0` before the semicolon. Once you make these functions pure `virtual` functions in the `class shape`, you no longer need to provide the definitions of these functions for the `class shape`.

Once a class contains one or more pure virtual functions, then that class is called an **abstract class**. Thus, the abstract definition of the `class shape` is similar to the following:

```
class shape
{
public:
    virtual void draw() = 0;
      //Function to draw the shape. Note that this is a
      //pure virtual function.

    virtual void move(double x, double y) = 0;
      //Function to move the shape at the position (x, y).
      //Note that this is a pure virtual function.
    .
    .
    .
};
```

Because an abstract class is not a complete class, as it (or its implementation file) does not contain the definitions of certain functions, you cannot create objects of that class.

Now suppose that we derive the `class rectangle` from the `class shape`. To make `rectangle` a nonabstract class, so that we can create objects of this class, the class (or its implementation file) must provide the definitions of the pure `virtual` functions of its base class, which is the `class shape`.

Note that in addition to the pure virtual functions, an abstract class can contain instance variables, constructors, and functions that are not pure virtual. However, the abstract class must provide the definitions of constructor and functions that are not pure virtual.

Array-Based Lists

Everyone is familiar with the term *list*. You might have a list consisting of employee data, student data, sales data, or a list of rental properties. One thing common to all lists is that all the elements of a list are of the same type. More formally, we can define a list as follows:

List: A collection of elements of the same type.

The **length** of a list is the number of elements in the list.

Following are some of the operations performed on a list:

1. Create the list. The list is initialized to an empty state.
2. Determine whether the list is empty.
3. Determine whether the list is full.
4. Find the size of the list.
5. Destroy, or clear, the list.
6. Determine whether an item is the same as a given list element.
7. Insert an item in the list at the specified location.
8. Remove an item from the list at the specified location.
9. Replace an item at the specified location with another item.
10. Retrieve an item from the list from the specified location.
11. Search the list for a given item.

Before discussing how to implement these operations, we must first decide how to store the list in the computer's memory. Because all the elements of a list are of the same type, an effective and convenient way to process a list is to store it in an array. Initially, the size of the array holding the list elements is usually larger than the number of elements in the list so that, at a later stage, the list can grow. Thus, we must know how full the array is; that is, we must keep track of the number of list elements stored in the array. C++ allows the programmer to create dynamic arrays. Therefore, we leave it for the user to specify the size of the array. The size of the array can be specified when a list object is declared. It follows that, to maintain and process the list in an array, we need the following three variables:

- The array holding the list elements
- A variable to store the length of the list (that is, the number of list elements currently in the array)
- A variable to store the size of the array (that is, the maximum number of elements that can be stored in the array)

Suppose that the variable `length` indicates the number of elements in the list and that `maxSize` indicates the maximum number of elements that can be stored in the list. Then `length` and `maxSize` are nonnegative integers and, therefore, we can declare them to be of type `int`. What about the type of the array, that is, the data type of the array elements? If we have a list of numbers, the array elements could be of type `int` or `double`. If we have a list of names, the array elements are of type `string`. Similarly, if we have a list of students, the array elements are of type `studentType` (a data type you can define). As you can see, there are various types of lists.

A list of sales data or a list of students' data is empty if its length is zero. To insert an item at the end of a list of any type would require you to add the element after the current last element and then increment the length by one. Similarly, it can be seen that, for the most part, the algorithms to implement operations on a list of names, on a list of sales data, or on a list of students' data are the same. We do not want to spend time and efforts to

develop separate code for each type of list we encounter. Instead, we want to develop a generic code that can be used to implement any type of list in a program. In other words, while designing the algorithms, we do not want to be concerned whether we are processing a list of numbers, a list of names, or a list of students' data. However, while illustrating a particular algorithm, we will consider a specific type of list. To develop generic algorithms to implement list operations, we make use of class templates.

Now that we know the operations to be performed on a list and how to store the list into the computer's memory, next we define the class implementing the list as an abstract data type (ADT). The following `class`, `arrayListType`, defines the list as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement the basic
// properties of array-based lists.
//*************************************************************

template <class elemType>
class arrayListType
{
public:
    const arrayListType<elemType>& operator=
                        (const arrayListType<elemType>&);
      //Overloads the assignment operator
    bool isEmpty() const;
      //Function to determine whether the list is empty
      //Postcondition: Returns true if the list is empty;
      //    otherwise, returns false.
    bool isFull() const;
      //Function to determine whether the list is full.
      //Postcondition: Returns true if the list is full;
      //    otherwise, returns false.
    int listSize() const;
      //Function to determine the number of elements in the list
      //Postcondition: Returns the value of length.
    int maxListSize() const;
      //Function to determine the size of the list.
      //Postcondition: Returns the value of maxSize.
    void print() const;
      //Function to output the elements of the list
      //Postcondition: Elements of the list are output on the
      //   standard output device.
    bool isItemAtEqual(int location, const elemType& item) const;
      //Function to determine whether the item is the same
      //as the item in the list at the position specified by
      //Postcondition: Returns true if list[location]
      //    is the same as the item; otherwise,
      //    returns false.
   void insertAt(int location, const elemType& insertItem);
      //Function to insert an item in the list at the
      //position specified by location. The item to be inserted
      //is passed as a parameter to the function.
```

```
    //Postcondition: Starting at location, the elements of the
    //    list are shifted down, list[location] = insertItem;,
    //    and length++;. If the list is full or location is
    //    out of range, an appropriate message is displayed.
  void insertEnd(const elemType& insertItem);
    //Function to insert an item at the end of the list.
    //The parameter insertItem specifies the item to be inserted.
    //Postcondition: list[length] = insertItem; and length++;
    //    If the list is full, an appropriate message is
    //    displayed.
  void removeAt(int location);
    //Function to remove the item from the list at the
    //position specified by location
    //Postcondition: The list element at list[location] is removed
    //    and length is decremented by 1. If location is out of
    //    range, an appropriate message is displayed.
  void retrieveAt(int location, elemType& retItem) const;
    //Function to retrieve the element from the list at the
    //position specified by location.
    //Postcondition: retItem = list[location]
    //    If location is out of range, an appropriate message is
    //    displayed.
  void replaceAt(int location, const elemType& repItem);
    //Function to replace the elements in the list at the
    //position specified by location. The item to be replaced
    //is specified by the parameter repItem.
    //Postcondition: list[location] = repItem
    //    If location is out of range, an appropriate message is
    //    displayed.
  void clearList();
    //Function to remove all the elements from the list.
    //After this operation, the size of the list is zero.
    //Postcondition: length = 0;
  int seqSearch(const elemType& item) const;
    //Function to search the list for a given item.
    //Postcondition: If the item is found, returns the location
    //    in the array where the item is found; otherwise,
    //    returns -1.
  void insert(const elemType& insertItem);
    //Function to insert the item specified by the parameter
    //insertItem at the end of the list. However, first the
    //list is searched to see whether the item to be inserted
    //is already in the list.
    //Postcondition: list[length] = insertItem and length++
    //     If the item is already in the list or the list
    //     is full, an appropriate message is displayed.
  void remove(const elemType& removeItem);
    //Function to remove an item from the list. The parameter
    //removeItem specifies the item to be removed.
    //Postcondition: If removeItem is found in the list,
    //      it is removed from the list and length is
    //      decremented by one.
```

```
    arrayListType(int size = 100);
      //constructor
      //Creates an array of the size specified by the
      //parameter size. The default array size is 100.
      //Postcondition: The list points to the array, length = 0,
      //    and maxSize = size

    arrayListType(const arrayListType<elemType>& otherList);
      //copy constructor

    ~arrayListType();
      //destructor
      //Deallocates the memory occupied by the array.

protected:
    elemType *list;  //array to hold the list elements
    int length;      //to store the length of the list
    int maxSize;     //to store the maximum size of the list
};
```

Figure 3-29 shows the UML class diagram of the `class arrayListType`.

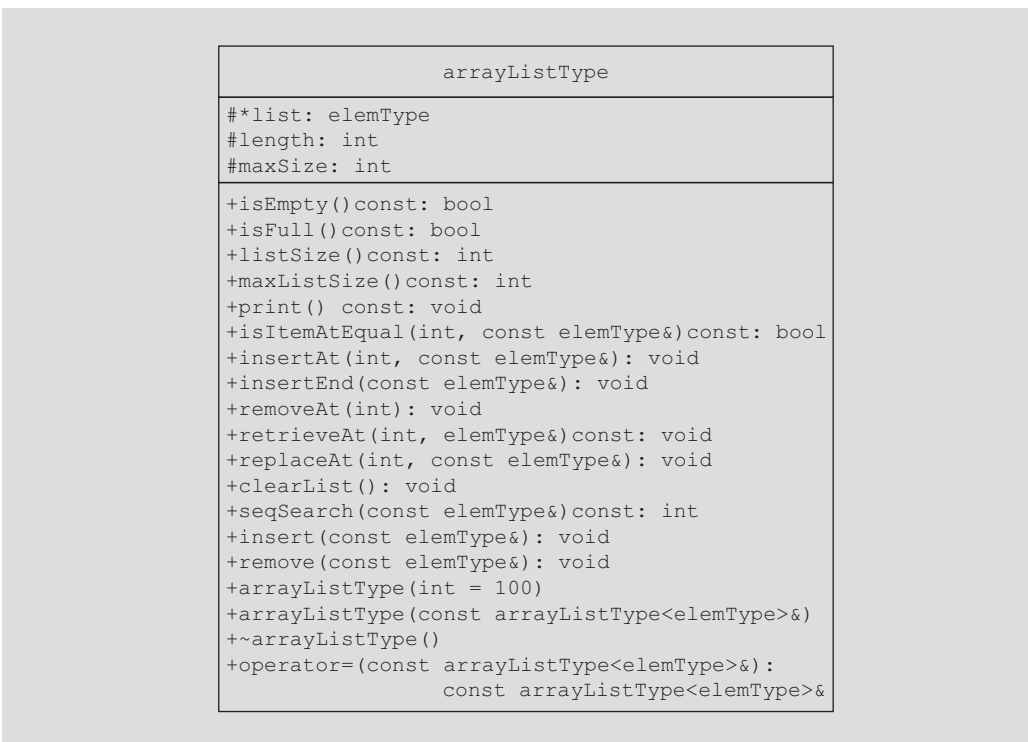


FIGURE 3-29 UML class diagram of the `class arrayListType`

Notice that the data members of the **class** **arrayListType** are declared as **protected**. This is because we want to derive classes from this class to implement special lists such as an ordered list. Next, we write the definitions of these functions.

The list is empty if **length** is **zero**; it is full if **length** is equal to **maxSize**. Therefore, the definitions of the functions **isEmpty** and **isFull** are as follows:

```
template <class elemType>
bool arrayListType<elemType>::isEmpty() const
{
    return (length == 0);
}

template <class elemType>
bool arrayListType<elemType>::isFull() const
{
    return (length == maxSize);
}
```

The data member **length** of the class stores the number of elements currently in the list. Similarly, because the size of the array holding the list elements is stored in the data member **maxSize**, **maxSize** specifies the maximum size of the list. Therefore, the definitions of the functions **listSize** and **maxListSize** are as follows:

```
template <class elemType>
int arrayListType<elemType>::listSize() const
{
    return length;
}

template <class elemType>
int arrayListType<elemType>::maxListSize() const
{
    return maxSize;
}
```

Each of the functions **isEmpty**, **isFull**, **listSize**, and **maxListSize** contain only one statement, which is either a comparison statement or a statement returning a value. It follows that each of these functions is of $O(1)$.

The member function **print** outputs the elements of the list. We assume that the output is sent to the standard output device.

```
template <class elemType>
void arrayListType<elemType>::print() const
{
    for (int i = 0; i < length; i++)
        cout << list[i] << " ";

    cout << endl;
}
```

The function **print** uses a loop to output the elements of the list. The number of times the **for** loop executes depends on the number of elements of the list. If the list has 100

elements, the **for** loop executes 100 times. In general, suppose that the number of elements in the list is *n*. Then the function `print` is of *O*(*n*).

The definition of the function `isItemAtEqual` is straightforward.

```
template <class elemType>
bool arrayListType<elemType>::isItemAtEqual
                            (int location, const elemType& item) const
{
    return(list[location] == item);
}
```

The body of function `isItemAtEqual` has only one statement, which is a comparison statement. It is easy to see that this function is of *O*(1).

The function `insertAt` inserts an item at a specific location in the list. The item to be inserted and the insert location in the array are passed as parameters to this function. To insert the item somewhere in the middle of the list, we must first make room for the new item. That is, we need to move certain elements right one array slot. Suppose that the data member `list` of an `arrayListType` object is as shown in Figure 3-30. (Note that this figure does not show the data members `length` and `maxSize`.)

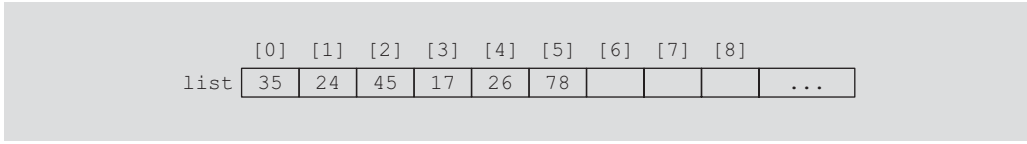


FIGURE 3-30 Array `list`

The number of elements currently in the list is `6`, so `length` is `6`. Thus, after inserting a new element, the `length` of the list is `7`. If the item is to be inserted at, say location `6`, we can easily accomplish this by copying the item into `list[6]`. On the other hand, if the item is to be inserted at, say location `3`, we first need to move elements `list[3]`, `list[4]`, and `list[5]` one array slot right to make room for the new item. Thus, we must first copy `list[5]` into `list[6]`, `list[4]` into `list[5]`, and `list[3]` into `list[4]`, in this order. Then we can copy the new item into `list[3]`.

Of course, special cases such as trying to insert in a full list must be handled separately. Other member functions can handle some of these cases.

The definition of the function `insertAt` is as follows:

```
template <class elemType>
void arrayListType<elemType>::insertAt
                    (int location, const elemType& insertItem)
{
    if (location < 0 || location >= maxSize)
        cerr << "The position of the item to be inserted "
             << "is out of range" << endl;
```

```
    else
        if (length >= maxSize)  //list is full
            cerr << "Cannot insert in a full list" << endl;
        else
        {
            for (int i = length; i > location; i--)
                 list[i] = list[i - 1];   //move the elements down

            list[location] = insertItem;  //insert the item at the
                                          //specified position

            length++;     //increment the length
        }
} //end insertAt
```


The function `insertAt` uses a **`for`** loop to shift the elements of the list. The number of times the **`for`** loop executes depends on where in the list the item is to be inserted. If the item is to be inserted at the first position, all the elements of the list are shifted. It can be easily shown that this function is of $O(n)$.

The function `insertEnd` can be implemented by using the function `insertAt`. However, the function `insertEnd` does not require the shifting of elements. Therefore, we give its definition directly.

```
template <class elemType>
void arrayListType<elemType>::insertEnd(const elemType& insertItem)
{
    if (length >= maxSize)  //the list is full
        cerr << "Cannot insert in a full list" << endl;
    else
    {
         list[length] = insertItem;   //insert the item at the end
         length++;   //increment the length
    }
} //end insertEnd
```

The number of statements and, hence, the number of operations executed in the body of the function `insertEnd` are fixed. Therefore, this function is of $O(1)$.

The function `removeAt` is the opposite of the function `insertAt`. The function `removeAt` removes an item from a specific location in the list. The location of the item to be removed is passed as a parameter to this function. After removing the item from the list, the length of the list is reduced by `1`. If the item to be removed is somewhere in the middle of the list, after removing the item we must move certain elements left one array slot because we cannot leave holes in the portion of the array containing the list. Suppose that the data member `list` of an `arrayListType` object is as shown in Figure 3-31. (Note that this figure does not show the data members `length` and `maxSize`.)

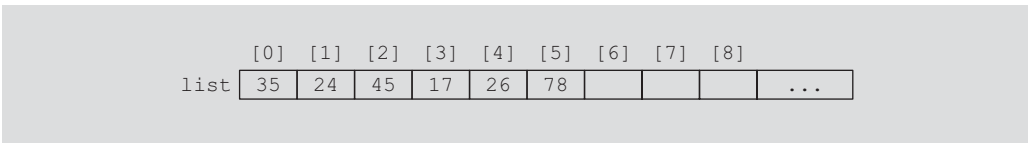


FIGURE 3-31 Array `list`

The number of elements currently in the list is `6`, so `length` is `6`. Thus, after removing an element, the `length` of the list is `5`. Suppose that the item to be removed is at, say location `3`. Clearly, we must move `list[4]` into `list[3]` and `list[5]` into `list[4]`, in this order.

The definition of the function `removeAt` is as follows:

```
template <class elemType>
void arrayListType<elemType>::removeAt(int location)
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be removed "
             << "is out of range" << endl;
    else
    {
        for (int i = location; i < length - 1; i++)
            list[i] = list[i+1];

        length--;
    }
} //end removeAt
```

Similar to the function `insertAt`, it is easily seen that the function `removeAt` is of $O(n)$.

The definition of the function `retrieveAt` is straightforward. The index of the item to be retrieved, and the location where to retrieve the item, are passed as parameters to this function. Similarly, the definition of the function `replaceAt` is straightforward. The definitions of these functions are as follows:

```
template <class elemType>
void arrayListType<elemType>::retrieveAt
                              (int location, elemType& retItem) const
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be retrieved is "
             << "out of range." << endl;
    else
        retItem = list[location];
} //end retrieveAt


template <class elemType>
void arrayListType<elemType>::replaceAt
                            (int location, const elemType& repItem)
```

```
{
    if (location < 0 || location >= length)
        cerr << "The location of the item to be replaced is "
             << "out of range." << endl;
    else
        list[location] = repItem;

} //end replaceAt
```


The function `clearList` removes the elements from the list, leaving it empty. Because the data member `length` indicates the number of elements in the list, the elements are removed by simply setting `length` to `0`. Therefore, the definition of this function is as follows:

```
template <class elemType>
void arrayListType<elemType>::clearList()
{
    length = 0;
} //end clearList
```

We now discuss the definition of the constructor and the destructor. The constructor creates an array of the size specified by the user, and initializes the `length` of the list to `0` and the `maxSize` to the size of the array specified by the user. The size of the array is passed as a parameter to the constructor. The default array size is 100. The destructor deallocates the memory occupied by the array holding the list elements. The definition of the constructor and the destructor are as follows:

```
template <class elemType>
arrayListType<elemType>::arrayListType(int size)
{
    if (size < 0)
    {
        cerr << "The array size must be positive. Creating "
             << "an array of size 100. " << endl;

        maxSize = 100;
    }
    else
        maxSize = size;

    length = 0;

    list = new elemType[maxSize];
    assert(list != NULL);
}

template <class elemType>
arrayListType<elemType>::~arrayListType()
{
    delete [] list;
}
```

As before, it is easy to see that each of the functions `retrieveAt`, `replaceAt`, `clearList`, as well as the constructor and destructor is of $O(1)$.

Copy Constructor

Recall that the copy constructor is called when an object is passed as a (value) parameter to a function, and when an object is declared and initialized using the value of another object of the same type. It copies the data members of the actual object into the corresponding data members of the formal parameter and the object being created. Its definition is as follows:

```
template <class elemType>
arrayListType<elemType>::arrayListType
                    (const arrayListType<elemType>& otherList)
{
    maxSize = otherList.maxSize;
    length = otherList.length;
    list = new elemType[maxSize]; //create the array
    assert(list != NULL);         //terminate if unable to allocate
                                  //memory space

    for (int j = 0; j < length; j++)  //copy otherList
        list [j] = otherList.list[j];
} //end copy constructor
```

Overloading the Assignment Operator

Next, because we are overloading the assignment operator for the `class arrayListType`, we give the definition of the function template to overload the assignment operator.

```
template <class elemType>
const arrayListType<elemType>& arrayListType<elemType>::operator=
                      (const arrayListType<elemType>& otherList)
{
    if (this != &otherList)   //avoid self-assignment
    {
        delete [] list;
        maxSize = otherList.maxSize;
        length = otherList.length;

        list = new elemType[maxSize];  //create the array
        assert(list != NULL);   //if unable to allocate memory
                                //space, terminate the program
        for (int i = 0; i < length; i++)
            list[i] = otherList.list[i];
    }

    return *this;
}
```

Similar to the function `print`, it is easy to see that both the copy constructor and the function to overload the assignment operator are of $O(n)$.

Search

The search algorithm described next is called a **sequential** or **linear** search.

Consider the list of seven elements shown in Figure 3-32.

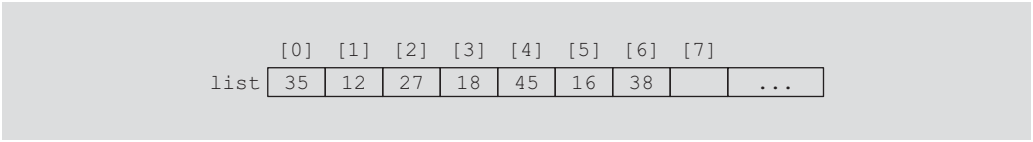


FIGURE 3-32 List of seven elements

Suppose that you want to determine whether `27` is in the list. The sequential search works as follows: First, you compare `27` with `list[0]`—that is, compare `27` with `35`. Because `list[0]` $\neq$ `27`, you then compare `27` with `list[1]` (that is, with `12`, the second item in the list). Because `list[1]` $\neq$ `27`, you compare `27` with the next element in the list—that is, compare `27` with `list[2]`. Because `list[2]` = `27`, the search stops. This is a successful search.

Let us now search for `10`. As before, the search starts with the first element in the list—that is, at `list[0]`. This time the search item, which is `10`, is compared with every item in the list. Eventually, no more data is left in the list to compare with the search item. This is an unsuccessful search.

It now follows that, as soon as you find an element in the list that is equal to the search item, you must stop the search and report "success." (In this case, you usually also tell the location in the list where the search item was found.) Otherwise, after the search item is compared with every element in the list, you must stop the search and report "failure."

Suppose that the name of the array containing the list elements is `list`. The following function performs a sequential search on a list:

```
template <class elemType>
int arrayListType<elemType>::seqSearch(const elemType& item) const
{
    int loc;
    bool found = false;

    for (loc = 0; loc < length; loc++)
        if (list[loc] == item)
        {
            found = true;
            break;
        }
```

```
    if (found)
        return loc;
    else
        return -1;
} //end seqSearch
```

Now that we know how to implement the (sequential) search algorithm, we can give the definitions of the functions `insert` and `remove`. Recall that the function `insert` inserts a new item at the end of the list if this item does not exist in the list and the list is not full. The function `remove` removes an item from the list if the list is not empty.

Chapter 9 explicitly shows that the function `seqSearch` is of $O(n)$.

Insert

The function `insert` inserts a new item in the list. Because duplicates are not allowed, this function first searches the list to determine whether the item to be inserted is already in the list. To determine whether the item to be inserted is already in the list, this function calls the member function `seqSearch`, described previously. If the item to be inserted is not in the list, the new item is inserted at the end of the list and the length of the list is increased by 1. Also, the item to be inserted is passed as a parameter to this function. The definition of this function is as follows:

```
template <class elemType>
void arrayListType<elemType>::insert(const elemType& insertItem)
{
    int loc;

    if (length == 0)   //list is empty
        list[length++] = insertItem;    //insert the item and
                                        //increment the length
    else if (length == maxSize)
        cerr << "Cannot insert in a full list." << endl;
    else
    {
        loc = seqSearch(insertItem);

        if (loc == -1)    //the item to be inserted
                          //does not exist in the list
            list[length++] = insertItem;
        else
            cerr << "the item to be inserted is already in "
                 << "the list. No duplicates are allowed." << endl;
    }
} //end insert
```

The function `insert` uses the function `seqSearch` to determine whether the `insertItem` is already in the list. Because the function `seqSearch` is of $O(n)$, it follows that the function `insert` is of $O(n)$.

Remove

The function `remove` deletes an item from the list. The item to be deleted is passed as a parameter to this function. To delete the item, the function calls the member function `seqSearch` to determine whether the item to be deleted is in the list. If the item to be deleted is found in the list, the item is removed from the list and the length of the list is decremented by 1. If the item to be removed is found in the list, the function `seqSearch` returns the `index` of the item in the list to be deleted. We can now use the `index` returned by the function `seqSearch`, and use the function `removeAt` to remove the item from the list. Therefore, the definition of the function `remove` is as follows:

```
template<class elemType>
void arrayListType<elemType>::remove(const elemType& removeItem)
{
    int loc;

    if (length == 0)
        cerr << "Cannot delete from an empty list." << endl;
    else
    {
        loc = seqSearch(removeItem);

        if (loc != -1)
            removeAt(loc);
        else
            cout << "The item to be deleted is not in the list."
                 << endl;
    }
} //end remove
```

The function `remove` uses the functions `seqSearch` and `removeAt` to remove an item from the list. Because each of these functions is of $O(n)$ and because they are called in sequence, it follows that the function `remove` is of $O(n)$.

Time Complexity of List Operations

The following table summarizes the time complexity of list operations.

TABLE 3-1 Time complexity of list operations

Function	Time-complexity
`isEmpty`	$O(1)$
`isFull`	$O(1)$
`listSize`	$O(1)$
`maxListSize`	$O(1)$

TABLE 3-1 Time complexity of list operations (continued)

Function	Time-complexity
print	$O(n)$
isItemAtEqual	$O(1)$
insertAt	$O(n)$
insertEnd	$O(1)$
removeAt	$O(n)$
retrieveAt	$O(1)$
replaceAt	$O(n)$
clearList	$O(1)$
constructor	$O(1)$
destructor	$O(1)$
copy constructor	$O(n)$
overloading the assignment operator	$O(n)$
seqSearch	$O(n)$
insert	$O(n)$
remove	$O(n)$

The following program tests the various operations on array-based lists.

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the class arrayListType.
//*****************************************************************

#include <iostream>                                          //Line 1

#include <string>                                            //Line 2
#include "arrayListType.h"                                   //Line 3
```

```
using namespace std;                                              //Line 4

int main()                                                        //Line 5
{                                                                 //Line 6
    arrayListType<int> intList(100);                              //Line 7
    arrayListType<string> stringList;                             //Line 8

    int number;                                                   //Line 9

    cout << "List 10: Enter 5 integers: ";                        //Line 10

    for (int counter = 0; counter < 5; counter++)                 //Line 11
    {                                                             //Line 12
        cin >> number;                                            //Line 13
        intList.insertAt(counter, number);                        //Line 14
    }                                                             //Line 15

    cout << endl;                                                 //Line 16
    cout << "List 19: The list you entered is: ";                 //Line 17
    intList.print();                                              //Line 18
    cout << endl;                                                 //Line 19

    cout << "Line 20: Enter the item to be deleted: ";            //Line 20
    cin >> number;                                                //Line 21
    intList.remove(number);                                       //Line 22
    cout << "Line 23: After removing " << number
         << ", the list is:" << endl;                             //Line 23
    intList.print();                                              //Line 24
    cout << endl;                                                 //Line 25

    string str;                                                   //Line 26

    cout << "Line 27: Enter 5 strings: ";                         //Line 27

    for (int counter = 0; counter < 5; counter++)                 //Line 28
    {                                                             //Line 29
        cin >> str;                                               //Line 30
        stringList.insertAt(counter, str);                        //Line 31
    }                                                             //Line 32

    cout << endl;                                                 //Line 33
    cout << "Line 34: The list you entered is: " << endl;         //Line 34
    stringList.print();                                           //Line 35
    cout << endl;                                                 //Line 36

    cout << "Line 37: Enter the string to be deleted: ";          //Line 37
    cin >> str;                                                   //Line 38
    stringList.remove(str);                                       //Line 39
    cout << "Line 40: After removing " << str
         << ", the list is:" << endl;                             //Line 40
```

```
    stringList.print();                                          //Line 41
    cout << endl;                                                //Line 42

    return 0;                                                    //Line 43
}                                                                //Line 44
```

Sample Run: In this sample run, the user input is shaded.

```
List 10: Enter 5 integers: 23 78 56 12 79

List 19: The list you entered is: 23 78 56 12 79

Line 20: Enter the item to be deleted: 56
Line 23: After removing 56, the list is:
23 78 12 79

Line 27: Enter 5 strings: hello sunny warm winter summer

Line 34: The list you entered is:
hello sunny warm winter summer

Line 37: Enter the string to be deleted: hello
Line 40: After removing hello, the list is:
sunny warm winter summer
```

The preceding program works as follows. The statement in Line 7 declares `intList` to be an object of type `arrayListType`. The data member `list` of `intList` is an array of 100 components and the component type is `int`. The statement in Line 8 declares `stringList` to be an object of type `arrayListType`. The data member `list` of `stringList` is an array of 100 components (the default size) and the component type is `string`. The statement in Line 10 prompts the user to enter five integers. The statement in Line 13 gets the next number from the input stream. The statement in Line 14 uses the member function `insertAt` of `intList` to store the number into `intList`. The statement in Line 18 uses the member function `print` of `intList` to output the elements of `intList`. The statement in Line 20 prompts the user to enter the number to be deleted from `intList`; the statement in Line 21 gets the number to be deleted from the input stream. The statement in Line 22 uses the member function `remove` of `intList` to remove the number from `intList`.

The statements in Lines 27 through 42 work the same way as the statements in Lines 10 through 25. These statements process a list of strings.

PROGRAMMING EXAMPLE: Polynomial Operations

You learned in a college algebra or calculus course that a polynomial, $p(x)$, in one variable, x, is an expression of the form:

$$p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n,$$

where a_i are real (or complex) numbers and n is a nonnegative integer. If $p(x) = a_0$, $p(x)$ is called a **constant** polynomial. If $p(x)$ is a nonzero constant polynomial, the degree of $p(x)$ is defined to be 0. Even though, in mathematics, the degree of the zero polynomial is undefined, for the purpose of this program, we consider the degree of such polynomials to be zero. If $p(x)$ is not constant and $a_n \neq 0$, n is called the degree of $p(x)$; that is, the degree of a nonconstant polynomial is defined to be the exponent of the highest power of x. (Note that the symbol $\neq$ means not equal to.)

The basic operations performed on polynomials are add, subtract, multiply, divide, and evaluate a polynomial at any given point. For example, suppose that

$$p(x) = 1 + 2x + 3x^2,$$

and

$$q(x) = 4 + x.$$

The degree of $p(x)$ is 2 and the degree of $q(x)$ is 1. Moreover,

$$p(2) = 1 + 2 \cdot 2 + 3 \cdot 2^2 = 17$$
$$p(x) + q(x) = 5 + 3x + 3x^2$$
$$p(x) - q(x) = -3 + x + 3x^2$$
$$p(x)^*q(x) = 4 + 9x + 14x^2 + 3x^3$$

The purpose of this programming example is to design and implement the `class polynomialType` to perform the various polynomial operations in a program.

To be specific, in this program, we will implement the following operations on polynomials:

1. Evaluate a polynomial at a given value.
2. Add polynomials.
3. Subtract polynomials.
4. Multiply polynomials.

Furthermore, we assume that the coefficients of polynomials are real numbers. You will be asked in Programming Exercise 8 to generalize it so that the coefficients can also be complex numbers.To store a polynomial, we use a dynamic array as follows:

Suppose $p(x)$ is a polynomial of degree $n \geq 0$. Let `list` be an array of size $n + 1$. The coefficient a_i of x^i is stored in `list[i]`. See Figure 3-33.

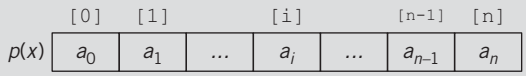


FIGURE 3-33 Polynomial $p(x)$

Figure 3-33 shows that if $p(x)$ is a polynomial of degree n, we need an array of size $n+1$ to store the coefficients of $p(x)$. Suppose that $p(x) = 1 + 8x - 3x^2 + 5x^4 + 7x^8$. Then the array storing the coefficient of $p(x)$ is given in Figure 3-34.

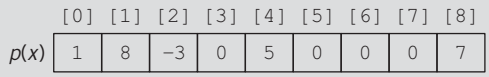


FIGURE 3-34 Polynomial $p(x)$ of degree 8 and its coefficients

Similarly, if $q(x) = -5x^2 + 16x^5$, the array storing the coefficient of $q(x)$ is given in Figure 3-35.

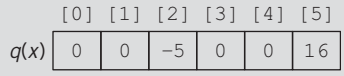


FIGURE 3-35 Polynomial $q(x)$ of degree 5 and its coefficients

Next, we define the operations +, –, and ⋆. Suppose that

$$p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n \text{ and}$$
$$q(x) = b_0 + b_1x + \ldots + b_{m-1}x^{m-1} + a_mx^m.$$

Let $t = max(n, m)$. Then

$$p(x) + q(x) = c_0 + c_1x + \ldots + c_{t-1}x^{t-1} + c_tx^t,$$

where for $i = 0, 1, 2, \ldots, t$

$$c_i = \begin{cases} a_i + b_i & if\ i \leq \min(n, m) \\ a_i & if\ i > m \\ b_i & if\ i > n \end{cases}$$

The difference, *p*(*x*) - *q*(*x*), of *p*(*x*) and *q*(*x*) can be defined similarly. It follows that the degree of the polynomials is $\leq$ *max*(*n*, *m*).

The product, *p*(*x*) * *q*(*x*), of *p*(*x*) and *q*(*x*) is defined as follows:

$p(x) * q(x) = d_0 + d_1x + \ldots + d_{n+m}x^{n+m}$,

The coefficient d_k, for $k = 0, 1, 2, \ldots, t$, is given by the formula

$d_k = a_0 * b_k + a_1 * b_{k-1} + \ldots + a_k * b_0$,

where if either a_i or b_i does not exist, it is assumed to be zero. For example,

$d_0 = a_0b_0$

$d_1 = a_0b_1 + a_1b_0$

...

$d_{n+m} = a_nb_m$

In Chapter 2, you learned how to overload various operators. This program overloads the operators `+`, `-`, and `*` to perform polynomial addition, subtraction, and multiplication. Moreover, we also overload the function call operator, `()`, to evaluate a polynomial at a given value. To simplify the input and output of polynomials, the operators `<<` and `>>` are also overloaded.

Because the coefficients of a polynomial are stored in a dynamic array, we use the `class arrayListType` to store and manipulate the coefficients of a polynomial. In fact, we derive the `class polynomialType` to implement polynomial operations from the `class arrayListType`, which requires us to implement only the operations needed to manipulate polynomials.

The following class defines polynomials as an ADT:

```
//***********************************************************
// Author: D.S. Malik
//
// This class specifies the members to implement the basic
// polynomial operations.
//***********************************************************

class polynomialType: public arrayListType<double>
{
    friend ostream& operator<<(ostream&, const polynomialType&);
      //Overloads the stream insertion operator
    friend istream& operator>>(istream&, polynomialType&);
      //Overloads the stream extraction operator
```

```
public:
    polynomialType operator+(const polynomialType&);
      //Overloads the operator +
    polynomialType operator-(const polynomialType&);
      //Overloads the operator -
    polynomialType operator*(const polynomialType&);
      //Overloads the operator *

    double operator() (double x);
      //Overloads the operator () to evaluate the polynomial at
      //a given point
      //Postcondition: The value of the polynomial at x is
      //    calculated and returned

    polynomialType(int size = 100);
      //constructor

    int min(int x, int y) const;
      //Function to return the smaller of x and y
    int max(int x, int y) const;
      //Function to return the larger of x and y
};
```

In Exercise 24 (at the end of this chapter), you are asked to draw the UML diagram of the `class polynomialType`.

If $p(x)$ is a polynomial of degree 3, we can create an object, say p, of type `polynomialType` and set the size of the array `list` to `4`. The following statement declares such an object p:

```
polynomialType p(4);
```

The degree of the polynomial is stored in the data member `length`, which is inherited from the `class arrayListType`.

Next we discuss the definitions of the functions.

The constructor sets the value of `length` to the size of the array and initializes the array `list` to `0`.

```
polynomialType::polynomialType(int size)
              : arrayListType<double>(size)
{
    length = size;

    for (int i = 0; i < size; i++)
        list[i] = 0;
}
```

The definition of the function to overload the operator `()` is quite straightforward and is given next.

```
double polynomialType::operator() (double x)
{
    double value = 0.0;

    for (int i = 0; i < length; i++)
    {
        if (list[i] != 0.0)
            value = value + list[i] * pow(x,i);
    }

    return value;
}
```

Suppose that $p(x)$ is a polynomial of degree n and $q(x)$ is a polynomial of degree m. If $n = m$, the operator + adds the corresponding coefficients of $p(x)$ and $q(x)$. If $n > m$, the first m coefficients of $p(x)$ are added with the corresponding coefficients of $q(x)$. The remaining coefficients of $p(x)$ are copied into the polynomial containing the sum of $p(x)$ and $q(x)$. Similarly, if $n < m$, the first n coefficients of $q(x)$ are added with the corresponding coefficients of $p(x)$. The remaining coefficients of $q(x)$ are copied into the polynomial containing the sum. The definition of the operator – is similar to the definition of the operator +. The definitions of these two operator functions are as follows:

```
polynomialType polynomialType::operator+
                              (const polynomialType& right)
{
    int size = max(length, right.length);

    polynomialType temp(size); //polynomial to store the sum

    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] + right.list[i];

    if (size == length)
        for (int i = min(length, right.length); i < length; i++)
            temp.list[i] = list[i];
    else
        for (int i = min(length, right.length); i < right.length;
                                                i++)
            temp.list[i] = right.list[i];

    return temp;
}

polynomialType polynomialType::operator-
                               (const polynomialType& right)
{
    int size = max(length, right.length);

   polynomialType temp(size); //polynomial to store the difference
```

```
    for (int i = 0; i < min(length, right.length); i++)
        temp.list[i] = list[i] - right.list[i];

    if (size == length)
        for (int i = min(length, right.length); i < length; i++)
            temp.list[i] = list[i];
    else
        for (int i = min(length, right.length); i < right.length;
                                                i++)
            temp.list[i] = -right.list[i];

    return temp;
}
```

The definition of the function to overload the operator * to multiply two polynomials is left as an exercise for you. See Programming Exercise 6 at the end of this chapter. The definitions of the remaining functions of the `class polynomialType` are as follows:

```
int polynomialType::min(int x, int y) const
{
    if (x <= y)
        return x;
    else
        return y;
}

int polynomialType::max(int x, int y) const
{
    if (x >= y)
        return x;
    else
        return y;
}

ostream& operator<<(ostream& os, const polynomialType& p)
{
   int indexFirstNonzeroCoeff = 0;

   for (int i = 0; i < p.length; i++)  //determine the index of the
                                       //first nonzero coefficient
        if (p.list[i] != 0.0)
        {
            indexFirstNonzeroCoeff = i;
            break;
        }

    if (indexFirstNonzeroCoeff < p.length)
    {
        if (indexFirstNonzeroCoeff == 0)
            os << p.list[indexFirstNonzeroCoeff] << " ";
```

```
        else
            os << p.list[indexFirstNonzeroCoeff] << "x^"
               << indexFirstNonzeroCoeff << " ";

        for (int i = indexFirstNonzeroCoeff + 1; i < p.length; i++)
        {
            if (p.list[i] != 0.0)
                if (p.list[i] >= 0.0)
                    os << "+ " << p.list[i] << "x^" << i << " ";
                else
                    os << "- " << -p.list[i] << "x^" << i << " ";
        }
    }
    else
        os << "0";

    return os;
}

istream& operator>>(istream& is, polynomialType& p)
{
    cout << "The degree of this polynomial is: "
         << p.length - 1 << endl;

    for (int i = 0; i < p.length; i++)
    {
        cout << "Enter the coefficient of x^" << i << ": ";
        is >> p.list[i];
    }

    return is;
}
```

MAIN PROGRAM

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the class polynomialType.
//*****************************************************************

#include <iostream>                                          //Line 1

#include "polynomialType.h"                                  //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    polynomialType p(8);                                     //Line 6
    polynomialType q(4);                                     //Line 7
    polynomialType t;                                        //Line 8
```

```
    cin >> p;                                                  //Line 9
    cout << endl << "Line 10: p(x): " << p << endl;            //Line 10

    cout << "Line 11: p(5): " << p(5) << endl << endl; //Line 11

    cin >> q;                                                  //Line 12
    cout << endl << "Line 13: q(x): " << q << endl
         << endl;                                              //Line 13

    t = p + q;                                                 //Line 14

    cout << "Line 15: p(x) + q(x): " << t << endl;             //Line 15

    cout << "Line 16: p(x) - q(x): " << p - q << endl; //Line 16

    return 0;                                                  //Line 17
}                                                              //Line 18
```

Sample Run: In this sample run, the user input is shaded.

```
The degree of this polynomial is: 7
Enter the coefficient of x^0: 0
Enter the coefficient of x^1: 1
Enter the coefficient of x^2: 4
Enter the coefficient of x^3: 0
Enter the coefficient of x^4: 0
Enter the coefficient of x^5: 0
Enter the coefficient of x^6: 0
Enter the coefficient of x^7: 6

Line 10: p(x): 1x^1 + 4x^2 + 6x^7
Line 11: p(5): 468855

The degree of this polynomial is: 3
Enter the coefficient of x^0: 1
Enter the coefficient of x^1: 2
Enter the coefficient of x^2: 0
Enter the coefficient of x^3: 3

Line 13: q(x): 1 + 2x^1 + 3x^3

Line 15: p(x) + q(x): 1 + 3x^1 + 4x^2 + 3x^3 + 6x^7
Line 16: p(x) - q(x): -1 - 1x^1 + 4x^2 - 3x^3 + 6x^7
```

QUICK REVIEW

1. Pointer variables contain the addresses of other variables as their values.
2. In C++, no name is associated with the pointer data type.
3. A pointer variable is declared using an asterisk, `*`, between the data type and the variable.

4. In C++, `&` is called the address of operator.
5. The address of operator returns the address of its operand. For example, if `p` is a pointer variable of type `int` and `num` is an `int` variable, the statement

   ```
   p = &num;
   ```

   sets the value of `p` to the address of `num`.
6. When used as a unary operator, `*` is called the dereferencing operator.
7. The memory location indicated by the value of a pointer variable is accessed by using the dereferencing operator, `*`. For example, if `p` is a pointer variable of type `int`, the statement

   ```
   *p = 25;
   ```

   sets the value of the memory location indicated by the value of `p` to `25`.
8. You can use the member access operator arrow, `->`, to access the component of an object pointed to by a pointer.
9. Pointer variables are initialized using either `0` (the integer zero), `NULL`, or the address of a variable of the same type.
10. The only integer value that can be directly assigned to a pointer variable is `0`.
11. The only arithmetic operations allowed on pointer variables are increment (`++`), decrement (`--`), addition of an integer to a pointer variable, subtraction of an integer from a pointer variable, and subtraction of a pointer from another pointer.
12. Pointer arithmetic is different from ordinary arithmetic. When an integer is added to a pointer, the value added to the value of the pointer variable is the integer times the size of the object to which the pointer is pointing. Similarly, when an integer is subtracted from a pointer, the value subtracted from the value of the pointer variable is the integer times the size of the object to which the pointer is pointing.
13. Pointer variables can be compared using relational operators. (It makes sense to compare pointers of the same type.)
14. The value of one pointer variable can be assigned to another pointer variable of the same type.
15. A variable created during program execution is called a dynamic variable.
16. The operator `new` is used to create a dynamic variable.
17. The operator `delete` is used to deallocate the memory occupied by a dynamic variable.
18. In C++, both `new` and `delete` are reserved words.
19. The operator `new` has two forms: one to create a single dynamic variable, and another to create an array of dynamic variables.
20. If `p` is a pointer of type `int`, the statement

    ```
    p = new int;
    ```

    allocates storage of type `int` somewhere in memory and stores the address of the allocated storage in `p`.

21. The operator `delete` has two forms: one to deallocate the memory occupied by a single dynamic variable, and another to deallocate the memory occupied by an array of dynamic variables.
22. If `p` is a pointer of type `int`, the statement `delete p;` deallocates the memory to which `p` points.
23. The array name is a constant pointer. It always points to the same memory location, which is the location of the first array component.
24. To create a dynamic array, the form of the `new` operator that creates an array of dynamic variables is used. For example, if `p` is a pointer of type `int`, the statement

    ```
    p = new int[10];
    ```

    creates an array of 10 components of type `int`. The base address of the array is stored in `p`. We call `p` a dynamic array.
25. Array notation can be used to access the components of a dynamic array. For example, suppose `p` is a dynamic array of 10 components. Then `p[0]` refers to the first array component, `p[1]` refers to the second array component, and so on. In particular, `p[i]` refers to the `(i + 1)`th component of the array.
26. An array created during program execution is called a dynamic array.
27. If `p` is a dynamic array, then the statement

    ```
    delete [] p;
    ```

    deallocates the memory occupied by `p`—that is, the components of `p`.
28. In a shallow copy, two or more pointers of the same type point to the same memory space; that is, they point to the same data.
29. In a deep copy, two or more pointers of the same type have their own copies of the data.
30. If a class has a destructor, the destructor automatically executes whenever a class object goes out of scope.
31. If a class has pointer data members, the built-in assignment operators provide a shallow copy of the data.
32. A copy constructor executes when an object is declared and initialized by using the value of another object, and when an object is passed by value as a parameter.
33. C++ allows a user to pass an object of a derived class to a formal parameter of the base class type.
34. The binding of virtual functions occurs at execution time, not at compile time, and is called dynamic or run-time binding.
35. In C++, virtual functions are declared using the reserved word virtual.
36. A class is called an abstract class if it contains one or more pure virtual functions.

37. Because an abstract class is *not* a complete classas—it (or its implementation file) does not contain the definitions of certain functions as you cannot create objects of that class.
38. In addition to the pure virtual functions, an abstract class can contain instance variables, constructors, and functions that are not pure virtual. However, the abstract class must provide the definitions of constructors and functions that are not pure virtual.
39. A list is a collection of elements of the same type.
40. The commonly performed operations on a list are create the list, determine whether the list is empty, determine whether the list is full, find the size of the list, destroy or clear the list, determine whether an item is the same as a given list element, insert an item in the list at the specified location, remove an item from the list at the specified location, replace an item at the specified location with another item, retrieve an item from the list from the specified location, and search the list for a given item.

EXERCISES

1. Mark the following statements as true or false.
   a. In C++, `pointer` is a reserved word.
   b. In C++, pointer variables are declared using the reserved word `pointer`.
   c. The statement `delete p;` deallocates the variable pointer `p`.
   d. The statement `delete p;` deallocates the dynamic variable to which `p` points.
   e. Given the declaration

      ```
      int list[10];
      int *p;
      ```

      the statement

      ```
      p = list;
      ```

      is valid in C++.
   f. Given the declaration

      ```
      int *p;
      ```

      the statement

      ```
      p = new int[50];
      ```

      dynamically allocates an array of 50 components of type `int`, and `p` contains the base address of the array.
   g. The address of operator returns the address and value of its operand.
   h. If `p` is a pointer variable, the statement `p = p * 2;` is valid in C++.

2. Given the declaration

```
int x;
int *p;
int *q;
```

Mark the following statements as valid or invalid. If a statement is invalid, explain why.

a. `p = q;`

b. `*p = 56;`

c. `p = x;`

d. `*p = *q;`

e. `q = &x;`

f. `*p = q;`

3. What is the output of the following C++ code?

```
int x;
int y;
int *p = &x;
int *q = &y;
*p = 35;
*q = 98;
*p = *q;
cout << x << "  " << y << endl;
cout << *p << "  " << *q << endl;
```

4. What is the output of the following C++ code?

```
int x;
int y;
int *p = &x;
int *q = &y;
x = 35;  y = 46;
p = q;
*p = 78;
cout << x << "  " << y << endl;
cout << *p << "  " << *q << endl;
```

5. Given the declaration

```
int num = 6;
int *p = &num;
```

which of the following statement(s) increment the value of `num`?

a. `p++;`

b. `(*p)++;`

c. `num++`

d. `(*num)++;`

6. What is the output of the following code?

```
int *p;
int * q;
p = new int;
q = p;
*p = 46;
*q = 39;
cout << *p << " " << *q << endl;
```

7. What is the output of the following code?

```
int *p;
int *q;
p = new int;
*p = 43;
q = p;
*q = 52;
p = new int;
*p = 78;
q = new int;
*q = *p;
cout << *p << " " << *q << endl;
```

8. What is wrong with the following code?

```
int *p;                                  //Line 1
int *q;                                  //Line 2

p = new int;                             //Line 3
*p = 43;                                 //Line 4

q = p;                                   //Line 5
*q = 52;                                 //Line 6

delete q;                                //Line 7

cout << *p << " " << *q << endl;         //Line 8
```

9. What is the output of the following code?

```
int x;
int *p;
int *q;
p = new int[10] ;
q = p;
*p = 4;

for(int j = 0; j < 10; j++)
{
    x = *p;
    p++;
    *p = x + j;
}
```

```
for (int k = 0; k < 10; k++)
{
    cout << *q << " ";
    q++;
}
cout << endl;
```

10. What is the output of the following code?

```
int *secret;

secret = new int[10];
secret[0] = 10;
for (int j = 1; j < 10; j++)
    secret[j] = secret[j -1] + 5;
for(int j = 0; j < 10; j++)
    cout << secret[j] << " ";
cout << endl;
```

11. Explain the difference between a shallow copy and a deep copy of data.

12. What is wrong with the following code?

```
int *p;                          //Line 1
int *q;                          //Line 2

p = new int [5];                 //Line 3
*p = 2;                          //Line 4

for (int i = 1; i < 5; i++)      //Line 5
    p[i] = p[i-1] + i;           //Line 6

q = p;                           //Line 7

delete [] p;                     //Line 8

for (int j = 0; j < 5; j++)      //Line 9
    cout << q[j] << " ";         //Line 10

cout << endl;                    //Line 11
```

13. What is the output of the following code?

```
int *p;
int *q;

p = new int [5];
p[0] = 5;

for (int i = 1; i < 5; i++)
    p[i] = p[i - 1] + 2 * i;

cout << "Array p: ";
for (int i = 0; i < 5; i++)
    cout << p[i] << " ";
cout << endl;
```

```
q = new int[5];

for (int i = 0; i < 5; i++)
    q[i] = p[4 - i];

cout << "Array q: ";
for (int i = 0; i < 5; i++)
    cout << q[i] << " ";

cout << endl;
```

14. What is the output of the following code?

```
int **p;

p = new int* [5];

for (int i = 0; i < 5; i++)
    p[i] = new int[3];

for (int i = 1; i < 5; i++)
    for (int j = 0; j < 3; j++)
        p[i][j] = 2 * i + j;

for (int i = 1; i < 5; i++)
{
    for (int j = 0; j < 3; j++)
        cout << p[i][j] << " ";
    cout << endl;
}
```

15. What is the purpose of the copy constructor?
16. Name three situations when a copy constructor executes.
17. Name three things that you should do for classes with pointer data members.
18. Suppose that you have the following definition of a class.

```
class dummyClass
{
public:
    void print();
    ...
private:
    int listLength;
    int *list;
    double salary;
    string name;
}
```

    a. Write the function prototype to overload the assignment operator for the `class dummyClass`.
    b. Write the definition of the function to overload the assignment operator for the `class dummyClass`.

c. Write the function prototype to include the copy constructor for the `class dummyClass`.

d. Write the definition of the copy constructor for the `class dummyClass`.

19. Suppose that you have the following classes, `classA` and `classB`:

```
class classA
{
public:
    virtual void print() const;
    void doubleNum();
    classA(int a = 0);

private:
    int x;
};

void classA::print() const
{
    cout << "ClassA x: " << x << endl;
}

void classA::doubleNum()
{
    x = 2 * x;
}

classA::classA(int a)
{
    x = a;
}

class classB: public classA
{
public:
    void print() const;
    void doubleNum();
    classB(int a = 0, int b = 0);

private:
    int y;
};

void classB::print() const
{
    classA::print();
    cout << "ClassB y: " << y << endl;
}
```

```
void classB::doubleNum()
{
    classA::doubleNum();

    y = 2 * y;
}

classB::classB(int a, int b)
       : classA(a)
{
    y = b;
}
```

What is the output of the following function `main`?

```
int main()
{
    classA *ptrA;
    classA objectA(2);

    classB objectB(3, 5);

    ptrA = &objectA;
    ptrA->doubleNum();
    ptrA->print();
    cout << endl;

    ptrA = &objectB;

    ptrA->doubleNum();
    ptrA->print();
    cout << endl;

    return 0;
}
```

20. What is the output of the function `main` of Exercise 19, if the definition of `classA` is replaced by the following definition?

```
class classA
{
public:
    virtual void print() const;
    virtual void doubleNum();
    classA(int a = 0);

private:
    int x;
};
```

21. What is the difference between compile-time binding and run-time binding?

22. Consider the following definition of the `class student`.

```
public studentType: public personType
{
public:
    void print();
    void calculateGPA();
    void setID(long id);
    void setCourses(const string c[], int noOfC);
    void setGrades(const char cG[], int noOfC);

    void getID();
    void getCourses(string c[], int noOfC);
    void getGrades(char cG[], int noOfC);
    void studentType(string fName = "", string lastName = "",
                     long id, string c[] = NULL,
                     char cG[] = NULL, int noOfC = 0);

private:
    long studentId;
    string courses[6];
    char coursesGrade[6]
    int noOfCourses;
}
```

Rewrite the definition of the `class student` so that the functions `print` and `calculateGPA` are pure `virtual` functions.

23. What is the effect of the following statements?

    a. `arrayListType<int> intList(100);`

    b. `arrayListType<string> stringList(1000);`

    c. `arrayListType<double> salesList(-10);`

24. Draw the UML diagram of the `class polynomialType`. Also show the inheritance hierarchy.

PROGRAMMING EXERCISES

1. The function `removeAt` of the `class arrayListType` removes an element from the list by shifting the elements of the list. However, if the element to be removed is at the beginning of the list and the list is fairly large, it could take a lot of computer time. Because the list elements are in no particular order, you could simply remove the element by swapping the last element of the list with the item to be removed and reducing the length of the list. Rewrite the definition of the function `removeAt` using this technique.

2. The function `remove` of the `class arrayListType` removes only the first occurrence of an element. Add the function `removeAll` to the `class arrayListType` that would remove all occurrences of a given element. Also, write the definition of the function `removeAll` and a program to test this function.

3. Add the function `min` to the `class arrayListType` to return the smallest element of the list. Also, write the definition of the function `min` and a program to test this function.
4. Add the function `max` to the `class arrayListType` to return the largest element of the list. Also, write the definition of the function `max` and a program to test this function.
5. The operators `+` and `-` are overloaded as member functions for the `class polynomialType`. Redo Programming Example Polynomial Operations so that these operators are overloaded as nonmember functions. Also write a test program to test these operators.
6. Write the definition of the function to overload the operator `*` (as a member function) for the `class polynomialType` to multiply two polynomials. Also write a test program to test the operator `*`.
7. Let $p(x) = a_0 + a_1x + \ldots + a_{n-1}x^{n-1} + a_nx^n$ be a polynomial of degree n, where a_i are real (or complex) numbers and n is a nonnegative integer. The derivative of $p(x)$, written $p'(x)$, is defined to be $p'(x) = a_1 + 2a_2x^2 + \ldots + na_nx^{n-1}$. If $p(x)$ is constant, then $p'(x) = 0$. Overload the operator $\sim$ as a member function for the `class polynomialType` so that $\sim$ returns the derivative of a polynomial.
8. The `class polynomialType` as given in the Programming Example Polynomial Operations processes polynomials with coefficients that are real numbers. Design and implement a similar class that can be used to process polynomials with coefficients as complex numbers. Your class must overload the operators `+`, `-`, `*` to perform addition, subtraction, and multiplication; and the operator `()` to evaluate a polynomial at a given complex number. Also write a program to test various operations.
9. Using classes, design an online address book to keep track of the names, addresses, phone numbers, and dates of birth of family members, close friends, and certain business associates. Your program should be able to handle a maximum of 500 entries.

    a. Define a `class`, `addressType`, that can store a street address, city, state, and zip code. Use the appropriate functions to print and store the address. Also, use constructors to automatically initialize the data members.

    b. Define a `class extPersonType` using the `class personType` (as defined in Example 1-12, Chapter 1), the `class dateType` (as designed in Programming Exercise 2 of Chapter 2), and the `class addressType`. Add a data member to this class to classify the person as a family member, friend, or business associate. Also, add a data member to store the phone number. Add (or override) the functions to print and store the appropriate information. Use constructors to automatically initialize the data members.

    c. Derive the `class addressBookType` from the `class arrayListType`, as defined in this chapter, so that an object of type `addressBookType` can store

objects of type `extPersonType`. An object of type `addressBookType` should be able to process a maximum of 500 entries. Add necessary operations to the `class addressBookType` so that the program should perform the following operations:

i. Load the data into the address book from a disk.
ii. Search for a person by last name.
iii. Print the address, phone number, and date of birth (if it exists) of a given person.
iv. Print the names of the people whose birthdays are in a given month or between two given dates.
v. Print the names of all the people having the same status, such as family, friend, or business.
vi. Print the names of all the people between two last names.

10. (**Safe Arrays**) In C++, there is no check to determine whether the array index is out of bounds. During program execution, an out-of-bound array index can cause serious problems. Also, recall that in C++ the array index starts at 0.

    Design a `class safeArray` that solves the out-of-bound array index problem and allows the user to begin the array index starting at any integer, positive or negative. Every object of type `safeArray` should be an array of type `int`. During execution, when accessing an array component, if the index is out of bounds, the program must terminate with an appropriate error message. For example,

```
safeArray list(2,13);
safeArray yourList(-5,9);
```

    In this example, `list` is an array of 11 components, the component type is `int`, and the components are `list[2]`, `list[3]`, ..., `list[12]`. Also, `yourList` is an array of 15 components, the component type is `int`, and the components are `yourList[-5]`, `yourlist[-4]`, ..., `yourList[0]`, ..., `yourList[8]`.

11. Programming Exercise 10 processes only `int` arrays. Redesign the `class safeArray` using class templates so that the class can be used in any application that requires arrays to process data.

12. Design a class to perform various matrix operations. A matrix is a set of numbers arranged in rows and columns. Therefore, every element of a matrix has a row position and a column position. If A is a matrix of 5 rows and 6 columns, we say that matrix A is of the size 5×6 and sometimes denote it as $A_{5\times6}$. Clearly, a convenient place to store a matrix is in a two-dimensional array. Two matrices can be added and subtracted if they have the same size. Suppose that $A = [a_{ij}]$ and

$B = [b_{ij}]$ are two matrices of the size $m \times n$, where a_{ij} denotes the element of A in the ith row and the jth column, and so on. The sum and difference of A and B is given by

$$A + B = [a_{ij} + b_{ij}]; \qquad A - B = [a_{ij} - b_{ij}]$$

The multiplication of A and B (A * B) is defined only if the number of columns of A are the same as the number of rows of B. If A is of the size $m \times n$ and B is of the size $n \times t$, then $A * B = [c_{ik}]$ is of the size $m \times t$ and the element c_{ik} is given by the formula

$$c_{ik} = a_{i1}b_{1k} + a_{i2}b_{2k} + \ldots + a_{in}b_{nk}$$

Design and implement a `class matrixType` that can store a matrix of any size. Overload the operators `+`, `-`, and `*` to perform the addition, subtraction, and multiplication operations, respectively, and overload the operator `<<` to output a matrix. Also, write a test program to test various operations on matrices.

13. The `class largeIntegers` in Programming Exercise 16, in Chapter 2, is designed to process large integers of at most 100 digits. Using dynamic arrays, redesign this class so that integers of any digits can be added and/or subtracted. Also overload the multiplication operator to multiply large integers.

This page intentionally left blank